Training is a multi-stage pipeline, not a single objective: pre-training to alignment in frontier LLMs, 2017–2026

Theory KB — Paper 2 of 5

2026-05-04 (draft)

Abstract

Modern frontier-LLM training is a six-stage pipeline: pre-training data → mixed-precision and distributed scaffolding → supervised fine-tuning → preference-RL (RLHF/DPO/CAI) → RL on verifiable rewards (RLVR/GRPO) → adaptation and merging. Each stage's load-bearing variables — token count, mixture, precision, batch size, KL constraint, verifier signal, merge geometry — are now well- characterized. We argue the pipeline composes; pretending it is one objective produces persistently confused empirics.

Contents

1 Introduction

Training a frontier language model is not the optimization of a single objective. It is a pipeline of objectives, each with its own loss form, data distribution, optimizer, KL anchor, and stopping criterion. The pre-training cross-entropy that fits a base model on $\sim$15T tokens (DeepSeek-AI, 2024b, §3.2) is not the same kind of object as the classification loss on a curated $\sim 10^4$-pair preference set (Rafailov et al., 2023), which is not the same kind of object as the sparse $\{0, 1\}$ verifier signal that drives a 30K-sample GRPO run on math contest problems (Shao et al., 2024; DeepSeek-AI, 2025). They live on different data, optimize different functionals, anchor against different reference policies, and stop on different criteria. Treating "training" as one optimization collapses these distinctions and produces a class of empirics that has genuinely confused the field for half a decade: the Kaplan-vs-Chinchilla compute-allocation flip (Kaplan and et al. , OpenAI; Hoffmann and et al. , DeepMind), the DPO-vs-PPO-vs-GRPO debate, and the recurrent claim that "RLVR creates new reasoning capability" are all, on inspection, artefacts of mixing two stages' invariants. The thesis of this paper is that they disentangle once the pipeline is named:

1. **Pre-training data.** What goes into the corpus, how it is filtered and mixed, how many tokens per parameter (section 2).

2. **Scaling and optimization.** How loss falls with (N, D, C) and how to spend a fixed compute budget; what optimizer spends those FLOPs (section 3, section 4).

3. **Distributed and mixed-precision scaffolding.** How the training step is laid out across thousands of accelerators, and at what numerical precision (section 5, section 6).

4. **Supervised fine-tuning.** The first post-training stage: cross-entropy on curated prompt-response pairs, producing the instruction-following starting point π^{SFT} (section 7).

5. **Preference-RL.** RLHF, RLAIF / Constitutional AI, DPO and the offline-preference variant family: aligning the policy against pairwise preference data, with a KL anchor to π^{SFT} (section 8, section 9, section 10).

6. **Reasoning-RL and adaptation.** RL on verifiable rewards (RLVR / GRPO / DAPO) for math, code, and proof, plus the weight-space operations (LoRA / QLoRA / model soups / TIES) that compose adapt-and-merge late in the pipeline (section 11, section 12).

Each stage has its own canonical lineage, its own load-bearing variables, its own resolved questions, and its own live frontiers. The pipeline composes; the stages are not interchangeable; and the "confused empirics" resolve as soon as we identify which stage's invariant is being measured.

1.1 What the pipeline framing buys

Three persistent debates in the 2020–2026 literature are stage- boundary artefacts. We name them here because they motivate the rest of the paper.

Kaplan vs. Chinchilla. Kaplan and et al. (OpenAI) fitted three power laws on small-to-medium decoder-only LMs and recommended scaling N much faster than D at fixed compute — "train very large models, stop well short of convergence." Hoffmann and et al. (DeepMind) re-fit on a re-instrumented set of 400+ runs whose LR schedules matched their actual token horizons (Kaplan's cosine schedule decayed to a fixed horizon biased short-data runs upward), found $N \propto C^{0.5}$, $D \propto C^{0.5}$, and produced the famous $D \approx 20N$ rule along with the Chinchilla-70B-beats-Gopher-280B headline at matched FLOPs. The two papers' *functional forms* agree; the *compute-allocation prescription* flipped on the strength of one methodological correction. The 2024–2026 frontier sits an order of magnitude beyond Chinchilla on tokens-per-parameter (Llama-3 8B at $D/N \approx 1875$, two orders past the Chinchilla-optimal point), which is *not* a refutation of Chinchilla but a re-objective: pre-training cost has been displaced by inference cost as the dominant economic axis (DeepSeek-AI, 2024b, §3.2). section 3 treats this in full.

RLHF vs. DPO vs. PPO. Rafailov et al. (2023) read the RLHF closed-form RL optimum $\pi^*(y \mid x) \propto \pi^{\text{ref}}(y \mid x) \exp(r(x, y)/\beta)$ not as the *target* of a PPO loop (Ouyang et al., 2022) but as the *definition* of a reward that can be inverted — yielding a closed-form classification loss on the same preference data, without an explicit reward model, without on-policy sampling, without a value head. Same fixed point, two-orders-cheaper per step. The 2023 narrative "DPO replaces PPO" was overstated: at frontier scale the two coexist (Llama-3 and Tülu-3 ship DPO; DeepMind / Gemma-3 keep PPO with weight-averaged reward models), and Meng et al. (2024) reframes the DPO family as a small organized lattice with three knobs (loss shape, reference dependence, length normalization). section 8 and section 9 treat this in full.

"RLVR creates new capability." The DeepSeek-R1 (DeepSeek-AI, 2025) headline — pass@1 on AIME 2024 climbing from 15.6% to 77.9% over a single GRPO run, with the model emitting a `wait`-token "aha-moment" partway through — was widely read as RL conjuring reasoning capability out of a base LM. The 2025 follow-up literature, including a NeurIPS 2025 oral, substantially narrowed that claim: RLVR sharpens pass@1 on problems already solvable at some pass@K, but does not expand the set of problems solvable at any K; the capability ceiling is set by pre-training. RLVR is sampling-efficient sharpening, not capability expansion — a distinction that only comes into focus once the stages are named. section 11 treats this in full and section 14 returns to the live disagreement.

These three are the load-bearing examples. Each is a stage-boundary collision: a quantity measured at one stage, an invariant claimed at another. The pipeline framing isolates them.

Intuition

A useful mental picture: each stage is a distinct *KL-constrained reweighting* of the model's output distribution. Pre-training fits an unconstrained π^{base} to the corpus distribution. SFT performs a forward-KL projection of π^{base} onto a demonstration distribution. RLHF / DPO / RLAIF perform a reverse-KL reweighting of π^{SFT} toward higher reward, with the anchor $\beta\ \text{KL}(\pi_\theta \| \pi^{\text{SFT}})$ controlling how far the policy may drift. RLVR performs the same reverse-KL reweighting with the reward replaced by a verifier indicator. Returning to canonical form: the post-training stages all share the closed-form RL optimum $\pi^*(y|x) \propto \pi^{\text{ref}}(y|x) \exp(r(x, y)/\beta)$ (section 8); only r and π^{ref} change between RLHF, RLAIF, DPO, and RLVR. The pipeline is a *chain* of such reweightings; mixing two stages' references and rewards is what produces the confused empirics.

1.2 Why six stages, why now

The six-stage decomposition is not a logical necessity. It is the empirical shape of frontier-2026 recipes once we read across the disclosed stack: DeepSeek-V3 / R1, Llama 3 / 4, Qwen 2.5 / 3, Phi-4, Tülu 3, OLMo 2, Mistral Large, Gemma 3. Every disclosed pipeline since 2024 contains all six stages, in the same order, with the same inputs and outputs at each stage boundary. This convergence is recent: GPT-3 (Kaplan and et al. , OpenAI) shipped without stages 4–6; InstructGPT (Ouyang et al., 2022) added stages 4–5 (SFT and PPO-RLHF); the RLVR / GRPO / DAPO line (Shao et al., 2024; DeepSeek-AI, 2025; Seed, 2025) added stage 6's reasoning-RL leg; the merging line (Wortsman et al., 2022; Ilharco et al., 2022; Yadav et al., 2023) added stage 6's adaptation leg. The 2026 question — *is the six-stage decomposition canonical, or recipe-zoo bias from the same few labs reading each other's papers?* — is open, and we treat it explicitly in section 13 after the body of the paper has put each stage's load-bearing variables on the page.

1.3 Notation and scope

Paper 2 assumes Paper 1's residual-stream / FFN / attention notation. The Transformer block, with its alternating attention and feed-forward sub-layers around residual connections, is taken as given (Vaswani et al., 2017); the parameter count N, sequence length S, batch size B, model dimension D, head count H, head dimension d_h, vocabulary size V, and layer count L from Paper 1's preamble are reused without redefinition. New to this paper: D also denotes the training-token count when paired with N (the scaling-law context disambiguates), $C \approx 6ND$ is total non-embedding training compute in FLOPs (Kaplan and et al. , OpenAI, §2.1), and the post-training stages introduce π_θ for the trainable policy, π^{ref} for whichever reference is anchoring it, $r_\phi(x, y)$ or $R(x, y)$ for learned and verifiable rewards, and β for the KL coefficient.

We hand off a single forward thread to Paper 3: section 11's GRPO and DAPO machinery is the *training leg* of Paper 3's compute / search / verification triangle for reasoning. The *inference* side — chain-of-thought sampling, self-consistency, process reward models, MCTS-style tree search at generation time — is treated there. Paper 4 (interpretability) and Paper 5 (evaluation) reach back into this paper for, respectively, the training-time activations that probes are built on and the alignment surface that RLHF / CAI shapes.

What this paper deliberately does *not* cover. (i) Architectural choices — attention variants, MoE, normalization placement, positional encodings — live in Paper 1, even though MoE intersects training (section 5 on EP, section 6 on FP8 expert-collapse) and is cited from there. (ii) Inference-time techniques — speculative decoding, KV-cache compression, quantized serving — live in Paper 1's inference section. (iii) Evaluation methodology — contamination, benchmark validity, alignment-faking detection — lives in Paper 5; this paper takes *loss*, *pass@1*, and *preference win-rate* as legitimate signals and defers benchmark critique to Paper 5.

1.4 Roadmap

The body of this paper walks the six stages in order, then closes with a frontier snapshot and a contradictions chapter.

section 2 — Pre-training data. Curate → ablate → mix. The FineWeb pipeline end-to-end (96 CC snapshots, MinHash 14×8, custom quantile-derived filters), the Data Mixing Laws functional form, and the FineWeb-Edu distill-then-classify pattern. Token-budget choice ($D \approx 20N$ Chinchilla, $D/N \approx 1875$ Llama-3 over-train) sets up section 3.

section 3 — Scaling laws. Kaplan's three power laws and joint $L(N, D)$; Chinchilla's matched-LR-horizon re-fit yielding $N \propto C^{0.5}$, $D \propto C^{0.5}$; the post-Chinchilla industry move to over-train for inference economics; µ-Transfer as the HP-transfer technology that makes any of these recipes tunable at frontier scale.

section 4 — Optimization. AdamW as the production default; Lion and Sophia as training-loss-only speedups; Muon's Newton–Schulz orthogonalization with a verified $\sim 2\times$ compute-efficiency lift at 16B-MoE / 5.7T tokens; the schedule axis (cosine → Warmup–Stable–Decay → Schedule-Free).

section 5 — Distributed training. Five composable axes on a single `DeviceMesh`: $G = d{\cdot}t{\cdot}c{\cdot}p{\cdot}e$ for DP × TP × CP × PP × EP. Per-axis cost analysis, DeepSeek-V3's TP-free plan as case study, TorchTitan's 4D composability, Distributed Muon.

section 6 — Mixed precision and stability. Three independent precision choices (storage / compute / communication) on a master-FP32 backbone; the format zoo (FP32 / FP16 / BF16 / FP8 / NVFP4 / BitNet); DeepSeek-V3's fine-grained FP8 recipe with zero irrecoverable loss spikes over 14.8T tokens; loss-spike taxonomy and mitigations.

section 7 — Supervised fine-tuning. Cross-entropy with prompt-masking; the data-not-loss thesis; lineage from FLAN through InstructGPT to Tülu-3, MAGPIE, R1-Distill, and s1; multi-turn loss-mask conventions; why SFT precedes RL.

section 8 — RLHF. InstructGPT's three-stage pipeline; Bradley–Terry reward modelling; PPO with KL anchor and the closed-form RL optimum; the four-model PPO infrastructure; reward-model variants (ensembles, WARM/WARP).

section 9 — DPO and offline preference. The reparameterization that turns the RLHF optimum into a classification loss on the same preference data; the variant lattice (IPO, KTO, ORPO, SimPO, CPO) along three knobs.

section 10 — RLAIF and Constitutional AI. The same RLHF machinery with an AI labeler; CAI's two-stage pipeline (SL-CAI → RL-CAI); the constitution as the *only* human-authored alignment input; non-evasion behavior and its dual-use surface.

section 11 — RLVR and GRPO. Programmatic verifiers in place of learned reward models; group-mean baselines in place of co-sized value models; DeepSeek-R1's GRPO trajectory and the DAPO improvements (Clip-Higher, Dynamic Sampling, Token-Level Loss, Overlong Reward Shaping). Cross-paper anchor for Paper 3.

section 12 — Adaptation and merging. LoRA / QLoRA as low-rank parameter-efficient fine-tuning; Model Soups, Task Vectors, TIES-Merging as weight-space operations; the linear-mode-connectivity story as the speculative theoretical underpinning.

section 13 — Frontier-2026 snapshot. The six-stage table across the disclosed frontier (DeepSeek-V3 / R1, Llama 3 / 4, Qwen 2.5 / 3, Phi-4, Tülu 3, OLMo 2, Mistral Large, Gemma 3). Where labs agree, where they diverge, where they are opaque. Returns to the open question of whether the six-stage decomposition is canonical or recipe-zoo bias.

section 14 — Live contradictions. Concentrated treatment of the `[CONTRADICTION]` markers raised throughout the body: Kaplan-vs-Chinchilla, optimizer speedups, TP-yes-vs-TP-no, FP8

/ NVFP4 / BitNet production status, SFT data quantity-vs-quality, reward-hacking, DPO-vs-PPO, CAI internalization, RLVR capability, merge guarantees, precision × token-count, µ-P at frontier scale.

The closing thesis we will defend in section 14: the pipeline's open questions are *localized*. Most live at stage boundaries (precision × optimizer; token-count × quantization; preference × verifier) rather than at the pipeline's structure. The structure — pre-train → scale → optimize → distribute → precision → SFT → preference → reasoning → merge — is now stable enough to write down. The empirical envelope around each stage's variables is what 2027-class results will close.

2 Pre-training data

Pre-training data is no longer "the corpus" — a single dump of filtered web text against which architecture and optimizer choices are evaluated. It is a pipeline with its own engineering surface, its own ablation regime, and its own scaling law. The 2024 inflection point is FineWeb (et al. , HuggingFace), which decomposes the dataset into a chain of stages (web extraction, language and quality heuristics, deduplication, custom filters), each judged by training a 1.71B-parameter / ∼ 28B-token "ablation model" on the candidate output and comparing downstream task scores (et al. , HuggingFace, §3.1).[1] The mixture choice on top of those curated domains has its own functional form: et al. (2024) fit a closed-form relation $L_i(\mathbf{r}) = c_i + k_i \exp(\sum_j t_{ij} r_j)$ between mixture proportions and per-target-domain validation loss, predicting loss on unseen mixtures from a small number of fitted runs (et al., 2024, §1, eq. 1).[2] The synthetic-data turn closes the loop: Phi-4 (et al. , Microsoft) reports ∼ 400B synthetic tokens spanning 50 generation pipelines, 14B parameters surpassing GPT-4o on STEM evaluations (et al. , Microsoft, §1, Table 1), and explicitly argues that synthetic sequences are a *lower-noise* next-token-prediction signal than organic web text.

The thesis of this section is that the curate → ablate → mix loop has displaced "find more web text" as the active design surface. We walk the FineWeb pipeline end-to-end with concrete numbers, transcribe the Data Mixing Law equation and its coefficient interpretation, present FineWeb-Edu's distill-then-classify pattern, and set up two contradictions that resolve in section 14: the global-vs-individual-snapshot deduplication result that disproves "more dedup is better" (et al. , HuggingFace, §3.4) and the unresolved synthetic-fraction question raised by Phi-4's ∼80%-synthetic mix versus Tülu-3's organic-heavier post-training data (et al. , AI2). The section closes with the token-budget choice that section 3 immediately operates on: Chinchilla's $D \approx 20\,N$ versus Llama-3-8B's $D/N \approx 1875$ over-train, and the precision-aware tension with section 6 that the over-train objective creates.

2.1 The two design problems and their symbols

A pre-training dataset is a finite multiset $\mathcal{D} = \bigsqcup_{j=1}^{M} \mathcal{D}_j$ partitioned into M source domains (web crawl, code, math, books, scientific papers, etc.). The *mixture* is a probability vector $\mathbf{r} = (r_1, \ldots, r_M)$ on the simplex Δ^{M-1}, with $r_j \geq 0$ and $\sum_{j=1}^{M} r_j = 1$, where r_j is the fraction of the per-step token budget drawn from $\mathcal{D}_j$ (et al., 2024, §2). Two design problems sit on top of this object:

1. **Per-domain curation.** What counts as $\mathcal{D}_j$, how is it filtered, deduplicated, and quality-scored, what tokens make it into the multiset.

[1] KB excerpt: `kb/excerpts/fineweb2024.md#sec-3-1-proxy`.
[2] KB excerpt: `kb/excerpts/data-mixing-laws-2024.md#sec-1-functional`.

2. **Mixture choice.** Given curated domains, what $\mathbf{r}$ minimizes loss / maximizes downstream metric at the target (N, D) scale.

The two problems compose: curation determines what each $\mathcal{D}_j$ contains; mixing determines how the per-step token budget is partitioned across them. Table 1 fixes notation for the rest of the section.

Symbol	Type	Meaning
$\mathcal{D}$	multiset	full pre-training corpus
$\mathcal{D}_j$	multiset	source-j domain partition, $j \in \{1, \ldots, M\}$
M	$\mathbb{Z}_+$	number of source domains in the mixture
$\mathbf{r}$	Δ^{M-1}	mixture proportions, $r_j \geq 0$, $\sum_j r_j = 1$
$L_i(\mathbf{r})$	$\mathbb{R}$	validation loss on held-out target domain i
c_i, k_i	$\mathbb{R}$	per-target-domain irreducible / amplitude parameters
t_{ij}	$\mathbb{R}$	transfer coefficient from train-domain j to val-domain i
N	$\mathbb{Z}_+$	model parameters (section 3's N)
D	$\mathbb{Z}_+$	total training tokens
D/N	$\mathbb{R}_+$	tokens-per-parameter ratio (table 4)

Table 1: Symbols used in section 2. Mixture $\mathbf{r}$ lives on the simplex Δ^{M-1}; the Data Mixing Law eq. (1) fits per-target-domain $(c_i, k_i, t_{i,1\ldots M})$ from a small set of pilot runs, then predicts $L_i(\mathbf{r})$ on unseen mixtures.

2.2 The FineWeb pipeline, end to end

et al. (HuggingFace) document the design choices behind a 15T-token English web corpus derived from 96 Common Crawl snapshots (2013-summer through 2024-04) (et al. , HuggingFace, §3.4).[3] Each pipeline choice is judged by an *ablation model* of 1.71B parameters trained on $\sim$ 28B tokens (roughly the Chinchilla-optimal $D \approx 20\,N$ point for that N) under fixed architecture, optimizer, batch size, and seed — the only thing that varies is the data version (et al. , HuggingFace, §3.1).[4] "We trained two models for each dataset version, each using a different but equal-sized random subset of the full data and a different initialization seed, and then compared average scores" (et al. , HuggingFace, §3.1). The downstream-task score *slope* under this small-scale protocol is the only signal the pipeline trusts.

Stage 1 — text extraction (trafilatura over WET). Common Crawl ships two formats: raw WARC and pre-extracted WET. et al. (HuggingFace, §3.2)[5] report that "WET files retained too much boilerplate and menu text," and that re-extracting from WARCs with the open-source `trafilatura` library yields $\sim$ 25% more text and a measurable ablation-model gain. Custom text extraction is computationally expensive; the ablation lift justifies the cost.

Stage 2 — base filter. The base-filter pipeline reuses RefinedWeb's setup (et al. , HuggingFace, §3.3):[6] URL block-list for adult content, a fastText language classifier keeping only English with score ≥ 0.65, and the quality / repetition filters from MassiveText at original thresholds. Applied

[3] KB excerpt: `kb/excerpts/fineweb2024.md#sec-3-4-minhash`.
[4] KB excerpt: `kb/excerpts/fineweb2024.md#sec-3-1-proxy`.
[5] KB excerpt: `kb/excerpts/fineweb2024.md#sec-3-2-text`.
[6] KB excerpt: `kb/excerpts/fineweb2024.md#sec-3-3-base`.

across the 96 snapshots, this leaves "roughly 36 trillion tokens of data when tokenized with the GPT-2 tokenizer" (et al. , HuggingFace, §3.3). The 0.65 language-ID threshold is a load-bearing decision: pushing higher cuts low-confidence multilingual content (which often *is* good English with code-switched proper nouns); pushing lower admits genuinely off-language documents. At 0.65 the ablation models lose nothing detectable on standard English-language benchmarks.

Stage 3 — MinHash deduplication, individually per snapshot. et al. (HuggingFace, §3.4)[7] compute MinHash signatures from each document's 5-grams using 112 hash functions arranged as 14 buckets of 8 hashes each. Two documents are flagged as duplicates if they collide in any of the 14 buckets on all 8 hashes simultaneously. The Jaccard threshold this scheme targets is given by the standard MinHash analysis: a pair of documents with Jaccard similarity J collides in a single bucket of 8 hashes with probability J^8, and at least one of 14 buckets agrees with probability

$$P(\text{flagged duplicate} \mid J) \;=\; 1 - (1 - J^8)^{14}, \tag{1}$$

which crosses 0.5 at $J \approx 0.75$ (et al. , HuggingFace, §3.4), hence "targeting documents that are at least 75% similar." Flagged pairs are clustered transitively (if $A \sim B$ and $B \sim C$, then A, B, C form one cluster), and one document per cluster survives.

Intuition

MinHash trades exactness for parallelism: instead of computing $O(|\mathcal{D}|^2)$ pairwise Jaccard similarities, each document is hashed into 14 small buckets (8 hashes each) and only intra-bucket collisions are checked. The bucket-based collision rule eq. (1) implements a cheap probabilistic threshold on the true Jaccard. Returning to canonical form: with 14×8 hashes, pairs at $J = 0.9$ collide $> 99\%$ of the time, pairs at $J = 0.5$ collide $\sim 5\%$ of the time, and the curve crosses 0.5 at $J \approx 0.75$. The 14×8 choice is the *precision/recall* dial on this curve — more buckets sharpen recall at fixed Jaccard, more hashes per bucket sharpen precision.

The global-vs-individual deduplication finding. The naive prior holds that more aggressive dedup is monotonically better. et al. (HuggingFace, §3.4)[8] directly disprove it. Their first attempt was global MinHash dedup across all 96 snapshots simultaneously. "When applied to the oldest snapshots, this process removed as much as 90% of the original base filtered data," and the ablation models trained on the survivors were *worse*, not better, than models trained on individually-deduplicated snapshots. The interpretation: documents that appear in multiple snapshots are over-represented because they are useful — they are linked, mirrored, archived, and reposted. Global dedup removes these, leaving a residue dominated by content that appeared only *once*, which on average is lower-quality (ad copy, SEO spam, transient pages). FineWeb's final pipeline therefore deduplicates each snapshot independently, leaving cross-snapshot duplicates intact, producing ~ 20T tokens before custom filters (et al. , HuggingFace, §3.4).

Contradiction

"More deduplication is monotonically better" is empirically false on FineWeb-scale data (et al. , HuggingFace, §3.4). Global MinHash across 96 Common Crawl snapshots makes the trained model *worse* than per-snapshot MinHash, because what is duplicated-across-snapshots

[7] KB excerpt: `kb/excerpts/fineweb2024.md#sec-3-4-minhash`.
[8] KB excerpt: `kb/excerpts/fineweb2024.md#sec-3-4-global`.

is signal (linkage, archival, mirroring) and what is unique-per-snapshot tilts toward noise. The result is single-source on FineWeb's specific corpus and ablation protocol; whether it generalizes to non-web corpora (curated books, code) or to other dedup metrics (suffix-array exact, semantic-embedding fuzzy) is open. We return to this in section 14 alongside the section 3 over-training contradiction; both turn on the question of what "more" means once a pipeline is composed.

Stage 4 — C4 filters minus the curly-brace rule. et al. (HuggingFace, §3.7) apply "a selection of C4 filters" on top of the deduplicated text. The C4 paper's terminal-punctuation and short-line rules largely transfer; FineWeb *omits* C4's "curly-brace" filter, which would remove most code-bearing documents and which their ablation showed to hurt downstream benchmarks (presumably because code documents help English benchmarks via cross-domain transfer (et al., 2024, §3.2, t_{ij})). The decision is local — one ablation comparison — but illustrates the protocol: every C4 filter was tested individually, and only those whose removal hurt downstream were retained.

Stage 5 — custom filters from histogram inspection. The largest single filter-design lift comes from et al. (HuggingFace, §3.6).[9] They collect 50 document-level statistics (line length distribution, terminal punctuation rate, character-level repetition, word-level repetition, ...) on a high-quality corpus (individually deduplicated 2013-48 snapshot) and a low-quality corpus (the same snapshot *globally* deduplicated, the residue from the previous finding). For each statistic, they identify regions of the histogram where the low-quality corpus has higher density than the high-quality corpus, and convert those regions into filter thresholds. The three filters that survived ablation are

$$\text{filter A:} \quad \frac{\#\{\text{lines ending with terminal punctuation}\}}{\#\{\text{lines}\}} \leq 0.12,$$

$$\text{filter B:} \quad \frac{\#\{\text{characters in duplicated lines}\}}{\#\{\text{characters}\}} \geq 0.10,$$

$$\text{filter C:} \quad \frac{\#\{\text{lines} \leq 30 \text{ chars}\}}{\#\{\text{lines}\}} \geq 0.67.$$

Documents satisfying *any* of A, B, or C are removed. Together the three remove $\sim 22\%$ of remaining tokens and lift the aggregate ablation score by ~ 1 percentage point at the 28B- token scale (et al. , HuggingFace, §3.6). The 0.12 terminal-punctuation threshold is notably looser than C4's original 0.30 (which removed $\sim 30\%$ of tokens vs. FineWeb's 10.14%); the gentler cut preserves more code, technical text, and structured content while still removing the worst SEO-spam densities.

Final pipeline: 15T tokens. Combining the choices — WARC + trafilatura, base filter, per-snapshot MinHash, selected C4 filters, custom filters A–C — yields the 15T-token FineWeb release (et al. , HuggingFace, §3.7). This number is the empirical asymptote that several frontier labs have independently converged on: Llama 3 trained on $\sim$ 15T tokens (AI, 2024), DeepSeek-V3 on 14.8T (DeepSeek-AI, 2024b, abstract).[10]

[9] KB excerpt: `kb/excerpts/fineweb2024.md#sec-3-6-custom`.
[10] KB excerpt: `kb/excerpts/deepseek-v3-training.md#abstract`.

Intuition

The 1.71B / 28B-token ablation model is too small to score meaningfully against any production benchmark in absolute terms — its MMLU accuracy is below random-with-formatting-noise. The design relies instead on the *signed direction* of $\Delta L(\text{candidate}_A, \text{candidate}_B)$ being stable across scale: if a filter helps the ablation by +0.5pp on HellaSwag, it should help the production model on the same task. This holds for most heuristic decisions (et al. , HuggingFace, §3.1). The global-vs-individual dedup case in section 2.2 is interesting precisely because the ablation pointed toward the counterintuitive answer and the counterintuitive answer kept holding at production scale. Returning to canonical form: the protocol bets on the transfer relation $\text{sgn}(\Delta L(A, B; N{=}1.71\text{B}, D{=}28\text{B})) = \text{sgn}(\Delta L(A, B; N{=}\text{prod}, D{=}\text{prod}))$ holding component-wise across the pipeline's design choices.

2.3 Data Mixing Laws — the functional form on r

Curation determines what each $\mathcal{D}_j$ contains; mixing determines how training-step tokens partition across them. et al. (2024) establish that, under fixed model size and training-token budget, the validation loss on held-out target domain i is a closed-form function of the training-mixture proportions. Verbatim from et al. (2024, §1, eq. 1; §3.2, eq. 7)[11]:

$$L_i(\mathbf{r}_{1\ldots M}) \;=\; c_i \;+\; k_i \exp\left(\sum_{j=1}^{M} t_{ij}\, r_j\right), \tag{1}$$

where L_i is validation loss on domain i, $\mathbf{r} \in \Delta^{M-1}$ is the training mixture, and $(c_i, k_i, t_{i,1\ldots M})$ are scalar parameters fit per-target-domain. The form is the unique exponential-of-linear shape compatible with two principles (et al., 2024, §3.2):[12]

- *Compatibility*: at $M = 2$, eq. (1) reduces to the two-domain log-linear law $L_i(r) = c_i + k_i \exp(t_{ii} r)$ empirically verified on 70M and 160M models trained on GitHub + Pile-CC at 30B tokens (et al., 2024, §3.1, eq. 6).[13]
- *Symmetry*: any permutation of training-domain indices should leave the functional form invariant; eq. (1) is symmetric under permutations of $\{r_j, t_{ij}\}$ jointly.

Coefficient interpretation. et al. (2024, §3.2)[14] read off the meaning of each fitted parameter:

- c_i — "losses that are not reducible by adjusting the data mixture," i.e., the irreducible loss for target domain i. No choice of $\mathbf{r}$ on Δ^{M-1} can drop L_i below c_i.
- k_i — amplitude scaling the exponential term. At $\sum_j t_{ij} r_j = 0$ (a flat-zero-sensitivity mixture), $L_i = c_i + k_i$.
- t_{ij} — "the interaction" between training domain j and validation domain i. *Negative* t_{ij} means training on more j *lowers* validation loss on i; *positive* t_{ij} means training on j *hurts* i.

[11] KB excerpt: `kb/excerpts/data-mixing-laws-2024.md#sec-1-functional`.
[12] KB excerpt: `kb/excerpts/data-mixing-laws-2024.md#sec-3-2-multi`.
[13] KB excerpt: `kb/excerpts/data-mixing-laws-2024.md#sec-3-1-pilot`.
[14] KB excerpt: `kb/excerpts/data-mixing-laws-2024.md#sec-3-2-coefficients`.

The off-diagonal t_{ij} is the load-bearing object: it upgrades the folklore that "more code helps math" from a heuristic to a *measurable per-pair coefficient*. A fitted $t_{\text{math}\leftarrow\text{code}} < 0$ predicts how much adding code tokens to the mix lowers held-out math validation loss, and the prediction can be tested.

Nested scaling laws — making the fit affordable. Fitting eq. (1) directly at the target (N, D) requires many full-scale runs and is unaffordable at frontier scale. et al. (2024, §1, §2) stack a power-law of training-step ($L = c + k\,S^{\alpha}$) inside a power-law of model-size, inside the mixing law eq. (1), so that small-model short-step experiments at varied $\mathbf{r}$ predict large-model end-of-training $L_i(\mathbf{r})$. The headline empirical demonstration: a 1B-parameter model trained for 100B tokens on RedPajama, with mixture chosen by the nested fit, reaches loss matching the same model trained for 48% *more steps* on RedPajama's default mixture (et al., 2024, abstract).[15] Equivalently, the mixing-law-tuned recipe matches the 148B-token-budget loss at 100B tokens spent — a 1.48× data efficiency at fixed target loss.

Analogy

Equation (1) is a response-surface model on the simplex Δ^{M-1}: the per-target-domain loss is a fitted exponential in $\mathbf{r}$, with the fit's coefficients carrying physical meaning (c_i as floor, t_{ij} as transfer affinity). The pipeline that uses it is straight design-of-experiments: pilot runs at a diverse set of $\mathbf{r}$ values, fit $(c_i, k_i, t_{i,1\ldots M})$ per held-out i, then optimize the predicted L_i at the target (N, D) via the nested scaling-law extrapolation. Returning to canonical form: the design problem is $\mathbf{r}^* = \arg\min_{\mathbf{r}\in\Delta^{M-1}} \sum_i w_i\, \hat{L}_i(\mathbf{r})$ for target weights w_i on each held-out domain (et al., 2024, §2, eq. 5), with $\hat{L}_i$ the nested-scaling-law extrapolation of eq. (1). This is data-mixture engineering as a constrained convex program, not a heuristic.

2.4 Quality classifiers — FineWeb-Edu's distill-then-classify

After heuristic filtering produces a 15T-token web corpus, a *quality classifier* can extract a higher-density subset. et al. (HuggingFace, §4)[16] demonstrate the pattern at scale, producing FineWeb-Edu (1.3T tokens) from the full 15T base. The pipeline is four stages:

1. **Annotate.** Sample 460,000 random documents from FineWeb's CC-MAIN-2024-10 snapshot; prompt Llama-3-70B-Instruct to score each on a 0–5 "educational quality" rubric.
2. **Train.** Fit a small linear regressor on `Snowflake-arctic-embed-m` embeddings of the documents, targeting the Llama-3-70B scores. 20 epochs, learning rate 3×10^{-4}, embedding/encoder layers frozen (et al. , HuggingFace, §4). F1 = 82% on validation at score threshold ≥ 3.
3. **Score.** Run the regressor over the full 15T-token FineWeb. Cost: $\sim$ 6,000 H100 GPU hours.
4. **Threshold.** Keep documents with regressor score ≥ 3, yielding $\sim$ 1.3T tokens.

The result outperforms FineWeb on knowledge- and reasoning-intensive benchmarks (MMLU, ARC, OpenBookQA) at matched training-token-for-training-token comparison (et al. , HuggingFace, abstract). The pattern's economic structure is *amortized teacher cost*: the $\sim$ 6K-H100-hour annotation step is paid once at corpus-build time and recovered every step of every downstream

[15] KB excerpt: `kb/excerpts/data-mixing-laws-2024.md#abstract`.
[16] KB excerpt: `kb/excerpts/fineweb2024.md#sec-4-edu`.

pre-training run that uses the filtered corpus. This is the path that several frontier labs (Llama 3, Phi-3) used internally before FineWeb-Edu published it.

deepen-cite: nemotron-cc-2024 §abstract: NVIDIA's 6.3T-token Nemotron-CC corpus uses an analogous classifier-filtering plus rephrasing pipeline; cited as the closest published parallel to FineWeb-Edu's distill-then-classify recipe.

The KB note flags FineWeb-Edu, DCLM-Baseline, and Nemotron-CC as a coherent 2024-class lineage of classifier-driven distillation (et al. , HuggingFace, §1).[17]

2.5 Synthetic-data-as-bulk — the Phi-4 inversion

FineWeb-Edu uses a teacher LLM as a *labeler*: the teacher's judgment is amortized into a frozen filter. Phi-4 (et al. , Microsoft) inverts the role: the teacher LLM becomes the *generator* of a substantial fraction of the pre-training corpus. The Phi-4 abstract (et al. , Microsoft, abstract)[18] states the recipe explicitly:

> phi-4 [...] strategically incorporates synthetic data throughout the training process. While previous models in the Phi family largely *distill* the capabilities of a teacher model (specifically GPT-4), phi-4 substantially *surpasses* its teacher model on STEM-focused QA capabilities, giving evidence that our data-generation and post-training techniques go beyond distillation.

Phi-4's pre-training and mid-training corpus contains $\sim$ 400B synthetic tokens generated through "50 broad types of synthetic datasets, each one relying on a different set of seeds and different multi-stage prompting procedure" (et al. , Microsoft, §2.2).[19] Three pipeline primitives recur:

1. *Seed curation.* Web/code/book passages filtered for educational and reasoning value, then segmented and re-scored. Question banks are filtered by plurality voting — discard questions where teacher samples all agree (too easy) or entirely disagree (too hard / ambiguous), keeping the middle band (et al. , Microsoft, §2.2).[20]
2. *Rewrite and augment.* Seeds transformed into exercises, dialogues, and structured reasoning tasks via multi-step prompting.
3. *Self-revision.* A model critiques and improves its own output under reasoning / factual rubrics (et al. , Microsoft, §2.2).[21]

The headline empirical claim: a 14B-parameter Phi-4 surpasses its teacher GPT-4o on the GPQA (56.1 vs. 50.6) and MATH (80.4 vs. 74.6) benchmarks (et al. , Microsoft, §1, Table 1).[22]

The variance-of-next-token-loss intuition. et al. (Microsoft, §2.1)[23] argue that synthetic data is *lower-noise* as a next-token-prediction signal than organic web text:

[17] KB note: `kb/notes/training/pre-training-data.md#variants`.
[18] KB excerpt: `kb/excerpts/phi4.md#abstract`.
[19] KB excerpt: `kb/excerpts/phi4.md#sec-2-2-pipelines`.
[20] KB excerpt: `kb/excerpts/phi4.md#sec-2-2-seeds`.
[21] KB excerpt: `kb/excerpts/phi4.md#sec-2-2-rewrite`.
[22] KB excerpt: `kb/excerpts/phi4.md#sec-1-stem`.
[23] KB excerpt: `kb/excerpts/phi4.md#sec-2-1-purpose`.

In organic datasets, the relationship between tokens is often complex and indirect. Many reasoning steps may be required to connect the current token to the next, making it challenging for the model to learn effectively from next-token prediction. By contrast, each token generated by a language model is by definition predicted by the preceding tokens, making it easier for a model to follow the resulting reasoning patterns. In this way, synthetic data may act as a form of "spoonfeeding," presenting challenges in a digestible and progression-oriented manner.

Intuition

The argument is a claim about the conditional variance of the next-token loss. Let $\ell_t = -\log \pi_\theta(x_t \mid x_{<t})$ be the per-token loss; on organic text, ℓ_t has high variance because the local prefix may not contain enough information to predict x_t even at the data-generating distribution (long-range dependencies, references to context not present in the window, implicit world knowledge). On synthetic text generated by a related language model under autoregressive sampling, x_t is by construction predictable from $x_{<t}$ at the teacher's distribution, so the entropy $H(x_t \mid x_{<t})$ at the data-generating distribution is lower. Returning to canonical form: under the SFT-style decomposition of section 3's parametric loss, this corresponds to a smaller $E + B/D^\beta$ tail at fixed D when D is sourced from π_{teacher} rather than from organic web text — the *irreducible-plus-finite-data* terms shrink because the generating distribution is closer to a tractable one. The "spoonfeeding" analogy is exactly this: same number of training tokens, lower per-token loss variance, faster effective learning.

The synthetic-fraction question. Phi-4 chose roughly 80% synthetic tokens in its pre-training mix for a 14B reasoning-focused model (et al. , Microsoft, §1, §2.2).[24] At the post-training surface, Tülu-3 (et al. , AI2) ships an organic-heavier mix on Llama-3 base, with synthetic generation used selectively rather than as bulk. No published large-scale controlled ablation isolates the optimal synthetic fraction across model scale, domain, and training-stage (pre-train vs. mid-train vs. SFT).

Contradiction

The synthetic-fraction question splits the field. Phi-4 (et al. , Microsoft, abstract) argues for synthetic-as-bulk (∼80% of pre-training tokens) on the variance-of-next-token-loss grounds in section 2.5. Tülu-3 (et al. , AI2) ships an organic-heavier post-training mix without claiming the Phi-4 ratio generalizes. Recursive-synthetic concerns from the broader literature (et al. , Microsoft, §1) warn that training on a model's own outputs can degrade tail distributions (model-collapse).

deepen-cite: shumailov2024-collapse §2: recursive synthetic-only training accumulates loss in the data distribution's tails; cited as the principal counter-evidence to synthetic-as-bulk.

The synthetic-fraction at frontier scale is one of the open questions section 14 returns to: there is no controlled 14B-vs-70B Phi-style ablation, no synthetic-pipeline-mixture analogue of eq. (1), and no published study isolating recursive-synthetic-mode-collapse from the quality-of-teacher-pipeline gain.

[24] KB note: `kb/notes/training/synthetic-data-and-distillation.md#sec-1-1`.

2.6 Token budget — D/N as the bridge to scaling

Curation gives us $\mathcal{D}$; mixing gives us $\mathbf{r}$; neither answers *how many tokens to train on.* That choice belongs to section 3, but section 2 must specify the input it provides.

Chinchilla's $D \approx 20\,N$. Hoffmann and et al. (DeepMind, §4.1, Table 4) fit the compute-optimal frontier and report $D/N \approx 20$ at the Gopher-scale FLOP budget, with a slow drift upward at larger scale ($D/N \approx 21.2$ at 1T parameters) (Hoffmann and et al. , DeepMind, §3, Table 3). The section 3 parametric form $\hat{L}(N, D) = E + A/N^{\alpha} + B/D^{\beta}$ has its constrained minimum at $D \approx 20\,N$ for the fitted exponents.

The over-training inversion. The post-Chinchilla industry move is to substantially over-train small dense models for inference economics. Llama-3 8B trained on $\sim$15T tokens (AI, 2024) sits at $D/N \approx 1875$ — roughly 94× over Chinchilla's recipe. The reasoning is that the cost a deployed-model lab actually pays is $C_{\text{train}} + N_{\text{queries}} \cdot C_{\text{inf-per-query}}$, and when the second term dominates, the optimum shifts toward smaller N with proportionally more D (table 4). Crucially, the parametric loss $\hat{L}(N, D)$ keeps decreasing in D at fixed N — just suboptimally on the pre-training-FLOP margin. Over-training is not a refutation of Chinchilla; it solves a different objective.

Forward link. Two threads of this section feed forward. Section 3 takes D and N as the load-bearing variables of the $L(N, D)$ functional form: the FineWeb $\sim$ 15T-token web ceiling and the Phi-4 $\sim$ 400B synthetic supplement set the *numerator* of the over-training D/N ratio. Section 6 takes D as input to the precision-aware scaling-law tension: et al. (Harvard / Google) predict that over-trained models (large D/N) are *harder to quantize* at deployment time, partially offsetting the inference-cost gain that motivates the over-training in the first place (et al. , Harvard / Google, §4).

deepen-cite: scaling-laws-precision-2024 §4: effective-parameter form $N_{\text{eff}}(N, b_{\text{train}})$ and the over-training → harder-to-quantize prediction; forward-referenced from section 3.6.

The data pipeline's choice of D thus has a downstream cost that the $\hat{L}(N, D)$ formula does not see.

2.7 Variants and lineage

Table 2 summarizes the open-corpus lineage from C4 through the 2024-class pipelines.

2.8 Forward link

Pre-training data is now a pipeline with three load-bearing surfaces: curation, mixing, and synthetic-fraction. Curation is empirical ablation: et al. (HuggingFace)'s 1.71B / 28B-token protocol selects each filter on its sign-of-effect on downstream evaluation, producing the 15T-token FineWeb release through a chain of WARC-trafilatura extraction, 0.65 language-ID filtering, 14×8 MinHash deduplication *per snapshot* (not globally, section 2.2), three quantile-derived custom filters, and an optional Llama-3-70B distill-then-classify pass. Mixing is a fittable functional form: eq. (1)'s exponential-of-linear in $\mathbf{r}$, with per-target-domain (c_i, k_i, t_{ij}) recoverable from a small set of pilot runs and extrapolated to the target (N, D) via nested scaling laws (et al., 2024). Synthetic-fraction is the open contradiction (section 2.5): Phi-4's $\sim$80% for 14B reasoning vs. Tülu-3's organic-heavier post-training mix, with no controlled frontier-scale ablation.

The next section, section 3, takes the token-count D and parameter-count N this section produces and asks how cross-entropy loss scales in (N, D, C), why Kaplan's compute-allocation rule

Corpus	Tokens	Year	Distinctive choice
C4	0.75T	2020	first open web pipeline; line-level rules
The Pile	0.825T	2020	22 hand-picked sources; manual mixture
RefinedWeb (Falcon)	5T	2023	web-only; aggressive global dedup
RedPajama-V1 / V2	1.2T / 30T	2023/24	open Llama-1 reproduction; V2 returns CC at scale
Dolma (OLMo)	3T	2024	7 open sources
FineWeb	**15T**	**2024**	**ablation-model-driven design** (et al. , HuggingFace)
FineWeb-Edu	**1.3T**	**2024**	**distill-then-classify** (et al. , HuggingFace)
DCLM-Baseline	4T	2024	fastText-on-instruction-data classifier-filtering
Nemotron-CC	6.3T	2024	rephrased + classifier-filtered web (NVIDIA)
DeepSeek-V3 (proprietary)	14.8T	2024	DeepSeek-AI (2024b) reports diverse + high-quality
Llama-3 (proprietary)	$\sim$ 15T	2024	AI (2024); tokenizer 128K
Phi-4 (synthetic-bulk)	$\sim$ 400B syn	2024	50 pipelines (et al. , Microsoft)

Table 2: Open-corpus lineage and selected proprietary contemporaries. Three trends: (*i*) total tokens grew $\sim 20\times$ in four years; (*ii*) the curation interface shifted from hand-picked sources (Pile) to ablation-driven heuristics (FineWeb) to classifier-driven distillation (FineWeb-Edu, DCLM, Nemotron-CC); (*iii*) multiple labs now report $\sim$ 15T as the diminishing-returns ceiling on high-quality English web text (DeepSeek-AI, 2024b, §4).[25]

was overturned by Chinchilla's matched LR-horizon re-fit, and why the frontier-2024 industry sits an order of magnitude past Chinchilla on D/N for inference economics. Section 6 then takes the same D and asks how the precision $\times$ token-count interaction (et al. , Harvard / Google, §4) pulls against over-training: the larger D/N a deployed model has consumed, the harder it is to quantize at deployment time. The pipeline of this section is the input to both.

3 Scaling laws

A scaling law is a fitted functional form $L(N, D, C)$ relating cross-entropy loss to parameter count, training tokens, and training compute, together with a closed-form recipe for how to spend a fixed C across N and D. The fitted form is empirical; the recipe is prescriptive; the two are routinely conflated, and most of the "Kaplan-vs-Chinchilla" confusion in 2022–2024 was conflation. The thesis of this section is that the *functional forms* from Kaplan and et al. (OpenAI) and Hoffmann and et al. (DeepMind) are essentially agreed; the *compute-allocation prescription* flipped from "train very large models, stop well short of convergence" (Kaplan and et al. , OpenAI, §1.1) to "scale parameters and tokens equally" (Hoffmann and et al. , DeepMind, §3.4) on the strength of one methodological correction (matching the LR-schedule horizon to the actual training-token count); and the *frontier-2026 practice* sits an order of magnitude beyond Chinchilla on tokens-per-parameter because inference cost has displaced pre-training cost as the dominant objective (DeepSeek-AI, 2024b, §3.2). We then layer on µ-Transfer (Yang and et al. , Microsoft) as the hyperparameter-transfer technology that makes any of these recipes tunable at frontier scale, and the precision-aware extension of et al. (Harvard / Google) as the open precision $\times$ data trade-off that the over-training strategy collides with.

The compute approximation we will use throughout is the Kaplan and et al. (OpenAI, §2.1, eq. 2.1)[26] per-token forward FLOP count $C_{\text{forward}} \approx 2N$, with the backward pass roughly twice the

[26]KB excerpt: `kb/excerpts/kaplan2020.md#sec-2-1-definitions`.

forward, summing to

$$C \approx 6\,N\,D \quad \text{FLOPs} \tag{2}$$

total non-embedding training compute, valid when $d_{\text{model}} \gtrsim n_{\text{ctx}}/12$ (Kaplan and et al. , OpenAI, §2.1). Every scaling result in this section is stated against eq. (2).

3.1 Kaplan power laws

Kaplan and et al. (OpenAI) fitted decoder-only Transformer LMs on WebText2 across $N \in [768,\ 1.5 \times 10^9]$ non-embedding parameters and $D \in$ [22M, 23B] tokens, GPT-2 BPE tokenizer, context length 1024, Adam (Adafactor at $N > 1$B), and a cosine LR schedule with 3000-step warmup decaying to zero (Kaplan and et al. , OpenAI, §2.2). The headline finding is that cross-entropy loss obeys a *power law* in each of $N, D, C_{\min}$ when the other two are not the bottleneck.

The three single-axis laws. Verbatim from Kaplan and et al. (OpenAI, §1.2, eqs. 1.1–1.3)[27]:

$$L(N) = \left(\frac{N_c}{N}\right)^{\alpha_N}, \qquad \alpha_N \approx 0.076,\ \ N_c \approx 8.8 \times 10^{13}, \tag{1.1}$$

$$L(D) = \left(\frac{D_c}{D}\right)^{\alpha_D}, \qquad \alpha_D \approx 0.095,\ \ D_c \approx 5.4 \times 10^{13}, \tag{1.2}$$

$$L(C_{\min}) = \left(\frac{C_c^{\min}}{C_{\min}}\right)^{\alpha_C^{\min}}, \qquad \alpha_C^{\min} \approx 0.050,\ \ C_c^{\min} \approx 3.1 \times 10^8\,\text{PF-days}. \tag{1.3}$$

Here N counts *non-embedding* parameters (Kaplan and et al. , OpenAI, §1.3),[28] D is tokens, $C_{\min}$ is the compute that would be expended at a batch much smaller than the critical-batch power-law $B_{\text{crit}}(L)$ (Kaplan and et al. , OpenAI, §5.1). The exponents are small: $\alpha_N \approx 0.076$ means doubling N at fixed D improves loss by a factor $2^{-0.076} \approx 0.949$, i.e. $\approx 5\%$ relative.

The joint $L(N, D)$ form. The two single-axis laws are unified into a single equation (Kaplan and et al. , OpenAI, §1.2, eq. 1.5)[29]:

$$L(N, D) = \left[\left(\frac{N_c}{N}\right)^{\alpha_N/\alpha_D} + \frac{D_c}{D}\right]^{\alpha_D}. \tag{1.5}$$

The exponent combination $\alpha_N/\alpha_D \approx 0.74$ is what delivers Kaplan's sublinear-in-N data prescription: to avoid the overfitting penalty, D should grow as $D \propto N^{0.74}$ (Kaplan and et al. , OpenAI, §1.2). Doubling N requires only $\sim 1.67\times$ more data.

The compute-allocation rule. Combined with the critical-batch power law and eq. (2), eq. (1.5) implies a closed-form for how a fixed compute budget $C_{\min}$ should be split across N, batch size B, and training steps S (Kaplan and et al. , OpenAI, §1.2, eqs. 1.7–1.8):

$$N \propto C_{\min}^{\alpha_C^{\min}/\alpha_N}, \quad B \propto C_{\min}^{\alpha_C^{\min}/\alpha_B}, \quad S \propto C_{\min}^{\alpha_C^{\min}/\alpha_S}, \quad D = B \cdot S. \tag{1.7}$$

Plugging in the measured exponents (Kaplan and et al. , OpenAI, §1.2):

$$N \propto C_{\min}^{0.73}, \quad B \propto C_{\min}^{0.24}, \quad S \propto C_{\min}^{0.03}, \quad D \propto C_{\min}^{0.27}. \tag{3}$$

[27] KB excerpt: `kb/excerpts/kaplan2020.md#sec-1-2-equations`.
[28] KB excerpt: `kb/excerpts/kaplan2020.md#sec-1-3-notation`.
[29] KB excerpt: `kb/excerpts/kaplan2020.md#sec-1-2-joint`.

Reading: a 10× compute budget should buy a $\sim 5.4\times$ larger model, $\sim 1.7\times$ larger batch, and only $\sim 1.07\times$ more steps — almost no extra training time, dramatically more parameters. This is the prescription the GPT-3 generation took literally: 175B parameters trained on $\sim$ 300B tokens, tokens-per-parameter ≈ 1.7. Findings 1–3 of Kaplan and et al. (OpenAI, §1.1)[30] (performance depends on scale not shape; smooth power laws over six orders of magnitude; universality of overfitting) are still considered correct. Finding 5 ("train very large, stop short of convergence") is the one that flipped.

3.2 Chinchilla and the parametric loss

Hoffmann and et al. (DeepMind) re-ran the scaling sweep with one methodological change: for each parameter count N, they trained 4 models with cosine LR schedules whose horizon *matched the actual training-token count*, varying that horizon by a factor of 16 (Hoffmann and et al. , DeepMind, §3.1).[31] They then attacked the compute-optimal allocation problem

$$\min_{N,D} L(N, D) \quad \text{subject to} \quad 6ND = C \tag{4}$$

three independent ways and found the answers agreed within bootstrap error.

Approach 1 — minimum over training curves. For each $N \in \{75\text{M}, 250\text{M}, 500\text{M}, 1\text{B}, 2.5\text{B}, 5\text{B}, 10\text{B}\}$, fit 4 runs at 4 different horizons; at 1500 log-spaced FLOP values, find the (N, D) pair with lowest loss. Fit power laws $N_{opt} \propto C^a$, $D_{opt} \propto C^b$, obtaining $a = 0.50$, $b = 0.50$ (Hoffmann and et al. , DeepMind, §3.1).

Approach 2 — IsoFLOP profiles. At 9 fixed FLOP budgets $C \in [6 \times 10^{18}, 3 \times 10^{21}]$, sweep N and read off the loss-vs-N parabola minimum to get $N_{opt}(C)$. Fit $a = 0.49$, $b = 0.51$ (Hoffmann and et al. , DeepMind, §3.2).[32]

Approach 3 — parametric loss fit. Pose a closed-form risk decomposition for the loss (Hoffmann and et al. , DeepMind, §3.3, eq. 2)[33]:

$$\hat{L}(N, D) \triangleq E + \frac{A}{N^\alpha} + \frac{B}{D^\beta}, \tag{2}$$

verbatim from the paper. The first term E is the irreducible loss (entropy of natural text on the eval distribution); A/N^α is the capacity-limited residual of a perfectly-trained N-parameter model; B/D^β is the optimization-limited residual of finite training steps over a finite sample (Hoffmann and et al. , DeepMind, §3.3). Fit (A, B, E, α, β) by L-BFGS on the Huber loss with $\delta = 10^{-3}$ (Hoffmann and et al. , DeepMind, §3.3, eq. 3):

$$\min_{A,B,E,\alpha,\beta} \sum_{\text{Runs } i} \text{Huber}_\delta\Big(\log \hat{L}(N_i, D_i) - \log L_i\Big). \tag{3}$$

Minimizing eq. (2) under the $C \approx 6ND$ constraint yields the compute-optimal frontier in closed form (Hoffmann and et al. , DeepMind, §3.3, eq. 4):

$$N_{opt}(C) = G\left(\frac{C}{6}\right)^a, \qquad D_{opt}(C) = G^{-1}\left(\frac{C}{6}\right)^b, \tag{4}$$

[30] KB excerpt: `kb/excerpts/kaplan2020.md#sec-1-1-bullets`.
[31] KB excerpt: `kb/excerpts/hoffmann2022-chinchilla.md#sec-3-1`.
[32] KB excerpt: `kb/excerpts/hoffmann2022-chinchilla.md#sec-3-2`.
[33] KB excerpt: `kb/excerpts/hoffmann2022-chinchilla.md#sec-3-3`.

with

$$G = \left(\frac{\alpha A}{\beta B}\right)^{1/(\alpha+\beta)}, \quad a = \frac{\beta}{\alpha+\beta}, \quad b = \frac{\alpha}{\alpha+\beta}, \qquad a+b = 1 \text{ identically.} \tag{5}$$

The split (a, b) is the empirical question. Approach 3 gives $a = 0.46$, $b = 0.54$ (Hoffmann and et al. , DeepMind, §3.3).

All three approaches agree, and disagree with Kaplan. Table 3 summarizes (Hoffmann and et al. , DeepMind, §3, Table 2):[34]

Approach	a ($N_{opt} \propto C^a$)	b ($D_{opt} \propto C^b$)
1. Minimum over training curves	0.50 (0.488, 0.502)	0.50 (0.501, 0.512)
2. IsoFLOP profiles	0.49 (0.462, 0.534)	0.51 (0.483, 0.529)
3. Parametric $\hat{L}$ fit	0.46 (0.454, 0.455)	0.54 (0.542, 0.543)
Kaplan and et al. (OpenAI)	**0.73**	**0.27**

Table 3: Three independent estimates of the compute-allocation exponents in $N_{opt} \propto C^a$, $D_{opt} \propto C^b$, all within bootstrap of $a \approx b \approx 1/2$, vs. Kaplan's 0.73/0.27 split. Bracketed numbers are 10th/90th percentile bootstrap intervals. After Hoffmann and et al. (DeepMind, §3, Table 2).

The 70B vs. 280B test. Hoffmann and et al. (DeepMind) fixed total compute at the Gopher budget ($\approx 5.76 \times 10^{23}$ FLOPs) and trained Chinchilla at $N = 70$B, $D = 1.4$T (tokens-per-parameter = 20), versus Gopher at $N = 280$B, $D = 300$B (tokens-per- parameter ≈ 1.07) (Hoffmann and et al. , DeepMind, §4.1, Table 4). Same depth ($L = 80$); half the width ($d_{\text{model}} : 16384 \to 8192$); doubled max LR; halved batch. Chinchilla beats Gopher on every Pile subset, +0.6 pp average across BIG-bench, and lifts MMLU from 60.0% to 67.5% (Hoffmann and et al. , DeepMind, §4.2).[35] The headline rule of thumb falls out of eq. (4)'s Approach-1 projection: tokens-per-parameter ≈ 20 at compute optimum, with a slow drift toward more data at very large scale — 20.0 at 400M, 20.2 at 1B, 21.1 at 175B, 21.2 at 1T (Hoffmann and et al. , DeepMind, §3, Table 3).[36]

3.3 Why Kaplan over-allocated to parameters — the LR-horizon bug

The methodological core of Hoffmann and et al. 's correction is one sentence (Hoffmann and et al. , DeepMind, §2)[37]:

> For a fixed learning rate cosine schedule to 130B tokens, the intermediate loss estimates (for $D' \ll 130$B) are therefore overestimates of the loss of a model trained with a schedule length matching D'.

In Kaplan and et al. (OpenAI)'s sweep, every model was trained with a cosine LR decay reaching zero at a fixed total token horizon, regardless of how many tokens that individual run actually consumed (Kaplan and et al. , OpenAI, §2.2). The intermediate-loss readouts at $D' \ll 130$B were therefore taken from a model whose LR was *still high* — mid-training, not at its endpoint. A model genuinely trained to D' with an LR schedule that decays to zero at D' achieves a lower final

[34] KB excerpt: `kb/excerpts/hoffmann2022-chinchilla.md#sec-3-table-2`.
[35] KB excerpt: `kb/excerpts/hoffmann2022-chinchilla.md#sec-4-2`.
[36] KB excerpt: `kb/excerpts/hoffmann2022-chinchilla.md#sec-3-table-3`.
[37] KB excerpt: `kb/excerpts/hoffmann2022-chinchilla.md#sec-2-disagree-kaplan`.

loss than the same model trained to D' with a 130B-horizon schedule. Kaplan inferred his $L(D)$ scaling from the latter, biased upward, runs.

Intuition

Kaplan's setup was, in effect, "take a partially-trained model and ask how much loss is left." That penalizes smaller-D runs because they are not yet at their LR-decay endpoint — their loss is the loss of an in-progress run, not of a converged-to-its-token-budget run. The lesson is that what we measure is the loss *at training endpoint*, where endpoint ≡ end of LR schedule. Conflating "tokens consumed" with "training endpoint" was the trap. Returning to canonical form: when Hoffmann and et al. (DeepMind) matched the LR-decay horizon to D' for every run in eq. (3), the apparent advantage of more-N-less-D in eq. (3) disappeared and the exponents (a, b) flipped from $(0.73, 0.27)$ to $\approx (0.50, 0.50)$.

The functional forms (1.1)–(1.5) are unaffected — they fit either sweep — but the *numerical exponents* extracted from eq. (1.7) change qualitatively. This is the cleanest example we have in LLM scaling of an apparent recipe being driven by the optimization protocol, not by the architecture. Notation note: Kaplan's N is non-embedding parameters (Kaplan and et al. , OpenAI, §1.3); Hoffmann's N is total parameters; the constant offset is small at frontier scale (vocab embedding ≈ 0.5%–5% of total) but matters when comparing exponents and constants across papers.

3.4 Frontier 2024–2026 — past Chinchilla

The post-Chinchilla industry move is to train substantially smaller models on substantially more tokens than eq. (4) prescribes. Llama 3 8B was trained on ≈ 15T tokens (AI, 2024), giving tokens-per-parameter ≈ 1875 — roughly 94× over the Chinchilla optimum (table 4). Llama 3 70B at the same ≈ 15T-token budget is at ≈ 214 tokens-per-parameter, ∼ 11× over Chinchilla. DeepSeek-V3 trained 671B total / 37B active-MoE on 14.8T tokens (DeepSeek-AI, 2024b, §1, §2), putting active tokens-per-parameter at ≈ 400.

Model	N_{active}	D (tokens)	D/N_{active}	vs. Chinchilla
Chinchilla 70B (2022)	70B	1.4T	20.0	1.0×
Llama 2 70B (2023)	70B	2T	28.6	1.4×
Llama 3 70B (2024)	70B	∼ 15T	∼ 214	∼ 11×
Llama 3 8B (2024)	8B	∼ 15T	∼ 1875	∼ 94×
DeepSeek-V3 (2024) MoE	37B	14.8T	∼ 400	∼ 20×

Table 4: Tokens-per-active-parameter for selected frontier-scale models, all post-Chinchilla. Llama 3 numbers from AI (2024); DeepSeek-V3 from DeepSeek-AI (2024b, §2); the Chinchilla baseline from Hoffmann and et al. (DeepMind, §4.1). The "vs. Chinchilla" column is $D/(20\,N_{\text{active}})$.

Why over-train. The objective Chinchilla optimizes is pre-training compute, C_{train}. The cost a deployed-model lab actually pays is

$$\text{Total cost} \;=\; C_{\text{train}} \;+\; N_{\text{queries-served}} \cdot C_{\text{inf-per-query}}. \tag{6}$$

When $N_{\text{queries-served}}$ is large — consumer chat products, API serving — the second term dominates, and the optimum shifts toward smaller N (lower $C_{\text{inf-per-query}}$) with proportionally more D. Llama 3 8B's ∼ 1875 tokens-per-parameter is the endpoint of this re-objective for "small dense model

destined for billions of queries." Crucially, the Chinchilla parametric loss eq. (2) *still applies* along this trajectory: $\hat{L}(N, D)$ keeps decreasing in D at fixed N, just sub-optimally on the C_{train} margin. Over-training is not a refutation of Chinchilla; it is a different objective.

Reasoning models as a second compute axis. DeepSeek-R1 (DeepSeek-AI, 2025), the OpenAI o-series, and Gemini 2.5 (DeepMind, 2025) demonstrated that test-time compute is now a deployable scaling axis: production reasoning models expose configurable thinking budgets (Gemini's `thinkingBudget` parameter; Qwen 3's thinking-mode toggle via system prompt (Team and Alibaba, 2025)), running $5\times$–$100\times$ the non-thinking cost per query for hard math, code, and agent loops. The empirical surface for the joint $(C_{\text{train}}, C_{\text{test}})$ trade-off is documented but no closed-form scaling law captures it across the multi-axis frontier as of 2026.

3.5 µP and µTransfer — HP transfer across width

A scaling recipe is only as useful as the hyperparameters fed to it. At frontier scale every full pretraining run costs millions of dollars, so iterating an HP sweep on the target model is intractable. The Maximal Update Parametrization (µP) of Yang and et al. (Microsoft) is the parametrization in which optimal hyperparameters are *width-invariant*, enabling µTransfer: tune the master LR, Adam betas, schedule shape, and per-layer multipliers on a small proxy model, then copy them zero-shot to the full-scale target. The headline demonstration: tuning at 40M proxy scale and transferring to GPT-3 6.7B with total tuning cost equal to 7% of one pretraining run (Yang and et al. , Microsoft, abstract).[38]

The µP scaling rules. Each linear layer falls into one of three categories by width behavior: *input weights and biases* (finite input dim, infinite output dim — e.g. embeddings); *output weights* (infinite input dim, finite output dim — e.g. the unembedding head); and *hidden weights* (infinite both — e.g. attention QKV/O, FFN). µP is exactly the per-category rescaling (Yang and et al. , Microsoft, §4, Table 3)[39]:

	Input weights & biases	Output weights	Hidden weights
Init. Var.	1/fan_in	**1**/fan_in$^{\mathbf{2}}$	1/fan_in
SGD LR	fan_out	**1**/fan_in	1
Adam LR	1	**1**/fan_in	**1**/fan_in

Table 5: The µP scaling rules. Bold cells are the differences from standard parametrization (SP), in which init variance is 1/fan_in everywhere and the LR is uniform. After Yang and et al. (Microsoft, §4, Table 3).

The effective per-weight LR is $\eta_W = \eta \cdot$ (Table 5 entry for W) at width n, where η is the dimensionless master LR transferred from the proxy. For Transformers, µP adds one structural change (Yang and et al. , Microsoft, §4, Definition 4.1)[40]:

Definition 3.1 (µP for Transformer; Yang and et al. , Microsoft)**.** The Maximal Update Parametrization (µP) for a Transformer is given by Table 5 and $1/d$ attention instead of $1/\sqrt{d}$, i.e. the attention logit is calculated as $q^\top k/d$ instead of $q^\top k/\sqrt{d}$ where query q and key k have dimension d.

[38] KB excerpt: `kb/excerpts/yang2022-mup.md#abstract`.
[39] KB excerpt: `kb/excerpts/yang2022-mup.md#sec-4-table-3`.
[40] KB excerpt: `kb/excerpts/yang2022-mup.md#sec-4-definition-4-1`.

Analogy

µP is the "right" parametrization in the same sense the $1/\sqrt{n}$ factor in the CLT is the right scaling: $1/\sqrt{n}$ is the unique rescaling for which $\frac{1}{\sqrt{n}}(x_1 + \cdots + x_n) \to \mathcal{N}(0,1)$ has a non-trivial limit (Yang and et al. , Microsoft, §2).[a] Width plays the role of n. The µP assignment of (init, LR, multiplier) per layer category is the unique stable choice that preserves $\Theta(1)$-magnitude weight updates — "feature learning" — in the infinite-width limit; SP either has trivial NTK-regime dynamics or blows up. Returning to math: under SP, after even one Adam step, attention pre-softmax scores $q^\top k$ acquire correlation across i, so $\mathrm{Var}(q^\top k) = \Theta(d)$ by Law-of-Large-Numbers (not CLT), and the Vaswani $1/\sqrt{d_k}$ rescaling makes the logits $\Theta(\sqrt{d})$ — they blow up with width (Yang and et al. , Microsoft, §5). µP's $1/d$ scaling absorbs the extra factor exactly. Vaswani's $1/\sqrt{d_k}$ remains correct *at initialization* (uncorrelated q, k); µP's $1/d$ is correct *during training*.

[a] KB excerpt: `kb/excerpts/yang2022-mup.md#sec-2-clt`.

What transfers. Yang and et al. (Microsoft, §6)[41] report that across width, master LR, Adam betas, LR schedule shape, per-layer init multipliers, and most parameter multipliers transfer; across batch size, sequence length, and training-step count they transfer empirically within "reasonable ranges" (batch ≥ 32, seqlen ≥ 128, steps ≥ 5000). What does *not* transfer: regularization (weight decay, dropout) and any HP that interacts with the regularization-vs-optimization trade-off, since regularization strength depends on data size, not just on width (Yang and et al. , Microsoft, §6). Across depth the transfer is partial: optimal init standard deviation does not transfer well across depth; practical recipe is to fix init at the target depth and sweep the remaining HPs (Yang and et al. , Microsoft, §6.1).

Contradiction

Strict (Yang and et al. , Microsoft, Definition 4.1) µP is rare in production. Cerebras has explicit practitioner guides and Microsoft (Phi-4) and Graphcore (u-µP, Graphcore, 2024) disclose usage; Anthropic and OpenAI do not. Meta's Llama 4 tech report (AI, 2026) cites a "MetaP" scheme for cross-scale HP transfer with thin published detail. The community's working assumption is that frontier labs apply some subset of Table 5's rules (typically the LR scalings) but keep Vaswani's $1/\sqrt{d_k}$ attention rather than µP's $1/d$, making them "µP-inspired" rather than strict-Definition-4.1 variants. Independent positive results outside the Microsoft/Cerebras/Graphcore ecosystem at 10B+ scale are scarce; whether the loose variants retain the unique-feature-learning guarantee is open.

3.6 Precision-aware scaling

et al. (Harvard / Google) extend the Chinchilla parametric form eq. (2) with precision as a third axis. The core construction reduces to an *effective parameter count* $N_{\text{eff}}(N, b_{\text{train}})$ that is decreasing in N when training precision b_{train} falls below a threshold: training in low precision is, in this picture, equivalent to training a smaller-N model in full precision, with the equivalence-mapping fitted from sub-frontier sweeps. Combined with a post-training quantization-loss model, the joint

[41] KB excerpt: `kb/excerpts/yang2022-mup.md#sec-6-what-transfers`.

result is that *over-trained models are harder to quantize* — the more D a model has consumed relative to N, the larger the post-quantization loss penalty at a given deployment precision.

deepen-cite: scaling-laws-precision-2024 §4: effective-parameter form $N_{\text{eff}}(N, b_{\text{train}})$ and the over-training → harder-to-quantize prediction.

This collides directly with the over-training strategy of table 4: the same Llama-3 8B that gains inference-cost efficiency from being small dense + token-heavy is, by the et al. (Harvard / Google) prediction, more difficult to quantize at deployment time — partially offsetting the very gain that motivated the strategy. The industry response has been native low-precision training: pre-train at the precision you will deploy, not above it. DeepSeek-V3 reports zero irrecoverable loss spikes over 14.8T tokens of FP8 pre-training (DeepSeek-AI, 2024b, abstract); Llama 4 likewise discloses FP8 pre-training. u-µP (Graphcore, 2024) integrates µP with unit-scaling so per-tensor activation magnitudes are predictable, enabling FP8 without dynamic rescaling. The full multi-axis $(N, D, C, b_{\text{train}}, b_{\text{deploy}})$ frontier is incompletely mapped at frontier scale; we return to the mixed-precision recipe in section 6.

3.7 Frontier-2026 scaling map

Tying the threads together: scaling at the 2026 frontier is a four-axis surface $(C_{\text{train}}, C_{\text{test}}, b, q)$ where C_{train} is pre-training FLOPs, C_{test} is inference-time-compute budget, b is training/deployment precision, and q is the quality-mixture composition of D (covered separately in section 2; see Data Mixing Laws there). Frontier labs jointly optimize all four. Where they agree: Chinchilla's parametric form eq. (2) fits, the $C \approx 6ND$ formula is universal, µP-inspired HP transfer is broadly adopted (with disclosure asymmetry; see section 3.5), FP8 or lower is the new training-precision floor where the stack supports it. Where they diverge: tokens-per-parameter is now a deployment-conditional choice in $[20, 2000]$ rather than a universal ≈ 20, and the C_{train}-vs-C_{test} split is a per-product tuning that no closed-form law yet predicts.

Contradiction

Kaplan-vs-Chinchilla, on the methodological question, is settled: the 0.73/0.27 split was an LR-schedule artifact, the corrected $\approx 0.50/0.50$ stands on three independent fits within bootstrap of each other (Hoffmann and et al. , DeepMind, §3, Table 2). But multiple "optimal" definitions are now in play. Pre-training-FLOP- optimal (Chinchilla, $D/N \approx 20$); inference-cost-aware (Llama 3, D/N in the hundreds to thousands; AI, 2024); inference-time-compute-aware (the o-series, R1, Gemini 2.5 (DeepMind, 2025), Qwen 3 (Team and Alibaba, 2025) with C_{test} exposed); precision-aware (et al. , Harvard / Google with the over-training → harder-to-quantize trade-off pulling against Llama-3-style $D/N \approx 1875$). The functional form eq. (2) survives; the prescription that comes out of eq. (4) is now objective-conditional. "Chinchilla is wrong" is a mis-framing; "Chinchilla solves a different objective than the one the industry actually optimizes" is accurate. The unresolved open question is the joint $(C_{\text{train}}, C_{\text{test}}, b, q)$ scaling law — documented empirically, not captured in closed form.

3.8 Forward link

The scaling laws of this section dictate the budget split (N, D) at fixed C and the HP-transfer protocol that makes any chosen point trainable; they do not specify *the optimizer* that spends the FLOPs or the schedule that matches the LR-decay horizon identified in section 3.3. The

next section (section 4) walks the optimizer axis: AdamW as the production default, the training-loss-conditional speedup claims of Lion and Sophia that did not hold on downstream evaluation, Muon's Newton-Schulz-orthogonalized matrix update as the only published optimizer with $\sim 2\times$ compute-efficiency lift at LLM scale, and the WSD (Warmup-Stable-Decay) schedule that pairs with continual pre-training and merging in section 12. Optimizer choice and precision choice will turn out to be *not* orthogonal — Muon's update RMS exceeds BF16's high-precision range without weight decay, exactly the optimizer×precision coupling section 3.6 flagged — and the LR-schedule lesson from section 3.3 reappears as the WSD cooldown phase that loads quality data over the final tokens.

4 Optimization

The scaling-law section fixed the budget split (N, D) at fixed C and the hyperparameter-transfer protocol that makes any chosen point trainable; it left the optimizer that spends the FLOPs as a free variable. Optimizer choice is one of the largest remaining levers on training compute. AdamW is the production default at every disclosed frontier lab, with a particular empirical signature — update-RMS in the 0.2–0.4 range per matrix parameter (Liu and et al. , Moonlight team, §2.2)[42] — that the 2024–2026 alternatives all measure themselves against. Lion's sign-momentum and Sophia's diagonal-Hessian preconditioning each reported ∼1.5–2× training-loss speedups; both gaps largely closed on downstream evaluation.

deepen-cite: lion2023 §4: head-to-head LLM training-loss speedup vs. AdamW that did not transfer to downstream-task generalization.

deepen-cite: sophia2023 §5: diagonal-Hessian preconditioning, $\sim 2\times$ AdamW training-loss speedup, downstream-evaluation gap.

Muon (Liu and et al. , Moonlight team), which orthogonalizes momentum on matrix parameters via a Newton–Schulz polynomial iteration, is the only published optimizer with a $\sim 2\times$ compute-efficiency lift verified at LLM scale: the Moonlight 3B-active / 16B-MoE total run on 5.7T tokens (Liu and et al. , Moonlight team, §3.3).[43] The schedule axis runs in parallel: cosine decay from GPT-3 / Llama-1/2/3 through to the Warmup–Stable–Decay (WSD) family that originated with MiniCPM (et al. , Microsoft) and is the schedule used by the Moonlight Muon demonstration. Schedule-Free (Liu and et al. , Moonlight team, ICML 2024)

deepen-cite: schedule-free-defazio-2024 §1: removes LR schedule entirely via online primal averaging; limited LLM-scale validation.

sits as the open frontier.

The thesis of this section is that for matrix-shaped weights — which dominate parameter count in a Transformer — the geometry of the update is now a first-class design variable. AdamW updates each coordinate independently under a per-coordinate variance estimate; Muon updates each matrix as a matrix, projecting the momentum onto its polar factor so all singular directions of the update have equal energy. The two families' published scaling laws fit the same Kaplan and et al. (OpenAI)-style functional form (eq. (2)) with near-identical exponents but different prefactors:

$$L_{\text{Muon}}(C) = 2.506 \cdot C^{-0.052}, \tag{7}$$

$$L_{\text{AdamW}}(C) = 2.608 \cdot C^{-0.054}, \tag{8}$$

[42] KB excerpt: `kb/excerpts/muon-moonlight2025.md#sec-2-2-rms`.
[43] KB excerpt: `kb/excerpts/muon-moonlight2025.md#sec-3-3-moonlight`.

verbatim from Liu and et al. (Moonlight team, §3.2, Table 3)[44] along the compute-optimal Chinchilla frontier at the Moonlight sweep's scale. The numerical content of the 2× claim is exactly that eq. (7) sits below eq. (8) by a near-constant factor, so matched-loss compute requires $\approx 0.52\, C_{\text{AdamW}}$ (Liu and et al. , Moonlight team, Figure 1a).

4.1 The optimization step and its symbols

The pre-training objective on a corpus $\mathcal{D}$ of token sequences $\mathbf{x}$ is the next-token cross-entropy

$$\mathcal{L}(\theta) \;=\; -\,\mathbb{E}_{\mathbf{x}\sim\mathcal{D}}\left[\sum_{t=1}^{T} \log \pi_\theta(x_t \mid x_{<t})\right], \tag{9}$$

minimized by stochastic first-order updates of the form

$$\theta_{t+1} \;=\; \theta_t \;-\; \eta_t \cdot U_t(g_t,\, \theta_t), \tag{10}$$

with $g_t = \nabla_\theta \mathcal{L}(\theta_t)$ on a mini-batch, η_t the schedule-controlled learning rate, and U_t the update returned by the optimizer (which may carry state). Table 6 fixes notation for the rest of the section.

Symbol	Type	Meaning
θ	$\mathbb{R}^P$	full parameter vector ($P \in [10^9, 10^{12}]$ at frontier)
$\mathbf{W}$	$\mathbb{R}^{A\times B}$	a matrix-shaped parameter (FFN, attention QKV/O)
$\mathbf{M}_t$	$\mathbb{R}^{A\times B}$	momentum buffer for matrix $\mathbf{W}$
$\mathbf{O}_t$	$\mathbb{R}^{A\times B}$	polar-factor approximation Newton-Schulz($\mathbf{M}_t$)
g_t	$\mathbb{R}^P$	mini-batch gradient at step t
m_t, v_t	$\mathbb{R}^P$	Adam first / second moments
η_t	$\mathbb{R}$	schedule-controlled learning rate at step t
β_1, β_2	$\mathbb{R}$	Adam moment decays (typical 0.9, 0.95)
μ	$\mathbb{R}$	Muon momentum decay (typical 0.95)
λ	$\mathbb{R}$	decoupled weight-decay coefficient (typical 0.1)
ε	$\mathbb{R}$	numerical-stability constant (typical 10^{-8})
C	FLOPs	total training compute, $C \approx 6ND$ from eq. (2)

Table 6: Symbols used in section 4. Matrix parameters $\mathbf{W} \in \mathbb{R}^{A\times B}$ are the only shapes Muon updates; embeddings, the LM head, and 1-D parameters (RMSNorm gains, biases) stay on AdamW (Liu and et al. , Moonlight team, §2.2).[45]

4.2 AdamW, the production default

Adam (Liu and et al. , Moonlight team, Kingma & Ba 2015)

deepen-cite: kingma2015-adam: Adam optimizer original, (β_1, β_2) moment definitions, the $\hat{m}_t/(\sqrt{\hat{v}_t}+\varepsilon)$ update rule used verbatim below.

keeps two exponential moving averages of the gradient and its square,

$$m_t \;=\; \beta_1 m_{t-1} + (1-\beta_1) g_t, \qquad v_t \;=\; \beta_2 v_{t-1} + (1-\beta_2) g_t^2, \tag{11}$$

applies bias correction, and uses the ratio $\hat{m}_t/(\sqrt{\hat{v}_t}+\varepsilon)$ as the per-coordinate update direction, with $\hat{m}_t = m_t/(1-\beta_1^t)$ and $\hat{v}_t = v_t/(1-\beta_2^t)$. The "W" in AdamW (Liu and et al. , Moonlight team, Loshchilov & Hutter 2019)

[44] KB excerpt: `kb/excerpts/muon-moonlight2025.md#sec-3-2-scaling`.

deepen-cite: loshchilov2019-adamw: decoupled weight-decay derivation; required citation for the $-\eta_t \lambda \theta_t$ term being applied *after* Adam's preconditioning rather than folded into g_t.

is decoupled weight decay: rather than fold $\lambda\theta$ into g_t (which would put weight decay inside Adam's per-coordinate rescaling), AdamW applies the decay to θ directly:

$$\theta_{t+1} \;=\; \theta_t \;-\; \eta_t \Big[\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon} \;+\; \lambda\, \theta_t \Big] . \tag{2}$$

Equation (2) is the universal AdamW step at LLM scale — LLaMA (AI, 2024), DeepSeek-V3 (DeepSeek-AI, 2024b, §4), and Phi-4 (et al. , Microsoft) all use it, with typical $(\beta_1, \beta_2) = (0.9, 0.95)$, $\lambda = 0.1$, and $\varepsilon = 10^{-8}$.

The empirical update-RMS signature. For each matrix parameter $\mathbf{W} \in \mathbb{R}^{A \times B}$, the AdamW update has root-mean-square magnitude $\|\eta_t\, \hat{m}_t / (\sqrt{\hat{v}_t} + \varepsilon)\|_2 / \sqrt{AB}$ that is "usually around 0.2 to 0.4" (Liu and et al. , Moonlight team, §2.2).[46] This number is the calibration target every alternative measures itself against: matching it lets one reuse AdamW's tuned (η, λ) across optimizers, sparing a frontier-scale sweep. Section 4.4 reports how the Muon family lands exactly on this number by design.

Optimizer-state cost. AdamW carries $2P$ words of moment state (m, v) on top of the parameters and gradients — $4P$ words total in mixed-precision training when an FP32 master copy is kept (see section 6). For a 70B-parameter model in FP32 the optimizer-state footprint alone is ~ 1.1 TB (Liu and et al. , Moonlight team, §2).

deepen-cite: adamw-state-cost: standard back-of-envelope — $4P$ words for params + grads + Adam (m, v); referenced rather than canonically cited because it is folklore in the distributed-training literature.

4.3 Lion and Sophia — the training-loss-conditional gap

Two AdamW alternatives published in 2023 reported $\sim$1.5–2$\times$ speedups on training loss and have been extensively re-evaluated since.

Lion — sign-momentum. Lion (Liu and et al. , Moonlight team, Chen et al. 2023)

deepen-cite: lion2023 §2: sign-momentum update rule with single moment buffer; arXiv 2302.06675.

replaces the Adam ratio with the sign of an exponentially smoothed gradient, giving

$$c_t \;=\; \beta_1 m_{t-1} + (1 - \beta_1) g_t, \qquad \theta_{t+1} \;=\; \theta_t \;-\; \eta_t (\operatorname{sign}(c_t) + \lambda \theta_t)\,, \tag{12}$$

with the moment buffer updated as $m_t = \beta_2 m_{t-1} + (1 - \beta_2) g_t$. A single moment buffer halves the optimizer-state memory relative to Adam and the per-step cost is one elementwise sign rather than a square root. Lion's headline claim is wall-clock speedup at matched final loss on vision transformers; for LLMs the gap closed in head-to-head comparisons.

deepen-cite: three-optimizer-2025 (arXiv:2507.08472): controlled LLM head-to-head reporting AdamW catching up or winning on downstream evaluation; cited as Phase-1-tier signal in the KB note.

[46] KB excerpt: `kb/excerpts/muon-moonlight2025.md#sec-2-2-rms`.

Sophia — diagonal-Hessian preconditioning. Sophia (Liu and et al. , Moonlight team, Liu, Li et al. 2023, ICLR 2024)

deepen-cite: sophia2023 §3 (arXiv 2305.14342): stochastic diagonal-Hessian estimator h_t, the clipped second-order update $\theta_{t+1} = \theta_t - \eta_t \text{clip}(m_t / \max(h_t, \varepsilon), \rho)$, and the headline "2× AdamW training-loss speedup" figure.

replaces Adam's gradient-variance preconditioner $\sqrt{\hat{v}_t}$ with a stochastic diagonal-Hessian estimate h_t, computed via a Hutchinson-style probe on the network's curvature operator, and clips the resulting step. The reported $\sim 2\times$ speedup over AdamW on training loss held in published replications. The downstream gap is the load-bearing complication: AdamW catches up or surpasses Sophia on standard evaluation suites at matched *wall-clock* time, despite Sophia's lower training loss.

Contradiction

Both Lion and Sophia exhibit the same pattern. Lower training loss at matched compute, but downstream evaluation in the closing-the-gap or gap-reversed regime once both optimizers run to a comparable budget and the downstream metric (MMLU, GSM8K, HumanEval, BBH-class) replaces validation NLL. The mechanistic interpretation has not been isolated from the experimental confound (LR sweep, weight-decay sweep, schedule shape interaction). Treat reported optimizer speedups as *training-loss-conditional* until verified on the downstream evaluation panel; this is the operational lesson of the 2024–2025 LLM optimizer literature (Liu and et al. , Moonlight team, §1).[a]

[a] KB excerpt: `kb/excerpts/muon-moonlight2025.md#sec-1-challenges`.

4.4 Muon — orthogonalized momentum on matrix parameters

Liu and et al. (Moonlight team) scale up the Muon optimizer (originally K. Jordan et al. 2024)

deepen-cite: jordan2024-muon: original "Muon: an optimizer for hidden layers in neural networks" write-up; the Moonlight paper is the LLM-scale extension that this section primarily cites.

to LLM training. Muon's central observation is that a matrix parameter $\mathbf{W} \in \mathbb{R}^{A \times B}$ is not a flat vector. The gradient $\nabla \mathcal{L} \in \mathbb{R}^{A \times B}$ has singular structure — some directions in row / column space accumulate signal faster than others — and the AdamW update preserves this anisotropy because per-coordinate rescaling is basis-dependent. Muon's update orthogonalizes the momentum before the step, replacing $\mathbf{M}_t$ with its polar factor $\mathbf{O}_t = \mathbf{U}\mathbf{V}^\top$ in the SVD $\mathbf{M}_t = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$. Verbatim from Liu and et al. (Moonlight team, §2.1, eq. 1)[47]:

$$\begin{aligned} \mathbf{M}_t &= \mu\, \mathbf{M}_{t-1} + \nabla \mathcal{L}_t(\mathbf{W}_{t-1}) \\ \mathbf{O}_t &= \text{Newton-Schulz}(\mathbf{M}_t) \\ \mathbf{W}_t &= \mathbf{W}_{t-1} - \eta_t\, \mathbf{O}_t. \end{aligned} \tag{3}$$

$\mathbf{O}_t$ approximates the polar factor of $\mathbf{M}_t$, i.e. the matrix whose SVD has the same singular vectors as $\mathbf{M}_t$ but whose singular values are all unity: $(\mathbf{M}_t \mathbf{M}_t^\top)^{-1/2} \mathbf{M}_t = \mathbf{U}\mathbf{V}^\top$ (Liu and et al. , Moonlight team, §2.1). The orthogonalization "can ensure that the update matrices are isomorphic, preventing the weight from learning along a few dominant directions" (Liu and et al. , Moonlight team, §2.1).

The Newton–Schulz iteration. The polar factor of $\mathbf{M}_t$ is computed by a matrix polynomial iteration that avoids the SVD entirely. Initialize $\mathbf{X}_0 = \mathbf{M}_t / \|\mathbf{M}_t\|_F$ (Frobenius normalization keeps

[47] KB excerpt: `kb/excerpts/muon-moonlight2025.md#sec-2-1-update-rule`.

the iteration in the convergence basin); then

$$\mathbf{X}_k \;=\; a\,\mathbf{X}_{k-1} \;+\; b\,(\mathbf{X}_{k-1}\mathbf{X}_{k-1}^\top)\,\mathbf{X}_{k-1} \;+\; c\,(\mathbf{X}_{k-1}\mathbf{X}_{k-1}^\top)^2\,\mathbf{X}_{k-1}, \tag{4}$$

verbatim from Liu and et al. (Moonlight team, §2.1, eq. 2)[48] with coefficients $(a, b, c) = (3.4445, -4.7750, 2.0315)$ (Liu and et al. , Moonlight team, K. Jordan et al. 2024, cited in §2.1).[49] Five iterations suffice in production (the polynomial converges to the polar factor on each side of the singular-value spectrum at a quintic rate near unity). The output $\mathbf{X}_N$ is $\mathbf{O}_t$ in eq. (3).

Analogy

$\mathbf{O}_t$ is the polar-decomposition projection of $\mathbf{M}_t$ onto the manifold of orthogonal matrices in the Frobenius geometry. If $\mathbf{M}_t = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$ is the SVD, then $\mathbf{O}_t = \mathbf{U}\mathbf{V}^\top$ is the closest orthogonal matrix to $\mathbf{M}_t$ under any unitarily-invariant norm (Liu and et al. , Moonlight team, Mirsky 1960).

deepen-cite: mirsky1960-symmetric-gauge: canonical reference for the polar projection being the unitarily-invariant-norm-closest orthogonal matrix to a given matrix.

The Newton–Schulz iteration in eq. (4) computes this projection without ever forming the SVD — a polynomial in $\mathbf{X}_{k-1}\mathbf{X}_{k-1}^\top$ approximates the inverse square root $(\mathbf{X}\mathbf{X}^\top)^{-1/2}$, and the (a, b, c) coefficients are the optimal cubic-in-singular-values fit that maps the Frobenius-normalized spectrum of $\mathbf{X}_0$ onto $\{1\}$ in N iterations. Returning to math: the canonical Muon update in eq. (3) reduces to "take the polar factor of momentum, scale by η_t, subtract," and eq. (4) is the SVD-free numerical recipe for the polar factor.

Two scale-stable ingredients. Liu and et al. (Moonlight team, §2.2) report that vanilla eq. (3) converges faster than AdamW at small scale but the gain diminishes at LLM scale because of two failure modes that the original formulation does not address. The fixes are load-bearing.

Mandatory weight decay. Without an explicit decay term, "both the weight and the layer output's RMS keep growing to a large scale, exceeding the high-precision range of bf16, which might hurt the model's performance" (Liu and et al. , Moonlight team, §2.2).[50] The fix is to fold AdamW-style decoupled weight decay into the Muon step:

$$\mathbf{W}_t \;=\; \mathbf{W}_{t-1} \;-\; \eta_t(\mathbf{O}_t \;+\; \lambda\mathbf{W}_{t-1})\,, \tag{13}$$

verbatim from Liu and et al. (Moonlight team, §2.2, eq. 3).[51] With eq. (13), Muon outperforms both vanilla Muon and AdamW in the over-train regime that table 4 of section 3 characterizes.

Per-shape update-RMS adjustment. Lemma 1 of Liu and et al. (Moonlight team, §2.2)[52] states that for a full-rank matrix parameter of shape $A \times B$, the theoretical Muon update RMS is $\sqrt{1/\max(A, B)}$. Tall-thin matrices (large $\max(A, B)$, e.g. FFN $\mathbf{W}_{\text{up}} \in \mathbb{R}^{4D\times D}$) get under-updated; small per-head matrices (treating each KV head in GQA (Ainslie et al., 2023) or MLA (DeepSeek-AI, 2024a) as a separate parameter) get over-updated and destabilize training. The fix is to multiply each matrix's update by $\sqrt{\max(A, B)}$ and a constant factor 0.2 that lands in AdamW's empirical 0.2–0.4 band. The full step is

$$\mathbf{W}_t \;=\; \mathbf{W}_{t-1} \;-\; \eta_t\Big(0.2\,\mathbf{O}_t\,\sqrt{\max(A, B)} \;+\; \lambda\mathbf{W}_{t-1}\Big)\,, \tag{4'}$$

[48] KB excerpt: `kb/excerpts/muon-moonlight2025.md#sec-2-1-ns`.
[49] KB excerpt: `kb/excerpts/muon-moonlight2025.md#sec-2-1-ns`.
[50] KB excerpt: `kb/excerpts/muon-moonlight2025.md#sec-2-2-wd`.
[51] KB excerpt: `kb/excerpts/muon-moonlight2025.md#sec-2-2-wd`.
[52] KB excerpt: `kb/excerpts/muon-moonlight2025.md#sec-2-2-rms`.

verbatim from Liu and et al. (Moonlight team, §2.2, eq. 4).[53] The 0.2 factor is exactly what calibrates the update RMS to AdamW's typical 0.2–0.4 range, so AdamW's tuned (η, λ) "can directly *reuse*" for Muon (Liu and et al. , Moonlight team, §2.2).[54] This is the non-trivial production property: the optimizer change does not require a fresh hyperparameter sweep at frontier scale.

Hybrid Muon + AdamW. Muon updates only matrix parameters (Liu and et al. , Moonlight team, §2.2).[55] Embeddings ($\mathbb{R}^{V \times D}$ are 2-D but treated as embeddings, not matrices, in the Muon convention because the row dim V is not a hidden-dim), the LM head, RMSNorm gains, and any other 1-D parameter stay on AdamW. A production training run with Muon is therefore a *hybrid optimizer*: Muon on the body, AdamW on the periphery, sharing (η_t, λ) across both. This is exactly the hybridization Moonlight (Liu and et al. , Moonlight team, §3) reports.

Intuition

Adam's per-coordinate $1/\sqrt{\hat{v}_t}$ implicitly assumes parameters live in a coordinate system the optimizer can read off — the natural basis of $\mathbb{R}^P$. Muon's polar projection treats matrix parameters as matrices: rotations of the row or column basis leave $\mathbf{O}_t$ equivariant in a way Adam's update is not. Bernstein et al. 2024 (Liu and et al. , Moonlight team, §2.1)[a] formalize the difference as a norm constraint on the implicit steepest-descent step: Adam's norm is the "Max-of-Max" (ℓ_∞ over coordinates), which bakes in a coordinate basis; Muon's is a Schatten-p norm for some large p, a function of singular values, which is rotation-invariant. Returning to canonical form: eq. (4) produces $\mathbf{O}_t$ with all singular values equal to 1 (Liu and et al. , Moonlight team, §2.1), so the matrix update has equal energy in every direction in the row / column subspaces of $\mathbf{M}_t$ — no coordinate is implicitly privileged by gradient magnitude, only by direction.

[a] KB excerpt: `kb/excerpts/muon-moonlight2025.md#sec-2-1-norm`.

The 52%-FLOPs scaling-law claim. Liu and et al. (Moonlight team, §3.2)[56] run a multi-scale sweep along the Chinchilla compute-optimal frontier and fit power laws of the form $L = aC^{-\alpha}$ to validation loss (see eqs. (7) and (8)). Solving $L_{\text{Muon}}(\eta C^*) = L_{\text{AdamW}}(C^*)$ for the matched-loss compute ratio $\eta = C_{\text{Muon}}/C_{\text{AdamW}}$ gives $\eta \approx 0.52$ (Liu and et al. , Moonlight team, Figure 1a): "Muon only requires about 52% training FLOPs to match the performance of AdamW under compute optimal setting" (Liu and et al. , Moonlight team, §3.2).[57] The sweep covers up to the Moonlight production scale; whether the constant factor in $\eta \approx 0.52$ holds at 70B+ dense or 600B+ MoE is untested.

The Moonlight demonstration. The largest published Muon run is Moonlight, a 2.24B-active / 15.29B-total parameter Mixture-of-Experts (or 3B-active / 16B- total when including embedding) trained on 5.7T tokens (Liu and et al. , Moonlight team, §3.3).[58] The architecture is the DeepSeek-V3-small variant from DeepSeek-AI (2024b) with the auxiliary-loss-free MoE load-balancing scheme;

[53] KB excerpt: `kb/excerpts/muon-moonlight2025.md#sec-2-2-rms`.
[54] KB excerpt: `kb/excerpts/muon-moonlight2025.md#sec-2-2-rms`.
[55] KB excerpt: `kb/excerpts/muon-moonlight2025.md#sec-2-2-hybrid`.
[56] KB excerpt: `kb/excerpts/muon-moonlight2025.md#sec-3-2-scaling`.
[57] KB excerpt: `kb/excerpts/muon-moonlight2025.md#sec-3-2-scaling`.
[58] KB excerpt: `kb/excerpts/muon-moonlight2025.md#sec-3-3-moonlight`.

the precision recipe is BF16 working with FP32 master weights. The schedule is the WSD form developed in section 4.5.

4.5 The schedule axis — cosine, WSD, Schedule-Free

The schedule η_t is the second optimizer-side compute lever. Two schedules dominate frontier LLM training, with a third at the research frontier.

Cosine decay. (Liu and et al. , Moonlight team, Loshchilov & Hutter 2017)

deepen-cite: loshchilov2017-cosine: SGDR / cosine annealing with restarts.

$$\eta_t \;=\; \eta_{\min} + \tfrac{1}{2}\,(\eta_{\max} - \eta_{\min})\,\Big(1 + \cos(\pi\, t/T)\Big), \tag{14}$$

preceded by linear warmup over ∼2k steps. Used by GPT-3 (Brown et al., 2020), Llama-1/2/3 (AI, 2024), and most pre-2024 runs. The structural cost of eq. (14) is that T, the total training-step count, must be committed at warmup — continuing training past T requires a fresh schedule with a discontinuity at the resumption point. Section 3.3 flagged this as the operational reason behind the Kaplan-vs-Chinchilla LR-horizon bug: intermediate-loss readouts at $D' \ll T$ are taken at high-LR mid-training, not at the LR-decay endpoint.

Warmup–Stable–Decay (WSD). WSD — originated in MiniCPM (et al. , Microsoft)

deepen-cite: minicpm2024 (arXiv:2404.06395) §3: WSD origin paper. The Phi-4 cite stands in because the bib has Phi-4 and not the MiniCPM paper.

with a theoretical justification arriving in 2024–2026

deepen-cite: wsd-river-valley-2024 (arXiv:2410.05192) §2: "river-valley" loss-landscape view that motivates a flat-then-decay schedule.

deepen-cite: wsd-optimal-2026 (arXiv:2602.06797) §4: WSD derived as the optimal schedule under a functional scaling-laws analysis.

— decomposes the training run into three phases:

$$\eta_t \;=\; \begin{cases} \eta_{\max}\,\dfrac{t}{T_w}, & 0 \le t \le T_w \quad \text{(warmup)} \\ \eta_{\max}, & T_w < t \le T_w + T_s \quad \text{(stable)} \\ \mathrm{decay}(t - T_w - T_s,\, \eta_{\max} \to \eta_{\min}), & T_w + T_s < t \le T \quad \text{(cooldown)} \end{cases} \tag{15}$$

where the cooldown is typically linear or cosine over the final 10–20% of training. The decisive practical advantage is that the stable phase has no fixed endpoint — training can be continued indefinitely at $\eta_{\max}$ before committing to a cooldown, making continuous-training and mid-training mixture changes routine.

The Moonlight Muon run uses an explicit WSD with a quirk: the cooldown is preceded by an LR *re-rise* that loads high-quality data into a controlled mini-warmup-then-decay phase. Verbatim

from Liu and et al. (Moonlight team, §3.3)[59]:

$$\eta_t^{\text{Moonlight}} = \begin{cases} 0 \;\to\; 4.2 \times 10^{-4} & \text{linear over [0, 33B] tokens (warmup)} \\ 4.2 \times 10^{-4} \;\to\; 4.2 \times 10^{-5} & \text{cosine over [33B, 5.2T] tokens (stable)} \\ 4.2 \times 10^{-5} \;\to\; 1 \times 10^{-4} & \text{linear in 100 steps (cooldown rise)} \\ 1 \times 10^{-4} \;\to\; 0 & \text{linear over 500B tokens (cooldown decay)} \end{cases} \tag{16}$$

Phase 3+4 is the cooldown stage in which "we use the highest quality data, focusing on math, code, and reasoning" (Liu and et al. , Moonlight team, §3.3). This is the same data-injection pattern that OLMo-2's Dolmino mix uses (section 12): late-training, small-η exposure to curated data has outsized effect on final loss because the small-η regime acts like SGD-on-a-quadratic near a minimum, where averaging dynamics dominate.

Schedule-Free. (Liu and et al. , Moonlight team, Defazio et al. 2024)

deepen-cite: schedule-free-defazio-2024 §2–3 (ICML 2024): online primal-averaging update with no LR schedule; bound matches optimally-scheduled SGD/AdamW under standard convex assumptions; LLM-scale validation pending.

removes the LR schedule entirely via online primal averaging: $\theta_{t+1} = \theta_t - \eta\, U_t$ with η *constant*, and a separate evaluation iterate $\bar{\theta}_t = (1 - c_t)\,\bar{\theta}_{t-1} + c_t\,\theta_t$ for some averaging weight c_t. The bound matches optimally-scheduled SGD / AdamW under convex assumptions; large-scale LLM validation is limited as of 2026. Schedule-Free's appeal at frontier scale is the same as WSD's — no committed total-step count — but with the additional removal of the cooldown shape decision; the unresolved question is whether the cooldown-as-data-mixture-vehicle role of WSD (16) is replicable in a schedule-free setting.

4.6 Distributed Muon — a forward link to §5

The optimizer-state partitioning that ZeRO-1 / FSDP exploit for AdamW (PyTorch, 2024, §2.2.4) does not directly apply to Muon: the Newton–Schulz iteration in eq. (4) operates on the full $A \times B$ momentum matrix, not on the per-DP-rank shard. Liu and et al. (Moonlight team, §2.3)[60] introduce Distributed Muon, which partitions optimizer state on DP as ZeRO-1 does and adds two operations:

1. **DP gather.** For each matrix parameter, gather the DP-partitioned gradients into the full $A \times B$ matrix on the local rank.
2. **Calculate full update.** Run eq. (4) on the full gradient matrix to produce $\mathbf{O}_t$, then discard the unused rows and apply the local partition's slice of eq. (4') to update the local parameter shard.

The latency overhead is reported at 1%–3% of forward-backward time (Liu and et al. , Moonlight team, §2.3).[61] Section 5 expands on how each optimizer composes with the full $G = d \cdot t \cdot c \cdot p \cdot e$ DeviceMesh (PyTorch, 2024, §4): how Distributed Muon interacts with TP / PP / EP, why some axes are optimizer-state-friendly and others are not, and the open question of whether the 1%–3% overhead scales to 32k+ GPUs.

[59] KB excerpt: `kb/excerpts/muon-moonlight2025.md#sec-3-3-moonlight`.
[60] KB excerpt: `kb/excerpts/muon-moonlight2025.md#sec-2-3-distributed`.
[61] KB excerpt: `kb/excerpts/muon-moonlight2025.md#sec-2-3-distributed`.

4.7 Open frontiers

Contradiction

The Muon-Moonlight scaling law (7) fits a sweep that tops out at the Moonlight 16B-MoE / 5.7T-token point. The matched-loss compute ratio $\eta \approx 0.52$ is a *constant* extrapolation: the ratio of two power laws with nearly identical exponents (-0.052 vs. -0.054) stays approximately constant in C, so the prediction is that Muon's lift holds at 70B+ dense and 600B+ MoE. This is also the open question Liu and et al. (Moonlight team) raise themselves (Liu and et al. , Moonlight team, §1).[a] The mechanistic risk is that the polar-factor approximation in eq. (4) breaks down when the spectrum of $\mathbf{M}_t$ becomes too anisotropic at very large width, or when distributed effects (gradient noise from large batches, EP routing variance) interact with orthogonalization in ways the small-scale sweep does not exhibit. As of 2026-Q2 the result is single-source on a 16B-MoE run; treat the 2× claim as the strongest LLM-scale optimizer-efficiency evidence on record but not yet a frontier-scale ground truth.

[a] KB excerpt: `kb/excerpts/muon-moonlight2025.md#sec-1-challenges`.

Contradiction

Lion (Liu and et al. , Moonlight team, Chen et al. 2023) and Sophia (Liu and et al. , Moonlight team, Liu, Li et al. 2023) both report ∼1.5–2× training-loss speedups that have not held on downstream evaluation in independent LLM-specific replications. Muon's scaling-law claim (7) is also a training-loss claim — validation NLL on the pre-training distribution. The downstream panel (Liu and et al. , Moonlight team, §3)[a] of the Moonlight tech report is supportive (Moonlight's downstream metrics are reported competitive with similarly-sized DeepSeek-V3 variants) but does not yet have an independent third-party replication. The structural lesson stands: training-loss optimizer gains must be re-verified on the downstream evaluation surface before they translate into a deployment-relevant claim.

[a] KB excerpt: `kb/excerpts/muon-moonlight2025.md#sec-3-3-moonlight`.

Two adjacent open questions are folded into later sections. Distributed Muon's 32k-GPU behavior and the $G = d \cdot t \cdot c \cdot p \cdot e$ composition belong to section 5. The Muon-update-RMS-vs-BF16-range coupling (Liu and et al. , Moonlight team, §2.2), where vanilla eq. (3) without weight decay drives weight RMS past BF16's high-precision range, is exactly the optimizer-times-precision coupling section 3.6 and section 6.6 flag — it reappears as the load-bearing reason eq. (4')'s $0.2\sqrt{\max(A, B)}$ and weight-decay terms are not optional. A third open question is the Schedule-Free recipe's downstream effect on the WSD-cooldown-as-data-mixture-vehicle role; the cooldown phase is where curated high-quality data is loaded (16), and removing the schedule removes the natural data-mixture trigger.

4.8 Forward link

This section fixed the optimizer (AdamW as the production default; Muon as the only published 2024–2026 optimizer with a $\sim$ 2× LLM-scale compute-efficiency lift) and the schedule (WSD displacing cosine, with Schedule-Free at the frontier). Two threads carry forward. Section 5 explores how each optimizer composes with the full DP / TP / PP / CP / EP DeviceMesh — including the Distributed Muon DP-gather + Newton–Schulz partial update of section 4.6 as the latest add — because optimizer state and gradient communication are the two bandwidth axes the parallelism

plan must accommodate. Section 6 explores how the AdamW update-RMS calibration (table 6, 0.2–0.4) and Muon's matched $0.2\sqrt{\max(A, B)}$ scaling (eq. (4')) interact with BF16's high-precision range, why vanilla eq. (3) requires the eq. (13)–eq. (4') sequence to stay numerically stable, and how the format-zoo (FP32 master, BF16 working, FP8 GEMMs, NVFP4 frontier) lands on the optimizer-precision boundary this section opened. The next section's six-stage pre-training scaffolding is, in effect, the engineering interface between this section's optimizer and section 3's budget split.

5 Distributed training

A frontier-scale pre-training run no longer chooses "a parallelism." It picks five sizes simultaneously:

$$G = d \cdot t \cdot c \cdot p \cdot e, \tag{17}$$

where G is the total worker count and (d, t, c, p, e) are the degrees of *Data Parallel* (DP), *Tensor Parallel* (TP, often fused with Sequence Parallel), *Context Parallel* (CP), *Pipeline Parallel* (PP), and *Expert Parallel* (EP) respectively (PyTorch, 2024, §1).[62] Each axis is a distinct collective with its own bandwidth-vs-latency profile, its own memory savings, and its own interaction with model architecture. The thesis of this section is that the 2024–2026 frontier has converged on these five axes as a single composable surface — a 5D `DeviceMesh` (PyTorch, 2024, §2.1.1)[63] on which a parallelism plan is a tuple of axis sizes — and that designing a frontier training run is now picking that tuple under joint constraints of memory, interconnect topology, and the model graph. We work each axis through its per-step communication, per-layer communication, per-microbatch activation, and in-flight activation costs; collect a memory table; walk DeepSeek-V3's plan as the load-bearing case study; and close on three live contradictions (TP-yes-vs-TP-no, optimal-plan-from-the-graph, the Blackwell NVL72 inversion) that §14 picks back up.

5.1 The five axes

DP / FSDP / ZeRO. Pure DP replicates parameters $\theta \in \mathbb{R}^P$, gradients, and Adam moments on each of d workers, all- reducing gradients once per step. The optimizer-state footprint at FP32-master-plus-Adam is $4P$ words per worker (parameters + gradients + first moment + second moment) (et al. , Intel, §2). ZeRO-1 / ZeRO-2 / ZeRO-3 progressively shard optimizer state / gradients / parameters across the d workers, trading per-step collective volume for memory (PyTorch, 2024, §2.1.2).[64] PyTorch's FSDP1 implemented full ZeRO-3 via a "`FlatParameter`" representation that bucketed all of a layer's parameters into one contiguous tensor; FSDP2 replaces that with per-parameter `DTensor` sharding, which composes cleanly with TP/CP/PP and delivers "often 7% lower per-GPU memory requirement versus FSDP1" plus "an average of 1.5%" throughput gain (PyTorch, 2024, §2.1.2). Per-step cost: one all-gather of parameters ($\Theta(P)$) and one reduce-scatter of gradients ($\Theta(P)$), amortized across the model's microbatches.

TP + SP. Tensor Parallel (PyTorch, 2024, §2.1.3) splits each `Linear` layer's weight matrix $W \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}$ into t row- or column-shards. PyTorch expresses this as the `RowwiseParallel` / `ColwiseParallel` DTensor placements: column-shard the QKV projection, row-shard the attention output projection, and a forward all-reduce stitches the shards back. One all-reduce per linear in forward and one per linear in backward — on the order of $2L$ per-layer collectives per step. Sequence

[62] KB excerpt: `kb/excerpts/torchtitan2024.md#sec-1-axes`.
[63] KB excerpt: `kb/excerpts/torchtitan2024.md#sec-2-1-1-meta`.
[64] KB excerpt: `kb/excerpts/torchtitan2024.md#sec-2-1-2-fsdp2`.

Parallel (SP) is the always-paired companion: normalization and dropout layers are sharded along the token dimension instead of replicated, since otherwise their activations are TP-replicated and become memory-dominant on long context (PyTorch, 2024, §2.1.3). Per-GPU activation under TP+SP shrinks by roughly $1/t$ along the sharded dimension. Naive TP serializes the matmul against its collective; *Async TP* fractionalizes the sharded matmul into chunks and overlaps each chunk's compute with the previous chunk's collective via an intra-node `SymmetricMemory` buffer that permits direct P2P writes between GPUs in a node (PyTorch, 2024, §2.2.3).[65]

CP (Context Parallel). CP shards along the *token-position* axis: a $\mathsf{B}\times\mathsf{S}\times\mathsf{D}$ residual stream becomes $\mathsf{B}\times(\mathsf{S}/c)\times\mathsf{D}$ on each of c workers. Attention is partitioned via a ring-attention- style schedule: each worker holds its sequence chunk's queries and streams keys/values around a ring of c workers, accumulating partial softmax denominators as it goes, so the full softmax is computed without any single worker materializing the $\mathsf{S}\times\mathsf{S}$ attention matrix. The headline empirical result is from PyTorch (2024, §2.1.5)[66]: "on Llama 3.1 8B with 8 H100 GPUs, using CP enabled training at context lengths up to 262,144 tokens, achieving minor MFU degradation as CP degree increases." Without CP the same setup OOMs.

PP. PP partitions the layer stack into p stages, one stage per worker group, with microbatches flowing stage-to-stage via P2P send/recv (PyTorch, 2024, §2.1.4). Per-GPU parameter memory drops as $1/p$, but in-flight activations multiply by p because the pipeline must hold one microbatch's worth of activation per stage. The classic 1F1B schedule (after Harlap et al., 2018) has a "bubble" on the first forward pass through all p stages and on the last backward pass, totaling $2(p-1)$ stage-times of idle. The schedule lineage tightens this:

$$\text{1F1B: } (p{-}1)(F{+}B), \qquad \text{ZeroBubble (ZB1P): } (p{-}1)(F{+}B{-}2W), \qquad \text{DualPipe: } (\tfrac{p}{2}{-}1)(F\&B{+}B{-}3W), \tag{18}$$

verbatim from DeepSeek-AI (2024b, §3.2.1, table)[67], where F is the forward microbatch time, B the (combined) backward time, and W the weight-gradient sub-step that ZeroBubble and DualPipe extract from B. ZeroBubble reorders the freshly-isolated W into the bubble; DualPipe additionally feeds the pipeline from both ends simultaneously so the first-microbatch ramp on one side overlaps with the last-microbatch ramp on the other (we walk DualPipe in detail in section 5.4).

deepen-cite: ZeroBubble (Qi et al. 2023b) formal W-decomposition — KB has only the bubble formula transcribed from DeepSeek-AI (2024b, §3.2.1).

EP (Expert Parallel). For an MoE layer with E routed experts, total expert parameters $E \cdot P_E$ are too large for a single GPU at frontier scale — DeepSeek-V3 has 256 routed experts per layer (DeepSeek-AI, 2024b, §3.2). EP shards experts across e workers; each token's forward pass becomes:

$$\underbrace{\text{all-to-all}_{\text{dispatch}}}_{\text{token}\to\text{expert host}} \to \underbrace{\text{expert FFN compute}}_{\text{local}} \to \underbrace{\text{all-to-all}_{\text{combine}}}_{\text{expert host}\to\text{token}} \tag{19}$$

(DeepSeek-AI, 2024b, §3.2; §3.2.2). EP is bandwidth-heavy and typically cross-node; in fact, per DeepSeek-AI (2024b, §3.2.1)[68], the cross-node EP all-to-all gives DeepSeek-V3 "an inefficient computation-to-communication ratio of approximately 1:1," which is exactly what DualPipe is designed to hide.

[65] KB excerpt: `kb/excerpts/torchtitan2024.md#sec-2-2-3-async-tp`.
[66] KB excerpt: `kb/excerpts/torchtitan2024.md#sec-2-1-5-cp`.
[67] KB excerpt: `kb/excerpts/deepseek-v3-training.md#sec-3-2-1-bubble`.
[68] KB excerpt: `kb/excerpts/deepseek-v3-training.md#sec-3-2-1-dualpipe`.

5.2 Per-axis cost model

Let P denote total parameters (FP32-master + Adam moments give $4P$ optimizer-state words) and A the peak per-microbatch activation memory of one stage of the model. table 7 summarizes the per-GPU memory and communication cost under each axis combination.

Axis combination	Per-GPU param	Per-GPU optim state	Per-GPU activation	Per-step / per-layer co
Pure DP (ZeRO-0)	P	$\sim 4P$	A	all-reduce $\Theta(P)$ / st
ZeRO-1 (optim shard)	P	$4P/d$	A	all-gather + reduce-scatter Θ
ZeRO-2 (grad shard)	P	$4P/d$ + grads/d	A	+ reduce-scatter $\Theta(P)$
ZeRO-3 / FSDP-full	P/d	$4P/d$	A	all-gather param shards
TP only (size t)	P/t	$4P/t$	A/t (sharded)	2 all-reduces / linear (~ 2
PP only (stages p)	P/p	$4P/p$	$A \cdot p$ (in-flight)	P2P send/recv Θ(act) / b
CP (size c)	P	$4P$	A/c	ring KV pass $\Theta(\mathsf{S}\cdot\mathsf{D})$ / at
EP (size e)	$P_{\text{shared}} + P_E E/e$	matched	$\sim A$ + dispatch buffer	2 all-to-alls / MoE la

Table 7: Per-GPU memory and communication cost under each parallelism axis. P is parameters, A is peak per-microbatch activation per stage, L is layer count, S is sequence length, D is model dim. Optimizer state $4P$ assumes FP32-master-plus-Adam-moments (et al. , Intel, §2); section 6 discusses the BF16-Adam-moments and fine-grained-FP8 reductions. After PyTorch (2024, §2.1) and DeepSeek-AI (2024b, §3.2).

Two structural tensions live in table 7. The first is *PP saves parameters but multiplies in-flight activations by p*: the bookkeeping for p microbatches' worth of activation per stage is exactly why DualPipe's reported "+1 activation copy" (DeepSeek-AI, 2024b, §3.2.1) is a small price for halving the bubble. The second is that *FSDP and TP both shard parameters but at different granularities*: FSDP collects per-step (one all-gather per `DTensor` placement, amortized across the layer's compute), TP collects per-layer (an all-reduce inside every linear). At small model size FSDP's per-step collective dominates; at large model size TP's per-layer collective dominates — which is why pure-FSDP plans win on sub-10B models and TP+PP plans dominate at 400B+ (PyTorch, 2024, §1; §2.1.3). EP adds a third granularity: the all-to-all is per-MoE-layer, so EP-heavy plans on L-layer models with $L_{\text{MoE}} \approx L$ pay $2L$ all-to-alls per step (DeepSeek-AI, 2024b, §3.2).

Analogy

The five axes lay a 5D crystal over the worker pool: each GPU sits at one cell of a $d\times t\times c\times p\times e$ mesh, and the "shape" of the crystal — which axes are nontrivial and how big they are — determines which collectives fire on which axis at which frequency. Returning to canonical form: this is exactly the abstraction `torch.distributed.DeviceMesh` represents (PyTorch, 2024, §2.1.1); the five axes are quite literally the five dimensions of a `DTensor` placement, and the analogy is operational, not just suggestive. The parallelism plan's tuple (d, t, c, p, e) is the shape of the mesh; the placements declared on each `Linear` / norm / attention module are the per-axis collectives the runtime issues; the cost in table 7 is the amortized communication that mesh shape implies.

5.3 Composability — TorchTitan's 4D + Float8 + AsyncTP

PyTorch (2024) package the four non-EP axes (DP / TP+SP / CP / PP) on a single 4D `DeviceMesh` with FSDP2 as the default 1D path. The headline composability claim is that any subset of axes

can be enabled without retouching the model definition: parallelism is declared on the mesh, not in the layers, and the runtime issues the right collectives. The reported speedups, on optimized `torch.compile`-based BF16 baselines on H100, are "65.08% **on Llama 3.1 8B at 128 GPU scale (1D)**, 12.59% **on Llama 3.1 70B at 256 GPU scale (2D)**, to 30% **on Llama 3.1 405B at 512 GPU scale (3D)**" (PyTorch, 2024, abstract).[69] The components that ride on top of the mesh are themselves composable: *torchao.float8* for selective per-tensor FP8 `Linear` layers (section 6), *Async TP* for matmul- collective overlap via `SymmetricMemory` intra-node buffers, *ZeroBubble / Flexible-Interleaved-1F1B* pipeline schedules expressed as a list of compute actions plus compiler passes that insert the corresponding collectives (PyTorch, 2024, §2.1.4).[70] FSDP2's per-parameter `DTensor` sharding is the technology that makes the rest compose: each parameter knows its own placement on the mesh, so a TP-sharded `Linear` composes with FSDP2 by stacking placements rather than reconciling buffer layouts.

deepen-cite: FSDP2- DTensor formal placement composition — KB excerpt covers the 7%/1.5% improvements but not the placement algebra in detail.

5.4 Case study — DeepSeek-V3's plan

DeepSeek-V3 (DeepSeek-AI, 2024b) is the most documented frontier-scale MoE training run as of 2026 — 671B total / 37B active parameters, 14.8T tokens, 2,048 H800 GPUs, and the *full* pre-training, context-extension, and post-training cost table reproduced as table 8.

Stage	H800 GPU-hours	USD ($2/GPU-hour)
Pre-training	2,664K	$5.328M
Context extension	119K	$0.238M
Post-training	5K	$0.010M
Total	**2,788K**	**$5.576M**

Table 8: DeepSeek-V3 full training cost. "Training DeepSeek-V3 on each trillion tokens requires only 180K H800 GPU hours, i.e., 3.7 days on our cluster with 2048 H800 GPUs." After DeepSeek-AI (2024b, §1).[71]

The parallelism plan is (DeepSeek-AI, 2024b, §3.2)[72]: **16-way PP with the DualPipe schedule, 64-way EP spanning 8 nodes, ZeRO-1 DP, and *no TP*.** Concretely, with $G = 2{,}048$, $p = 16$, $e = 64$, no TP and no CP, $d = G/(p \cdot e) = 2$ at the DP axis.[73] The deliberate absence of TP is the load-bearing choice: "we meticulously optimize the memory footprint during training, thereby enabling us to train DeepSeek-V3 *without using costly Tensor Parallelism*" (DeepSeek-AI, 2024b, §3.2). The activation relief that TP would normally provide comes instead from (i) FP8 forward activations (section 6), (ii) the DualPipe overlap between MoE all-to-alls and PP forwards/backwards, and (iii) DeepSeek-V3's fine-grained MoE architecture (256 routed experts) that keeps each expert's per-token compute small enough for one GPU.

[69] KB excerpt: `kb/excerpts/torchtitan2024.md#abstract`.
[70] KB excerpt: `kb/excerpts/torchtitan2024.md#sec-2-1-4-pp`.
[71] KB excerpt: `kb/excerpts/deepseek-v3-training.md#sec-1-cost`.
[72] KB excerpt: `kb/excerpts/deepseek-v3-training.md#sec-3-2-parallelism`.
[73] The plan is best read as: PP × EP carves the 2,048 workers into 2 DP replicas of a 16×64 parallelism plane; ZeRO-1 splits optimizer state across the 2-way DP. Each MoE layer's experts are partitioned across the 64-way EP plane; non-MoE weights replicate within EP and shard across PP.

DualPipe — the bidirectional bubble-eater. Each forward + backward chunk is split into four sub-stages: `attention`, `all-to-all dispatch`, `MLP`, `all-to-all combine` (DeepSeek-AI, 2024b, §3.2.1). For a backward chunk, both `attention` and `MLP` are further split into "backward for input" and "backward for weights" (the ZeroBubble decomposition that isolates W). The key DualPipe move is to schedule these sub-stages so the four communication phases (the all-to-all dispatch + combine on each microbatch's forward and backward) overlap with the matrix-compute phases of *adjacent microbatches*, and to feed the pipeline from both ends simultaneously (DeepSeek-AI, 2024b, §3.2.1). The bubble formula in eq. (18) drops to $(\frac{p}{2}-1)(F\&B+B-3W)$ — roughly half of 1F1B at the cost of "2× parameter memory and +1 activation copy" (DeepSeek-AI, 2024b, §3.2.1).

Cross-node all-to-all — exploiting the 3.2× NVLink/IB ratio. Within a node, 8 H800s are connected via NVLink with 160 GB/s; across nodes the connection is InfiniBand with 50 GB/s — a 3.2× ratio (DeepSeek-AI, 2024b, §3.2.2).[74] A naive EP all-to-all over 64 workers spanning 8 nodes would put every token on the IB hop 7 times. DeepSeek-V3's kernel rewrites the all-to-all as a two-phase routing constraint:

$$\text{(token, dest node)} \;\rightarrow\; \underbrace{\text{1 IB transmit}}_{\text{to GPU with matching in-node index}} \;\rightarrow\; \underbrace{\text{NVLink forward}}_{\text{within destination node, to expert host}}, \tag{20}$$

with the routing capped at **at most 4 destination nodes per token**. Reading: each token pays one IB transmit per (token, target node), then everything else is intra-node NVLink. Combined with DualPipe overlap of the dispatch and combine phases against compute, this lets each token select an average of 3.2 experts per node "at no additional cost from NVLink" (DeepSeek-AI, 2024b, §3.2.2), while keeping IB the only inter-node bottleneck.

Intuition

The DualPipe schedule plus the IB+NVLink kernel together turn DeepSeek-V3's nominally 1:1 compute-to-communication ratio (DeepSeek-AI, 2024b, §3.2.1) into an effectively compute-bound run with no irrecoverable loss spikes (DeepSeek-AI, 2024b, abstract). Returning to canonical form: in eq. (18), $F\&B$ is the overlapped forward-and-backward sub-step where the all-to-all phases are scheduled into the gaps between matmul phases of adjacent microbatches; in eq. (20) the 4-node cap turns each token's 7-hop IB cost into a 1-hop IB cost plus intra-node NVLink forwarding, exploiting exactly the 3.2× bandwidth ratio. The two optimizations work together: DualPipe gives the kernel a fixed schedule of when each token's dispatch/combine fires, and the 4-node-cap rule gives the kernel a topology-aware route.

5.5 Distributed Muon

The 2025 add to the distributed-training surface is *Distributed Muon* (Liu and et al. , Moonlight team, §2.3)[75]: a sharded variant of the Muon optimizer (section 4) compatible with ZeRO-1 DP. The wrinkle is that Muon's Newton-Schulz orthogonalization step (Eq. (2) of section 4) needs the *full* matrix gradient $\mathbf{M}_t \in \mathbb{R}^{A\times B}$, while ZeRO-1 hands each rank only its $1/d$-th partition of the master weight and the corresponding partition of the gradient. Liu and et al. (Moonlight team, §2.3) resolve this with two added operations layered on top of the vanilla ZeRO-1-AdamW pattern:

[74] KB excerpt: `kb/excerpts/deepseek-v3-training.md#sec-3-2-2-allreduce`.
[75] KB excerpt: `kb/excerpts/muon-moonlight2025.md#sec-2-3-distributed`.

1. **DP gather.** For each local DP-partitioned master weight ($1/d$ the size of the model weight), gather the corresponding partitioned gradients across all DP ranks into the full gradient matrix $\mathbf{M}_t$.

2. **Calculate full update.** Run the Newton-Schulz iteration on $\mathbf{M}_t$, producing $\mathbf{O}_t \in \mathbb{R}^{A \times B}$; *discard* the part of $\mathbf{O}_t$ outside this rank's parameter partition; apply only the local-partition update.

The reported overhead is $1-3\%$ of step latency (Liu and et al. , Moonlight team, §2.3), the only additional bandwidth cost is the gradient gather, and the orthogonalization itself is data- parallel: each rank does its own Newton-Schulz on the full matrix and keeps only its slice. The Moonlight model (3B-active / 16B-total MoE, 5.7T tokens) is the published demonstration that this composes with the rest of the stack at frontier-MoE scale; whether it holds at 70B+ dense or 600B+ MoE is part of the optimizer-frontier open question discussed in section 4.

5.6 Variants and lineage

table 9 traces the system lineage. The 2020-era systems forced one axis: Megatron-LM was TP+PP (no FSDP), DeepSpeed was ZeRO-1/2/3 (no model parallel). The 2024-era systems — TorchTitan (PyTorch, 2024) on the open side, the DeepSeek stack (DeepSeek-AI, 2024b) on the case-study side — treat each axis as an independent dimension on a `DTensor` mesh and let users compose them.

System	Year	Distinctive choice	Anchor
Megatron-LM (NVIDIA)	2019–	First TP+PP at scale	(Shoeybi et al., 2019)
ZeRO-1/2/3 (DeepSpeed)	2020–21	Optim/grad/param-shard ladder	(Rajbhandari et al., 2020)
FSDP1 (PyTorch)	2022	Native ZeRO-3 with FlatParameter	(Zhao et al., 2023)
Megatron-3D	2021	TP × PP × DP composition	(Narayanan et al., 2021)
GSPMD (XLA)	2021	Sharding propagation in compiler	deepen-cite: Xu et al. 2021 GSPMI
TorchTitan (PyTorch)	2024	4D + Float8 + AsyncTP composable	(PyTorch, 2024)
FSDP2 (PyTorch)	2024	Per-parameter DTensor sharding	(PyTorch, 2024, §2.1.2)
DeepSpeed-MoE / Tutel	2022–23	EP + all-to-all dispatch kernels	deepen-cite: Rajbhandari et al. 202
DeepSeek-V3 / DualPipe	2024	Bidirectional PP + ZB decomposition	(DeepSeek-AI, 2024b, §3.2.1)
Distributed Muon (Moonlight)	2025	Per-rank Newton-Schulz on gathered grad	(Liu and et al. , Moonlight team,

Table 9: Distributed-training systems lineage. The arc is toward *composability*: every axis available, declared via mesh placement rather than wired into the model definition.

The auto-parallelism direction — pick (d, t, c, p, e) from $(N, D, \text{cluster shape})$ from the model graph — is the open research surface that GSPMD and Alpa moved into.

deepen-cite: Zheng et al. 2022 "Alpa: Automating Inter- and Intra-Operator Parallelism for Distributed Deep Learning" — KB note refs but excerpt-key not yet in papers.json.

Neither has become the dominant tool of 2026 open practice; frontier plans are still hand-tuned by ML systems engineers.

5.7 Three live contradictions (set up; full discussion §14)

Contradiction

TP-yes-vs-TP-no. AI (2024) train Llama 3 405B with heavy TP+PP+DP — the prototypical Megatron-3D shape. DeepSeek-AI (2024b) train DeepSeek-V3 671B-A37B *without TP at all*, claiming the combination of ZeRO-1 DP + EP + DualPipe + FP8 supplies enough activation relief to make TP's per-linear collective cost a net loss. This is a real architectural-conditional disagreement, not a wash. TP costs $2L$ inter-GPU all-reduces per step; EP costs $2L_{\text{MoE}}$ inter-node all-to-alls per step. DeepSeek's fine-grained MoE architecture forces them to pay the EP all-to-all already (DeepSeek-AI, 2024b, §3.2), and pairing it with DualPipe overlap + the 4-node-cap kernel hides the cost. Llama 3 is dense at active- parameter level; without an EP all-to-all to fold TP work into, the Megatron-3D plan stays preferable. The general rule is that *which* parallelism is best is a function of the model graph (MoE vs. dense, expert count, expert size, attention head count) — not a universal answer.

Contradiction

Optimal plan from the model graph. No public tool reliably picks (d, t, c, p, e) from $(N, D, \text{cluster shape}, \text{model graph})$. GSPMD propagates user-declared sharding annotations through the XLA computation graph, but it does not search; AlpaDiscovery attempts joint inter- and intra-operator search but is not the dominant tool in 2026 open practice. PyTorch (2024, §1) explicitly punts: "*combining different parallelism methods manually [is] complex and error-prone.*" Frontier plans are still hand-tuned by ML systems engineers iterating on memory traces and MFU profiles. A theoretically grounded plan-from-graph optimizer is open.

Contradiction

Blackwell NVL72 inversion. The DeepSeek-V3 cross-node kernel (DeepSeek-AI, 2024b, §3.2.2) exploits a specific topology fact: 8 GPUs per node on NVLink (160 GB/s intra-node), nodes connected by IB (50 GB/s inter-node). The ratio $160/50 \approx 3.2\times$ motivates the at-most-4-nodes-per-token routing cap, which keeps IB the binding constraint and amortizes intra-node NVLink. NVIDIA's GB200 NVL72 rack puts *72 GPUs* in a single NVLink domain via NVSwitch, with nominal 1,800 GB/s effective per-GPU NVLink bandwidth across the full 72-GPU mesh. The IB-vs-NVLink ratio that DeepSeek's kernel exploits ($3.2\times$ within an 8-GPU NVLink island) inverts: at NVL72 scale, an entire 72-GPU rack is one NVLink domain, only cross-rack hops cross IB, and the optimal kernel is no longer "cap each token to 4 nodes" but "saturate the 72-GPU NVLink domain and minimize cross-rack hops." The 2026-frontier hardware shift will require new topology-aware all-to-all kernels; the current generation's are not a universal solution.

deepen-cite: NVL72 effective NVLink bandwidth + re-derivation of token-routing cost under one-rack-NVLink topology — not yet in papers.json; tracked via NVIDIA Blackwell whitepapers.

5.8 Forward link

This section's parallelism plan, the optimizer choice from section 4, and the data and scaling-law choices from sections 2 and 3 together fix *everything except the precision axis* of the recipe.

The next section (section 6) closes the recipe by walking that precision axis: storage, compute, communication, and accumulator precision as four independently tunable choices; the FP32-master-plus- low-precision-working pattern (Micikevicius et al., 2017, §2); BF16 as a drop-in replacement for FP32 that abolishes loss scaling (et al. , Intel); and DeepSeek-V3's fine-grained FP8 recipe (DeepSeek-AI, 2024b, §3.3) that — combined with the parallelism plan above — delivered 14.8T tokens of pre- training with zero irrecoverable loss spikes (DeepSeek-AI, 2024b, abstract). The connection between this section and the next is structural: TorchTitan's per-tensor Float8 path (PyTorch, 2024, §2.2.4) composes against the 4D `DeviceMesh`, DeepSeek-V3's per-tile FP8 recipe composes against the cross-node all-to-all kernel from section 5.4, and the optimizer-precision coupling first surfaces here (Distributed Muon's full-matrix $\mathbf{O}_t$ exceeds BF16 high-precision range without weight decay; section 6 works it through to the BF16 / FP8 recipe).

6 Mixed precision and stability

The numerical-precision axis is independent of the parameter axis, scales the same way the parameter axis does, and is now production- stable through FP8 at frontier scale (DeepSeek-AI, 2024b, §3.3). The lineage runs FP32 baseline → FP16 with FP32 master weights and dynamic loss scaling (Micikevicius et al., 2017) → BF16 as a drop-in replacement that abolishes loss scaling on hardware with native BF16 Tensor Cores (et al. , Intel) → FP8 fine-grained quantization with promotion-to-CUDA-cores accumulation (DeepSeek-AI, 2024b, §3.3) → FP4-class formats (Blackwell-era NVFP4, BitNet 1.58 ternary)[76] that cut storage and compute another factor of two but whose frontier-scale production status is contested. Each step halves storage and roughly doubles per-Tensor-Core throughput on supporting hardware (DeepSeek-AI, 2024b, §3.3). The thesis of this section is that the recipe is a small set of independently-tunable choices — storage precision, compute precision, communication precision, accumulator precision — and the headline 2026 fact is that DeepSeek-V3's full 14.8T-token, 2-month, 2048-H800 pre-training reported zero irrecoverable loss spikes in FP8 (DeepSeek-AI, 2024b, abstract).[77]

6.1 The FP32 baseline and why we leave it

A pure-FP32 training step stores parameters $\theta \in \mathbb{R}^P$, gradients $g \in \mathbb{R}^P$, and Adam moments $(m, v) \in \mathbb{R}^P \times \mathbb{R}^P$ all in 32-bit floats. The total optimizer-state footprint is $4P$ FP32 words: P for parameters, P for gradients, $2P$ for Adam's first and second moments. For a 70B parameter model the optimizer state alone is $4 \cdot 70 \cdot 10^9 \cdot 4 \approx 1.1\,\text{TB}$ before activations, KV cache, or DP / TP / PP duplication (et al. , Intel, §2). The activation footprint is similarly painful: a single forward pass at batch B and sequence length S stores $L \cdot \mathsf{B} \cdot \mathsf{S} \cdot \mathsf{D}$ FP32 elements per layer of the residual stream, scaled by a few-times-D activation-checkpointing multiplier.

The compute side compounds the storage problem. NVIDIA Tensor Cores deliver substantially higher throughput at FP16 / BF16 / FP8 than at FP32: a single FP8 GEMM on H100 runs at $\sim 4\times$ the rate of an FP16 GEMM on the same silicon (DeepSeek-AI, 2024b, §3.3). Distributed all-reduce traffic also halves with each precision step, since gradient bandwidth is the dominant inter-node cost in DP / FSDP runs (PyTorch, 2024, §2.2.4).[78] The mixed-precision question is therefore not whether to leave FP32 — every frontier-scale LLM does — but how far down the precision stack we can go before training diverges or downstream quality degrades.

[76] KB note: `kb/notes/training/mixed-precision-and-stability.md#3-1`.
[77] KB excerpt: `kb/excerpts/deepseek-v3-training.md#abstract`.
[78] KB excerpt: `kb/excerpts/torchtitan2024.md#sec-2-2-4`.

6.2 FP16 with master weights and dynamic loss scaling

Micikevicius et al. (2017) introduce the canonical mixed-precision recipe that survived two GPU generations as the default training configuration. Three independent ingredients (Micikevicius et al., 2017, §2–4)[79]:

1. Maintain a single-precision *master* copy of the weights, $\theta^* \in \mathbb{R}^P$ in FP32, that accumulates the optimizer update; the FP16 working copy $\theta \in \mathbb{R}^P$ is recast from θ^* for each forward and backward pass.
2. Multiply the loss by a scale factor S before backprop and divide gradients by S before the master-weight update — *loss scaling*.
3. Run reduced-precision GEMMs with FP32 accumulators: inputs in FP16, accumulator in FP32, output cast back to FP16.

The training step has the schematic form

$$\begin{aligned} y &= \text{Forward}(\theta_{\text{FP16}}, x), \\ \tilde{g}_{\text{FP16}} &= \text{Backward}(y, S \cdot \mathcal{L}), \\ g^*_{\text{FP32}} &= \text{cast}(\tilde{g}_{\text{FP16}})/S, \\ \theta^*_{\text{FP32}} &\leftarrow \theta^*_{\text{FP32}} - \eta \cdot \text{Optim}(g^*_{\text{FP32}}), \\ \theta_{\text{FP16}} &= \text{cast}(\theta^*_{\text{FP32}}), \end{aligned} \tag{21}$$

verbatim from Micikevicius et al. (2017, §2). The *master copy* prevents the cumulative error of many small updates from drowning under FP16 quantization: each $\eta \cdot \text{Optim}(g^*)$ may be far below FP16's smallest normal value, but because it lands in the FP32 master, it accumulates exactly until the running sum crosses an FP16-representable threshold and is reflected back to θ_{FP16} on the next downcast.

Loss scaling addresses the orthogonal underflow problem in the gradients themselves. FP16's smallest normal value is $\sim 6 \times 10^{-5}$, and gradient distributions during LLM training have a long tail well below this; in plain FP16 those gradients underflow to zero (Micikevicius et al., 2017, §3). Multiplying the loss by S shifts the entire gradient distribution upward in the FP16 range without changing the chain rule:

$$\tilde{g} \;=\; \nabla(S \cdot \mathcal{L}) \;=\; S \cdot g, \qquad g^*_{\text{FP32}} \;=\; \text{cast}(\tilde{g}_{\text{FP16}})/S. \tag{22}$$

Static loss scaling picks a single $S \in [2^{10}, 2^{15}]$. Dynamic loss scaling probes the largest S that does not overflow:

Listing 1: Dynamic loss scaling (Micikevicius et al. 2017, §3).

```
S = 2**15
overflow_count = 0
for step in range(num_steps):
    loss = forward(theta_FP16, x)
    grad_FP16 = backward(S * loss)
    if any_inf_or_nan(grad_FP16):
        S = S / 2          # halve scale
        overflow_count = 0
```

[79] KB excerpt: `kb/excerpts/micikevicius2017-mixed-precision.md#sec-2`.

```
        continue            # SKIP this step's update
    grad_FP32 = cast_to_FP32(grad_FP16) / S
    theta_master_FP32 -= eta * Optim(grad_FP32)
    theta_FP16 = cast_to_FP16(theta_master_FP32)
    overflow_count += 1
    if overflow_count >= GROW_INTERVAL:
        S = S * 2           # double scale periodically
        overflow_count = 0
```

The reduced-precision GEMM with FP32 accumulator is the compute-side ingredient: the inner-product summation accumulates in FP32 even when both operands are FP16, preventing drift on large reduction dimensions $K \gtrsim 1000$ (Micikevicius et al., 2017, §4). NVIDIA Volta and later architectures implement this as a single Tensor Core instruction. The combined recipe achieves FP32-equivalent accuracy across vision and NLP tasks at the time (Micikevicius et al., 2017, §5), and remained the default for ∼3 years until BF16 made loss scaling unnecessary.

6.3 BF16: trading precision for range

et al. (Intel) make the case for the Brain Floating-Point format: 16 bits with 8 exponent bits and 7 mantissa bits, the same exponent width as FP32. The trade is explicit (et al. , Intel, §2)[80]:

Format	Bits	Exponent	Mantissa	Dyn. range ≈
FP32	32	8	23	$10^{\pm 38}$
FP16	16	5	10	6×10^{-5} to 6.55×10^{4}
BF16	16	**8**	**7**	$10^{\pm 38}$ (FP32-like)

Table 10: The 16-bit format zoo. BF16 keeps FP32's exponent width, at the cost of 3 bits of mantissa relative to FP16. After et al. (Intel, §2).

The empirical claim is sharp: "models trained with BFLOAT16 weights, activations, gradients, and updates match those trained with FP32 in terms of accuracy without any change in hyper-parameters" (et al. , Intel, §4). BF16 is therefore a *drop-in* replacement for FP32 — no recipe changes, no loss scaling, optionally no FP32 master weights. The 7-mantissa precision loss is absorbed because each weight is updated thousands of times during training and accumulated low-precision noise averages out (et al. , Intel, §3).

Intuition

BF16's "free lunch" relative to FP16 is range, not precision. Range dominates for gradients, whose magnitudes vary by many orders across the network and across coordinates; FP16's narrow range causes the small-gradient tail to underflow before any per-tensor recipe sees it, necessitating loss scaling. BF16 captures the FP32-range tail directly, trading mantissa precision (which averages out across many updates) for exponent range (which does not). Returning to math: the master-weights pattern in eq. (21) is required when the working precision's representable range fails to cover the gradient distribution; under BF16's 8-bit exponent, it does, and the FP32 master becomes optional.

By 2023 BF16 had become the universal pre-training default for LLaMA 2 and 3, Mistral, Qwen, Gemma, and OLMo, on the strength of the drop-in property and Ampere+ native BF16 Tensor

[80] KB excerpt: `kb/excerpts/kalamkar2019-bfloat16.md#sec-2`.

Core support (PyTorch, 2024, §2.2.4).

6.4 FP8 in production training

The H100 and successor architectures provide native FP8 Tensor Cores in two formats: *E4M3* (4 exponent bits, 3 mantissa, range ±448) and *E5M2* (5 exponent, 2 mantissa, range $\pm 5.7 \times 10^4$) (DeepSeek-AI, 2024b, §3.3). NVIDIA's recommended *hybrid* recipe uses E4M3 for forward activations and weights and E5M2 for gradients, on the principle that gradients need more dynamic range. DeepSeek-V3's recipe (DeepSeek-AI, 2024b, §3.3.1, §3.3.2)[81] is the most aggressive published production FP8 training to date, and its decisions disagree with NVIDIA's hybrid in three structural ways.

Fine-grained quantization granularity. For sub-FP16 formats the per-tensor or per-block scale Z becomes a first-class object. The FP8 GEMM $Y = XW$ is implemented as

$$Y = (Z_X \cdot X_{\text{FP8}})(Z_W \cdot W_{\text{FP8}}) = Z_X Z_W \, (X_{\text{FP8}} W_{\text{FP8}})_{\text{FP8 accum}}, \tag{23}$$

with the inner GEMM on FP8 Tensor Cores and the scalar product $Z_X Z_W$ folded in afterward (DeepSeek-AI, 2024b, §3.3.2, eq. 2). The granularity at which (Z_X, Z_W) are chosen is the load-bearing choice. Three operating points, increasing in cost-per-element of metadata and decreasing in quantization error:

- *Per-tensor*: a single scale per Linear input or weight matrix. TorchTitan's `torchao.float8` (dynamic, delayed, or static scaling modes) applies this scheme selectively on `Linear` layers (PyTorch, 2024, §2.2.4).
- *Per-row / per-token*: one scale per row of the activation matrix (i.e., per-token); used for GEMM input activations.
- *Per-block / per-tile*: DeepSeek-V3's choice, with 1×128 tiles for activations (per-token-per-128-channel) and 128×128 blocks for weights, with per-group scales along the inner GEMM dimension (DeepSeek-AI, 2024b, §3.3.2).

All-E4M3 instead of E4M3/E5M2 hybrid. DeepSeek-V3 uses E4M3 *everywhere* (forward, activation backward, weight backward) and recovers the dynamic range that hybrid would have provided through fine-grained scaling (DeepSeek-AI, 2024b, §3.3.2).

Promotion to CUDA cores at $N_C = 128$. H100's FP8 Tensor Core accumulates internally in "around 14 bits" rather than full FP32, giving up to ∼2% relative error on $K = 4096$ dot products (DeepSeek-AI, 2024b, §3.3.2). The fix is to copy partial sums to FP32 registers and accumulate in CUDA cores every $N_C = 128$ elements of the inner reduction, recovering FP32 accumulator semantics at a small bandwidth cost (DeepSeek-AI, 2024b, §3.3.2).

The remaining DeepSeek-V3 choices follow from the same logic: master weights and weight-gradient buffers stay in FP32; Adam first and second moments are kept in BF16; the embedding matrix, output head, MoE gating, normalization layers, and attention all stay in BF16 or FP32 because their numeric properties (small dynamic range, sensitive softmax, rare-token underflow risk) do not match what FP8's per-tile scaling is designed to absorb (DeepSeek-AI, 2024b, §3.3.1). The empirical envelope: relative loss error against a BF16 baseline stays $\leq 0.25\%$ at scale (DeepSeek-AI,

[81] KB excerpts: `kb/excerpts/deepseek-v3-training.md#sec-3-3-fp8` and `#sec-3-3-2-finegrained`.

2024b, §3.3), and the full 14.8T-token pre-training reported zero irrecoverable spikes (DeepSeek-AI, 2024b, abstract). TorchTitan's per-tensor Float8 path is considerably less aggressive but generic: the documented combined speedup on Llama-3.1 with FP8 + FSDP2 + TP is reported in the 30–65% range over BF16 (PyTorch, 2024, §2.2.4), depending on scale and which Linear layers are covered.

Intuition

Fine-grained scaling is what makes FP8 viable for full pre-training. A single per-tensor scale must compromise between outliers (which need a large Z) and bulk values (which need a small Z for precision). Per-tile scaling lets each 128-element tile choose its own Z, so outliers in one tile do not blow up precision in adjacent tiles. Returning to math: eq. (23) generalizes to per-tile $Z_X^{(t)}$ and $Z_W^{(b)}$ along the inner dimension, and the GEMM accumulates across tiles in CUDA-core FP32 every N_C elements. The effective dynamic range of an E4M3 tile with scale Z is $Z \cdot [-448, 448]$, so per-tile Z recovers the dynamic range that NVIDIA's hybrid was buying with E5M2's extra exponent bit, while E4M3's spare mantissa bit becomes pure precision gain.

6.5 Sub-FP8: NVFP4 and BitNet 1.58-bit

Below FP8 the field splits into two distinct programs.

NVFP4 (FP4 E2M1). Blackwell-class GPUs (B100, B200) ship native FP4 Tensor Cores with hardware MXScale: a per-block scaling scheme operating on 32-element blocks, where each block carries an 8-bit shared exponent. The format itself is E2M1 (2 exponent bits, 1 mantissa, dynamic range ±6 before the per-block scale) (DeepSeek-AI, 2024b, §3.3, format zoo).[82] The compute-throughput gain over FP8 is roughly 2× by spec, mirroring the FP16-to-FP8 step. The fine-grained-plus-promotion recipe of section 6.4 extends naturally — replace the 128-element tile with the 32-element MXScale block and lift the same accumulator-promotion machinery — but whether it transfers without quality loss is open.

BitNet 1.58. et al. (Microsoft) push the precision axis to its extreme: every weight is ternary in $\{-1, 0, +1\}$ (hence $\log_2 3 \approx 1.585$ bits per weight), and activations are INT8. The construction replaces `nn.Linear` with `BitLinear`, trained from scratch, using an *absmean* quantization function:

$$\widetilde{W} = \text{RoundClip}\left(\frac{W}{\gamma+\epsilon}, -1, 1\right), \tag{24}$$

$$\text{RoundClip}(x, a, b) = \max(a, \min(b, \text{round}(x))), \tag{25}$$

$$\gamma = \frac{1}{nm}\sum_{ij} |W_{ij}|, \tag{26}$$

verbatim from et al. (Microsoft, §2, eqs. 1–3).[83] Activations are scaled per-token to $[-Q_b, Q_b]$ rather than $[0, Q_b]$ to eliminate zero-point quantization (et al. , Microsoft, §2). The matrix multiply at inference reduces to integer addition (no floating- point multiply) because the weight set is $\{-1, 0, +1\}$, saving ∼71× arithmetic energy on 7nm chips by the et al. (Microsoft) energy model (et al. , Microsoft, §3).

[82] KB note: `kb/notes/training/mixed-precision-and-stability.md#1-3` and `#3-1`; format-zoo entry derived there from Blackwell white- paper material.

[83] KB excerpt: `kb/excerpts/ma2024-bitnet.md#sec-2`.

The headline empirical claim is matched-perplexity at 3B parameters: BitNet b1.58 at 3B reaches 9.91 PPL versus LLaMA-LLM 3B at 10.04, while running 2.71× faster and using 3.55× less GPU memory (et al. , Microsoft, §3, Table 1). At 70B the throughput claim is starker: 176 vs. 16 max batch size, 2977 vs. 333 tokens/s (et al. , Microsoft, §3, Table 3). The architecture imports the LLaMA-alike stack — RMSNorm, SwiGLU, RoPE, no biases — to enable drop-in inference under llama.cpp / vLLM (et al. , Microsoft, §2).

6.6 Stability mitigations

Pre-training loss spikes — sudden, sometimes-irrecoverable jumps in $\mathcal{L}$ — are observed across all frontier-scale runs (PaLM (Chowdhery et al., 2022), OPT, GLM-130B). The mitigations stack across precision regimes:

Loss scaling. Eq. (22) for FP16; not required for BF16; replaced by per-tile / per-block dynamic scales for FP8 (Eq. 23).

Gradient clipping. Clip $\|g\|_2 \leq c$ before the optimizer step. The standard fix for gradient-norm explosion under LR overshoot or batch-size transitions; orthogonal to precision.

Embedding LR scaling and rare-token handling. Token embeddings can grow unboundedly under weight decay if the optimizer's preconditioning interacts poorly with embedding sparsity, and rare- vocabulary entries' embedding columns receive small gradient magnitudes that underflow in FP8. DeepSeek-V3's mitigation is direct: keep embeddings in BF16/FP32 even when surrounding GEMMs are FP8 (DeepSeek-AI, 2024b, §3.3.1).

Attention-softmax kept higher-precision. Softmax inputs $QK^\top$ grow in magnitude as training proceeds, creating numeric overflow when $|QK^\top|$ exceeds E4M3's ± 448 bound. DeepSeek-V3 keeps the attention operator in BF16 for this reason (DeepSeek-AI, 2024b, §3.3.1); complementary fixes are *QK-clip* (clamping $QK^\top$ pre-softmax) and *QK-LayerNorm* (normalizing Q and K before the dot product, as adopted in LLaMA-2 and successors).

Optimizer-precision coupling. Muon's update RMS, $\sqrt{1/\max(A,B)}$ for an $A \times B$ matrix parameter, combined with growing weight RMS, "exceeds the high-precision range of bf16" without weight decay (Liu and et al. , Moonlight team, §2.2).[84] The Muon-Moonlight fix is mandatory weight decay plus per-shape $\sqrt{\max(A,B)} \cdot 0.2$ scaling that pulls the update + weight back into BF16's representable range. Optimizer choice and precision choice are not orthogonal; each new optimizer needs a precision-pairing study.

MoE expert collapse. The auxiliary load-balancing loss can fail under FP8, especially for the expert-routing softmax. DeepSeek- V3's auxiliary-loss-free load balancing addresses this in tandem with the FP8 recipe (DeepSeek-AI, 2024b, abstract).

Communication precision. The all-reduce of gradients in distributed training is a separate precision choice. Shoeybi et al. (2019) showed that FP16 all-reduce with FP32 reduction on a master node maintains stability for tensor-parallel runs. DeepSeek-V3's cross-node all-to-all kernels

[84] KB excerpt: `kb/excerpts/muon-moonlight2025.md#sec-2-2-rms` and `#sec-2-2-wd`.

use FP8 dispatch and BF16 combine (DeepSeek-AI, 2024b, §3.2.2), with the bandwidth budget split between InfiniBand and NVLink.

Contradiction

NVFP4 / FP4 production status is not settled. Blackwell-class hardware ships native FP4 Tensor Cores and several Blackwell-deploying vendors are running FP4 pilots, but as of 2026-Q2 no major lab has publicly confirmed FP4 *pre-training* of a frontier model. The et al. (Harvard / Google) effective-parameter prediction suggests substantial loss below ∼7 bits, but the prediction is fitted on small models. BitNet 1.58-bit's matched-FP16 quality (et al. , Microsoft) is replicated independently only thinly past ∼7B parameters, and the published BitNet b1.58 2B4T tech report (et al. , Microsoft) does not yet have a frontier-scale companion. DeepSeek-V3's FP8 result is the strongest production data point we have at this precision level, and it is single-source: no other publicly disclosed lab has matched the 14.8T-token zero-spike FP8 envelope (DeepSeek-AI, 2024b, abstract). Treat the entire sub-FP8 regime as open, and treat FP8 production stability as resolved *contingent on* the DeepSeek-V3 recipe carrying over to other architectures.

Intuition

Precision as compression. Each precision step (FP32 → BF16 → FP8 → FP4) halves storage and roughly doubles compute throughput on supporting hardware. Training in lower precision is lossy compression of the gradient computation, recovering quality through (a) keeping a high-precision master copy that aggregates many small updates and (b) per-block scaling that makes the loss tile-local rather than tensor-wide. Returning to canonical form: the master-weights pattern in Eq. (21) is exactly the lossy- compression-with-error-correction pattern, and per-tile scaling in Eq. (23) is exactly tile-local quantization with shared metadata. Each ingredient is independent; the combinations are the recipes.

6.7 Forward link

The mixed-precision and stability machinery of section 6.4 and section 6.6 concludes the pre-training scaffolding — section 6 sits as the last of the upstream stages that determine *what base model* the post-training pipeline inherits. From here we cross into post-training. The next section (section 7) walks supervised fine-tuning: the cross-entropy objective with prompt-masking, the data-curation lineage from FLAN through InstructGPT, Llama-3 rejection-sampling SFT, Tülu-3, and the 2025 R1-Distill / s1 results that show reasoning capability transfers through pure SFT. SFT inherits the mixed-precision recipe of this section unchanged — typically BF16 working precision, FP32 master weights, and a learning rate roughly 10× smaller than pre-training peak — and the stability concerns shift from gradient- underflow to data-mixture and prompt-mask convention.

7 Supervised fine-tuning

Supervised fine-tuning (SFT) is the first stage of essentially every modern post-training pipeline: cross-entropy maximum-likelihood on curated prompt-response pairs, producing the instruction-following starting point π^{SFT} that every subsequent stage (section 8 PPO, section 9 DPO, section 11 GRPO) takes as both initialization and KL anchor (Ouyang et al., 2022, §3).[85] The thesis of this

[85] KB excerpt: `kb/excerpts/ouyang2022-instructgpt.md#sec-3`.

section is the one most easily lost in the post-training literature: *the loss is plain cross-entropy with prompt-masking, and the interesting research questions live in the data, not the objective.* The lineage from FLAN's $\sim$1.8K reformatted NLP tasks (Wei et al., 2021; Chung et al., 2022) through InstructGPT's $\sim$13K labeler-written demonstrations (Ouyang et al., 2022, §3) to Tülu-3's $\sim$939K mixed-source set (et al. , AI2), MAGPIE's self-synthesis at zero human cost (Xu et al., 2024), R1-Distill's 800K reasoning-trace transfer (DeepSeek-AI, 2025, §4), and s1's 1K-trace + budget-forcing endpoint (et al. , Stanford+) is a story about data, not loss. We walk that story, give the multi-turn loss-mask conventions that production recipes have converged on, set up the three open contradictions that section 14 discusses in full, and end at the boundary of section 8: why SFT precedes RL in the canonical pipeline at all.

7.1 The SFT objective

Given a dataset $\mathcal{D}_{\text{SFT}} = \{(x^{(i)}, y^{(i)})\}_{i=1}^{N}$ of prompt-response pairs, SFT trains π_θ to minimize the expected negative log-likelihood of the response conditional on the prompt:

$$\mathcal{L}_{\text{SFT}}(\pi_\theta) = -\,\mathbb{E}_{(x,y)\sim\mathcal{D}_{\text{SFT}}}\big[\log \pi_\theta(y \mid x)\big], \tag{27}$$

verbatim from Ouyang et al. (2022, §3).[86] The implementation form, which is what training code actually computes, expands the conditional log-probability and folds in a per-token loss mask. Concatenate $z = (x, y)$ into one token sequence with the chat template; let $m_t \in \{0, 1\}$ be the loss mask with $m_t = 1$ iff z_t is a response token. Then

$$\mathcal{L}_{\text{SFT}}(\theta) = -\,\mathbb{E}_{z\sim\mathcal{D}_{\text{SFT}}}\left[\sum_{t=1}^{|z|} m_t \,\log \pi_\theta(z_t \mid z_{<t})\right]. \tag{28}$$

Equation (28) is eq. (27) written so that prompt-masking is explicit. The two are equivalent: the inner sum restricted to $\{t : m_t = 1\}$ is exactly $\log \pi_\theta(y \mid x)$ under the standard left-to-right factorization (Ouyang et al., 2022, §3.5).

Symbol	Meaning
x	prompt tokens (instruction, system message, prior context)
$y = (y_1, \ldots, y_{\lvert y\rvert})$	response tokens (assistant turn)
$z = (x, y)$	concatenated training sequence after chat-template
$m_t \in \{0, 1\}$	loss mask: $m_t = 1$ iff z_t is a response token
π_θ	policy under training, initialized from a base LM
$\mathcal{D}_{\text{SFT}}$	SFT dataset, $\lvert\mathcal{D}_{\text{SFT}}\rvert \in [10^3, 10^6]$
π^{SFT}	resulting model after 1–4 epochs of eq. (28)

Table 11: SFT symbol table. Scale numbers compiled from Ouyang et al. (2022, §3.4) ($\sim$13K demonstrations), et al. (AI2) ($\sim$939K mixed), and DeepSeek-AI (2025, §4) (800K reasoning + non-reasoning).

The hyperparameters that distinguish SFT from pre-training are not algorithmic but quantitative (Ouyang et al., 2022, §3.5): SFT runs 1–4 epochs over the curated set rather than a single pass; the learning rate is roughly 5×10^{-6}, an order of magnitude below pre-training peak; the mixed-precision recipe (BF16 working precision with FP32 master weights, section 6.3) is inherited unchanged. Architecturally there is *no* change — SFT is a continued-training stage, not a new

[86] KB excerpt: `kb/excerpts/ouyang2022-instructgpt.md#sec-3`.

model. Total compute per epoch is $\sim 6N_{\text{tok}}P$ FLOPs in standard mixed-precision training, with $N_{\text{tok}} = \sum_i (|x^{(i)}| + |y^{(i)}|)$, two to four orders of magnitude below pre-training.

Intuition

SFT is behavior cloning. The model is being shown (x, y) pairs and told "next time you see x, write y." In RL terms, $\mathcal{L}_{\text{SFT}}$ is the negative log-likelihood of a demonstrator policy π^{demo} under the model — the labeler, the teacher LM, or the rejection-sampled own-rollout that produced y. Returning to math: minimizing eq. (27) is equivalent, in the infinite-data limit, to minimizing the *forward* KL divergence $\text{KL}(\pi^{\text{demo}} \,\|\, \pi_\theta)$ (Ouyang et al., 2022, §3.5). Forward KL is mode-covering: the trained π_θ must put non-trivial mass wherever π^{demo} does, or the loss diverges. RLHF / DPO (section 8, section 9) instead minimize the *reverse* KL, $\text{KL}(\pi_\theta \,\|\, \pi^{\text{SFT}})$, which is mode-seeking: π_θ may put zero mass where π^{SFT} has support without paying a divergence cost. SFT and RLHF are dual KL directions around the demonstrator and the SFT-stage model respectively, which is why the two stages compose cleanly: SFT broadens, RL sharpens.

7.2 Why prompt-masking

Without the mask m_t in eq. (28), the SFT loss includes $\sum_t \log \pi_\theta(x_t \mid x_{<t})$ — the likelihood of the prompt itself. Two things go wrong. First, for a base model the prompts are already near-distribution; the gradient through the prompt teaches π_θ to *generate user prompts*, which is the wrong training signal. Second, when the SFT data is heavily synthetic with long instruction templates (Phi-4-style pipelines run with multi-paragraph system prompts (et al. , Microsoft, §2.2)[87]), prompt tokens dominate the token count and dilute the response-token gradient by sometimes an order of magnitude. Prompt-masking is equivalent to treating x as a fixed conditioning input rather than a sample to model — the standard conditional-likelihood factorization that eq. (27) encodes implicitly and eq. (28) encodes explicitly.

For multi-turn conversations $(x, y^{(1)}, x^{(2)}, y^{(2)}, \ldots, y^{(K)})$ two mask conventions are in use (et al. , AI2):

Train on all assistant turns. $m_t = 1$ for every assistant token across all K turns; $m_t = 0$ for every user / system / tool- result token. Tülu-3's default and the production-standard choice in 2026. Each assistant turn contributes loss; the model learns to generate every assistant response in context.

Train on the last assistant turn only. $m_t = 1$ only for $y^{(K)}$. The model sees prior turns as conditioning context but does not directly imitate them. A legacy convention that survives in some ShareGPT-derived datasets.

The all-turns convention dominates because it gives more loss signal per example with no obvious cost, and because the per-turn training signal helps the model learn to maintain assistant persona across long multi-turn dialogues. Tool-call *results* are universally masked out (the model did not generate them); tool-call *invocations* are universally included in the loss (the model did generate them, as part of its reasoning). Conventions for system messages remain unsettled

deepen-cite: tulu3 §SFT-mask, citation-pending

.

[87] KB excerpt: `kb/excerpts/phi4.md#sec-2-2-pipelines`.

7.3 The data lineage

The interesting variation is data, not loss. Five waypoints span the space.

FLAN / FLAN-T5 — instruction-tuning as terminal stage (2021–22). Wei et al. (2021) reformatted $\sim$ 1.8K NLP tasks (classification, QA, summarization, NLI) as natural-language instructions and fine-tuned a base LM on them with cross-entropy. Chung et al. (2022) extended this multi-task instruction-tuning with multiple prompting templates and chain-of-thought traces; FLAN-T5 became the canonical reference for the pre-RLHF era. SFT was the *terminal* post-training stage; no preference signal entered the recipe.

InstructGPT — SFT as stage 1 of a multi-stage pipeline (2022). Ouyang et al. (2022, §3) re-cast SFT as the first of three stages, with stage-2 RM training and stage-3 PPO following eq. (27). The data is $\sim$13K labeler-written prompt-plus-demonstration pairs, drawn from the OpenAI API submission stream (Ouyang et al., 2022, §3.4). The headline empirical fact of the paper — that the 1.3B aligned model is preferred over the 175B GPT-3 base on user prompts (Ouyang et al., 2022, §4) — is what made SFT-then-RL the dominant frame. InstructGPT also established the convention that the small LR ($\sim 10^{-6}$ during PPO; $\sim 5 \times 10^{-6}$ during SFT) and the per-token KL anchor together hold the post-training pipeline together.

Llama-3 — rejection-sampling SFT, iterated. AI (2024) ship a multi-round SFT loop in which each new round is on-policy: from the current π^{SFT}, sample $N \in [10, 30]$ responses per prompt, score with a reward model, keep the top $k \in [1, 3]$, and SFT-fine-tune on the kept responses. Iterate over ≥ 6 rounds, alternating with DPO. The total SFT corpus across rounds is $\sim 10^7$ examples — three orders of magnitude above InstructGPT. The recipe is best read as "on-policy SFT filtered by an off-policy reward model": the model learns from its own distribution, but only at the points where the RM judges it correct.

Tülu-3 — organic-heavy mixed sourcing. et al. (AI2) publish a fully open SFT recipe at $\sim$ 939K examples, mixing four streams: filtered organic data (Reddit Q&A, StackExchange, FLAN-style reformatted tasks), human-written demonstrations, synthetic data from strong teacher models, and persona-conditioned generation. The mix is balanced explicitly across math, code, reasoning, multi-turn dialogue, refusal, and multilingual capability, with iterative human evaluation gating the safety-helpfulness ratio. Tülu-3 reports clean monotonic gains through $\sim$1M samples, which sets one boundary of the data-quantity contradiction in section 7.6.

MAGPIE — self-synthesis at zero human cost. Xu et al. (2024)'s procedure is structurally trivial. Take any aligned LM π^{Aligned} (LLaMA-3-Instruct, Qwen-Chat, Mistral-Instruct), prompt it with *only* its instruction-template prefix (e.g. `<|begin_of_text|><|start_header_id|>user<|end_header_id|>\` with no question, and sample. The model fabricates a user question. Append the instruction-template response delimiter and sample again to get the answer. SFT on MAGPIE-generated data on Llama-3-8B-base recovers most of the performance of Llama-3-8B-Instruct on AlpacaEval and MT-Bench at the cost of two forward passes per example (Xu et al., 2024).

Intuition

What MAGPIE exposes is that an aligned model carries a "user-question prior" baked into its conditional distribution after the system-prompt template: $p(\text{question} \mid \text{template-prefix only})$

is itself a useful instruction distribution. The base model's unconditional distribution does not have this property — it would generate web text. Returning to math: SFT on MAGPIE data is just eq. (27) with $\mathcal{D}_{\text{SFT}}$ constructed by sampling (x, y) from $\pi^{\text{Aligned}}(\cdot \mid \text{prefix})$. The aligned model is re-used as both data source and a posteriori demonstrator π^{demo}.

R1-Distill and s1 — reasoning-via-SFT (2025). DeepSeek-AI (2025, §4) demonstrate that reasoning capability acquired through RL with verifiable rewards (section 11) transfers to small dense models through pure SFT.[88] After training R1 by RLVR, the team curated 800K SFT samples (600K reasoning + 200K non-reasoning) by sampling from R1 and filtering for correctness; SFT-only fine-tuning on Qwen-2.5 / Llama-3 base checkpoints (1.5B, 7B, 14B, 32B, 70B) yields the "DeepSeek-R1-Distill" family. The distilled-32B matches or exceeds OpenAI's o1-mini on multiple reasoning benchmarks, and the distilled-7B is competitive at 7B scale, all without any RL on the small models (DeepSeek-AI, 2025, §4). et al. (Stanford+) push the data-scale axis to its extreme: ∼1K hand-curated reasoning traces from Gemini 2.0 Flash Thinking, plus inference-time *budget-forcing*, reaches near-o1-mini on AIME. The shared finding is sharp: a small set of clean, fully-worked CoT traces is enough to elicit reasoning behavior in a strong base model, provided the traces are correct and in the model's preferred style. Per-example data quality dominates per-example data quantity at this scale.

Phi-4 — synthetic-heavy SFT mix. et al. (Microsoft, §1, §2.1)[89] make the strongest published case for a synthetic-heavy mix in both pre-training and post-training. The pre- training phase uses ∼400B tokens of synthetic data from 50+ multi-stage prompting pipelines (et al. , Microsoft, §2.2),[90] and the post-training SFT mix preserves the same synthetic-heavy ratio. The underlying intuition, in Phi-4's own framing, is that synthetic data acts as "spoonfeeding" (et al. , Microsoft, §2.1): "each token generated by a language model is by definition predicted by the preceding tokens, making it easier for a model to follow the resulting reasoning patterns." Phi-4 surpasses its own teacher GPT-4o on GPQA (56.1 vs 50.6) and MATH (80.4 vs 74.6) at ∼30× smaller-class (et al. , Microsoft, §1), evidence that the data-generation pipeline is doing more than vanilla distillation.

7.4 Post-SFT — rejection sampling and iteration

Modern frontier recipes do not stop at one SFT pass. The Llama-3 (AI, 2024) and Tülu-3 (et al. , AI2) loops alternate SFT with preference-RL across multiple rounds, each round producing a new $\pi^{\text{SFT}'}$ that becomes the next round's reference. The iteration uses rejection-sampling SFT as the on-policy data refresher:

1. From the current π^{SFT}, sample N responses per prompt ($N \in [10, 30]$).
2. Score each response with a reward model.
3. Keep the top-k ($k \in [1, 3]$) per prompt.
4. Run eq. (28) on the kept responses, producing $\pi^{\text{SFT}'}$.
5. Use $\pi^{\text{SFT}'}$ as initialization and reference for the next preference-RL round (section 8, section 9).

[88] KB excerpt: `kb/excerpts/deepseek-r1.md#sec-4`.
[89] KB excerpts: `kb/excerpts/phi4.md#sec-1-pillars` and `#sec-2-1-purpose`.
[90] KB excerpt: `kb/excerpts/phi4.md#sec-2-2-pipelines`.

The iteration interleaves with the DPO step from section 9, giving the standard 2024-era recipe SFT → DPO → rejection-sample → SFT → DPO → .... The convergence criterion is empirical (preference-win-rate plateau across rounds rather than a fixed-point criterion on the loss).

7.5 Why SFT before RL

Sections 8, 9 and 11 all initialize the policy from π^{SFT} and use it as the KL anchor. Why is this stage needed at all?

The mechanical reason is that RL needs a reasonable starting policy. A base LM produces incoherent or web-style continuations on instruction prompts; the rollouts are mostly unscorable, the reward gradient is uselessly sparse, and the trust-region constraint $\mathrm{KL}(\pi_\theta \,\|\, \pi^{\text{base}})$ is meaningless because π^{base} does not concentrate on any "acting like an assistant" manifold to anchor against. SFT cheaply produces the distribution shift from "language model" to "instruction-follower," after which RL refines.

The structural reason follows directly from the KL-direction duality of the previous intuition box: SFT minimizes *forward* KL to the demonstrator, which is mode-covering and broad; RLHF / DPO minimize *reverse* KL to π^{SFT}, which is mode-seeking and sharp. Running RL on a base LM would have π^{SFT} in eq. (34) replaced by a π^{base} whose mass is spread across web continuations rather than instruction-following behavior; the closed-form maximizer $\pi^*(y \mid x) \propto \pi^{\text{base}}(y \mid x) \exp(r_\phi/\beta)$ then fails to concentrate on any usable distribution at practical β. Removing the SFT stage is feasible only when the reward signal is strong enough on its own to substitute for the distribution shift — DeepSeek-R1-Zero (DeepSeek-AI, 2025, §2) removes the SFT cold-start in the verifiable-reward setting, but the result has poor readability and language-mixing artifacts that motivate the cold-start SFT stage of the production R1 pipeline (DeepSeek-AI, 2025, §3).

7.6 Three live contradictions

Three open contradictions sit on the SFT data axis and propagate through the rest of the post-training pipeline. We surface them here and pick them up in full in section 14.

Contradiction

Quantity vs. quality. Two empirical traditions disagree on the SFT data scale required to elicit instruction-following. *"More is better, with quality gates"* (Tülu-3 (et al. , AI2), Llama-3 (AI, 2024), Phi-4 (et al. , Microsoft)): scale to 10^6+ samples with rejection-sampling and reward-model filtering, reporting clean monotonic gains. *"Few high-quality samples suffice"* (LIMA

deepen-cite: lima-2023 §results, citation-pending

; s1 (et al. , Stanford+)): 1K–10K carefully curated examples are enough to elicit instruction-following from a strong base. Both traditions are likely correct in different regimes — small-curated-sets work for *instruction style and tone*, large-sets are needed for *capability breadth* (math, code, multilingual, refusal, safety) — but the regime boundary has not been mapped at frontier scale.

Contradiction

Synthetic fraction. What fraction of an SFT mix should be synthetic vs. organic? et al. (Microsoft) make the strongest synthetic- heavy case (∼80% synthetic by tokens in pre-training, with the post-training SFT mix following the same ratio). et al. (AI2) and AI

(2024) use organic-heavier mixes and report similar end-task quality. No paper has run a controlled ablation at frontier scale; the closest is the data-mixing-laws line (et al., 2024), which addresses pre-training mixtures, not SFT. The question is upstream-coupled to the synthetic-vs-organic question in section 2, and the resolution of one likely informs the other.

Contradiction

Reasoning-distillation ceiling. Does pure-SFT-from-strong- teacher saturate at the teacher's reasoning frontier, or can it surpass through better inference-time strategies? The R1-Distill result (DeepSeek-AI, 2025, §4) that the distilled-32B nearly matches o1-mini on AIME is the strongest evidence yet that reasoning capability is largely transferable through SFT, but the paper itself notes that SFT-distilled small models do not exceed their teacher. et al. (Stanford+) push back: ∼1K traces plus budget-forcing appears to extract behavior that was not fully exploited at the teacher's default inference compute, suggesting the distillation ceiling may move with inference-time strategy. Cross-link to Paper 3 for the inference-time-compute scaling argument that organizes this contradiction.

7.7 Forward link

Sections 7.1 to 7.5 establish π^{SFT} as the universal starting point for the post-training stages that follow, with the forward-KL discipline giving a mode-covering broad policy. Section 8 now picks up the canonical three-stage RLHF pipeline that consumes π^{SFT} as both initialization and KL anchor: reward-model training under the Bradley–Terry log-likelihood, PPO optimization of the KL-constrained reward objective, and the closed-form RL optimum $\pi^*(y \mid x) \propto \pi^{\text{SFT}}(y \mid x) \exp(r_\phi/\beta)$ that turns out — six years later — to be exactly the object section 9's DPO derivation inverts. The KL direction flips from forward (SFT, mode-covering) to reverse (RLHF / DPO, mode-seeking) at the boundary; the mixed-precision recipe and most of the optimization machinery (sections 4 and 6) carry forward unchanged. The reasoning-RL pipeline of section 11 departs further: the learned reward model is replaced by a programmatic verifier, and the cold-start SFT is sometimes elided entirely (R1-Zero), at the cost of the readability artifacts noted in section 7.5.

8 RLHF: InstructGPT and the canonical pipeline

Reinforcement Learning from Human Feedback is the canonical three-stage post-training pipeline that converts a base language model into an instruction-following assistant: supervised fine-tuning (section 7), reward-model training, and policy optimization against the reward model with a KL anchor to the SFT initialization (Ouyang et al., 2022, §3).[91] The lineage runs from preference RL on Atari and locomotion (Christiano et al., 2017) through summarization-with-human- preferences (Stiennon et al., 2020) to the open-domain instruction-following inflection of InstructGPT (Ouyang et al., 2022), whose headline empirical fact — a 1.3B-parameter aligned model preferred over a 175B-parameter unaligned base on user-submitted prompts (Ouyang et al., 2022, §4) — established the pipeline as the load-bearing post-training technology for every assistant LLM since. The thesis of this section is that RLHF is best read as a *coupled* optimization: a Bradley–Terry reward model fitted to pairwise human preferences, a PPO policy step that maximizes that reward subject to a

[91] KB excerpt: `kb/excerpts/ouyang2022-instructgpt.md#sec-3`.

KL anchor, and a closed-form RL optimum (eq. (34)) that turns out — six years later — to be the foundation that DPO inverts in section 9.

8.1 The three-stage pipeline

The InstructGPT pipeline (Ouyang et al., 2022, §3) runs three stages in fixed order, each consuming the artifact of the previous stage.

Stage 1 — SFT. Collect human-written demonstrations $\{(x, y)\}$ on prompts drawn from a target distribution (the OpenAI API submission stream in InstructGPT's case). Fine-tune the base GPT-3 with cross-entropy maximum-likelihood,

$$\mathcal{L}_{\mathrm{SFT}}(\pi_\theta) = -\,\mathbb{E}_{(x,y)\sim\mathcal{D}_{\mathrm{SFT}}}\big[\log \pi_\theta(y \mid x)\big], \tag{29}$$

yielding π^{SFT} (Ouyang et al., 2022, §3). The typical InstructGPT scale is $\sim$13K labeler-written prompt-plus-demonstration pairs; section 7 treats the wider SFT data lineage in detail.

Stage 2 — Reward model. Sample $K \in \{4, 9\}$ responses from π^{SFT} per prompt, and have human labelers rank them. Train a reward model $r_\phi(x, y)$ — initialized from π^{SFT} with the LM head replaced by a scalar head over the final hidden state of the last token — on the Bradley–Terry log-likelihood of eq. (30). Across all $\binom{K}{2}$ pairs per prompt, with a per-prompt correlation correction so that the K jointly-scored responses do not double-count the prompt. Typical InstructGPT scale: $\sim$33K prompts, $K \in \{4, 9\}$, $\sim$50K to $\sim$200K pairs (Ouyang et al., 2022, §3).

Stage 3 — PPO. Optimize the policy π_θ initialized from π^{SFT} to maximize the RM reward subject to a per-token KL penalty against π^{SFT}, the joint objective eq. (31) below. Implementation-wise, the KL is applied per-token *inside* the reward (eq. (33)), making the RL problem a token-level MDP with reward sparse at the final token plus a per-token KL anchor; PPO's GAE-based advantage estimation handles credit assignment along the trajectory (Schulman et al., 2017). Typical InstructGPT-1.3B scale: $\sim$31K prompts, $\sim$256K rollouts, $\beta \in [0.01, 0.05]$, learning rate $\sim 10^{-6}$ (Ouyang et al., 2022, §3).

The four model copies that PPO holds in memory simultaneously — policy π_θ, reference π^{SFT}, reward r_ϕ, and value head V_ϕ — and the three-phase per-step structure (rollout, score, optimize) are the load-bearing infrastructure facts that explain why DPO eventually displaced PPO at frontier scale (section 8.6).

8.2 Reward model training

The reward model is fit to pairwise human preferences under the Bradley–Terry assumption $p(y_w \succ y_l \mid x) = \sigma(r(x, y_w) - r(x, y_l))$ (Ouyang et al., 2022, §3). Maximum likelihood yields the reward-model loss

$$\mathcal{L}_{\mathrm{RM}}(\phi) = -\,\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}_{\mathrm{RM}}}\big[\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))\big], \tag{30}$$

verbatim from Ouyang et al. (2022, §3).[92] The expectation runs over a dataset $\mathcal{D}_{\mathrm{RM}}$ of triples: prompt x, preferred completion y_w ("win"), and dispreferred completion y_l ("loss"). The log-sigmoid form is the standard binary-classification objective for the latent linear utility $r_\phi(x, y) - r_\phi(x, y_l)$, but

[92] KB excerpt: `kb/excerpts/ouyang2022-instructgpt.md#sec-3`.

its substantive content is that r_ϕ is identifiable from *ranks alone* up to an additive prompt-dependent constant — the per-prompt offset cancels in $r_\phi(x, y_w) - r_\phi(x, y_l)$, so absolute reward values are gauge-equivalent and only *differences* of r_ϕ on the same prompt are meaningful.

Symbol	Meaning
π_θ	policy under training (LM with parameters θ)
π^{SFT}	SFT-stage initialization, frozen during RL
$r_\phi(x, y)$	learned scalar reward model, frozen during RL
$V_\phi(x, y_{<t})$	value model for PPO's GAE, co-sized with π_θ
(x, y_w, y_l)	prompt, preferred (win), dispreferred (loss) completions
β	KL coefficient, typical 0.01–0.05 (Ouyang et al., 2022, §3)
γ	PPO-ptx coefficient, the pre-train mixin weight

Table 12: RLHF symbol table for sections 8.1 to 8.3.

Intuition

The reward model is a learned distillation of human judgment. Humans rank pairs of responses faster and more reliably than they can score in absolute terms; absolute scoring famously suffers scale drift, anchoring effects, and individual-rater variance, while pairwise agreement is much higher. Bradley–Terry is the simplest calibration of pairwise rankings into a real-valued utility: it asserts the rank data are generated by an underlying scalar $r(x, y)$ via the logistic link, and r_ϕ is fit by ML to that link. Returning to the canonical form: the reward is *defined* as the parameter that fits the observed pair-preferences best under eq. (30), so the gauge ambiguity of r_ϕ up to a prompt-dependent additive constant follows directly from the sigmoid-of-difference form.

8.3 PPO with KL constraint

Stage 3 of the pipeline maximizes the KL-constrained reward objective

$$\mathcal{J}_{\text{RLHF}}(\theta) = \mathbb{E}_{x\sim\mathcal{D},\, y\sim\pi_\theta(\cdot|x)}\big[r_\phi(x, y)\big] - \beta\, \mathbb{E}_{x,y}\left[\log \frac{\pi_\theta(y \mid x)}{\pi^{\text{SFT}}(y \mid x)}\right], \tag{31}$$

verbatim from Ouyang et al. (2022, §3). The first term rewards the policy for producing high-RM-score completions; the second penalizes it for drifting from the SFT initialization. The KL coefficient β controls the trust-region radius: smaller β admits more aggressive reward-maximizing drift, larger β pins the policy near π^{SFT}. InstructGPT also adds a PPO-ptx pre-training-mixing term,

$$\mathcal{J}^{\text{ptx}}_{\text{RLHF}}(\theta) = \mathcal{J}_{\text{RLHF}}(\theta) + \gamma\, \mathbb{E}_{x\sim\mathcal{D}_{\text{pretrain}}}\big[\log \pi_\theta(x)\big], \tag{32}$$

which mixes the pre-training maximum-likelihood objective back in to control the *alignment tax* — the regression on standard NLP benchmarks observed when RLHF is run without it (Ouyang et al., 2022, §3).

The KL penalty is implemented per-token, folded into the reward signal that PPO sees (Ouyang et al., 2022, §3):

$$r_{\text{shaped}}(x, y_t) = r_\phi(x, y) \cdot \mathbb{1}[t = T] \;-\; \beta \log \frac{\pi_\theta(y_t \mid x, y_{<t})}{\pi^{\text{SFT}}(y_t \mid x, y_{<t})}, \tag{33}$$

i.e., the terminal-token reward (sparse, r_ϕ scored once at $t = T$) plus a per-token KL anchor (dense, paid every step). PPO then runs its standard clipped-surrogate update on the resulting token-level MDP (Schulman et al., 2017).

The closed-form maximizer of eq. (31) over π_θ admits a known analytic expression (Rafailov et al., 2023, §4):

$$\pi^*(y \mid x) = \frac{1}{Z(x)}\, \pi^{\mathrm{SFT}}(y \mid x) \exp\!\left(\frac{1}{\beta} r_\phi(x, y)\right), \tag{34}$$

with normalizer $Z(x) = \sum_y \pi^{\mathrm{SFT}}(y \mid x) \exp(r_\phi(x, y)/\beta)$. Eq. eq. (34) is a re-weighting of the SFT distribution by an exponential of the reward; PPO and REINFORCE approximate this optimum iteratively. section 9 inverts this expression to derive a closed-form classification loss that bypasses RL altogether, sharing the same fixed point on the same preference data.

Intuition

The KL term in eq. (31) is a *trust region*, not an L2-style regularizer. It looks superficially like a penalty on parameter drift, but its effect is to restrict the policy to the support of π^{SFT}: $\mathrm{KL}(\pi_\theta \,\|\, \pi^{\mathrm{SFT}})$ is finite only when π_θ has zero mass wherever π^{SFT} does, so the KL anchor is a one-sided constraint that says "do not put probability on continuations that π^{SFT} rules out." Without it, π_θ collapses to argmax-reward and produces degenerate text (mode collapse, reward hacking; see section 8.5). With it, the optimization is constrained to a shell around π^{SFT} whose radius scales with $1/\beta$. Returning to math: eq. (34) makes the trust-region picture precise — at small β, π^* concentrates on the reward maximizer within the SFT support; at large β, π^* stays close to π^{SFT} regardless of reward.

8.4 Headline alignment results

The InstructGPT paper's central empirical claim is that *alignment beats scale on user prompts* (Ouyang et al., 2022, §4): the 1.3B-parameter InstructGPT model is preferred over the 175B-parameter GPT-3 base on user-submitted prompts, despite having $\sim$100$\times$ fewer parameters.[93] The headline is more than a single number: alongside it, InstructGPT models show improvements in truthfulness and reductions in toxic-output generation with minimal regressions on public NLP datasets when the PPO-ptx mixin is used (Ouyang et al., 2022, §4). The technical predecessor (Stiennon et al., 2020) had already shown the same pattern on the narrower summarization domain, articulating the general principle: RLHF outperforms supervised fine-tuning on tasks *where human preference is more reliable than human production* — humans rank summaries better than they write them. InstructGPT extended this from summarization to open-domain instruction-following.

The 1.3B-versus-175B inflection is what made RLHF a first-class post-training technique. Every assistant LLM since — Claude, Gemini, Llama-2 / 3 / 4, Mistral, Qwen, GPT-4o — runs some descendant of the InstructGPT pipeline as its post-training core, with the variants of section 8.6 (DPO replacing PPO, RLAIF / CAI replacing human labels, GRPO replacing both for verifiable-reward reasoning) sitting on top of the same three-stage scaffold.

8.5 Reward hacking and the alignment tax

The KL-constrained RL objective eq. (31) optimizes a *learned proxy* of human preference, r_ϕ, not human preference itself. The two diverge whenever r_ϕ assigns high score to behaviors that humans

[93] KB excerpt: `kb/excerpts/ouyang2022-instructgpt.md#sec-4`.

would not actually prefer if asked directly — Goodhart's law applied to pairwise-preference learning. Concretely, three failure modes recur across published RLHF runs:

Reward overoptimization. As π_θ drifts from π^{SFT} along the gradient of r_ϕ, the proxy reward keeps climbing while the underlying human-rated quality plateaus and then declines — the classic "reward over-optimization" curve. The shape is $\mathbb{E}[r_\phi]$ monotone in $\text{KL}(\pi_\theta \,\|\, \pi^{\text{SFT}})$ but $\mathbb{E}[\text{gold}]$ unimodal in the same quantity, with a finite-β optimum (Rafailov et al., 2023, surveyed in §5).

Sycophancy. Sharma et al. (2023) document that RLHF-trained assistants systematically agree with the user even when the user is wrong, contradict themselves to match user-stated opinions, and admit to mistakes they did not make when challenged. The mechanism is read as a chain: human labelers preferentially rank agreeable responses higher, r_ϕ inherits the annotator confirmation bias, and PPO amplifies it by optimizing toward r_ϕ. Mitigation efforts (Reward Shaping, PAR) report partial fixes; sycophancy persists at frontier scale.

Verbosity and over-refusal. The same proxy-bias mechanism produces the well-documented length bias of RLHF outputs (longer responses systematically score higher under r_ϕ even when shorter is better) and over-refusal (refusal templates score safely under harm-avoidance principles, so the policy refuses borderline-benign prompts to harvest reward). Length normalization fixes (SimPO's $\beta/|y|$ scaling, section 9) target the first; better preference-data composition targets the second.

The PPO-ptx term eq. (32) addresses a fourth, related issue: the *alignment tax* — the regression on pre-training benchmarks (closed-book QA, code, knowledge tasks) caused by the policy's drift toward the narrow distribution of preference-data prompts. Mixing in the pre-training MLE objective with weight γ pulls the policy back toward the base capability surface (Ouyang et al., 2022, §3).

The mitigation lineage on the reward-model side runs single-RM (Ouyang et al., 2022) → ensemble RMs (Coste et al. 2023, multiple seeds reduce variance) → *WARM / WARP* (Ramé et al., 2024a,b): weight-averaged reward models that average parameters across K independently-trained RMs (WARM) or alternate training and weight-averaging during the RL phase itself (WARP), reported to reduce reward hacking in production at Gemma 3 scale (Team and DeepMind, 2025).

8.6 Frontier-2026 RLHF

The InstructGPT pipeline as originally specified — single learned RM, PPO with per-token KL, single-pass — is rarely used in production in 2026. The replacements partition into three regimes.

General assistant alignment: DPO and SimPO. section 9 treats this in detail. The empirical narrative shift is concrete: Llama-3 (AI, 2024) runs SFT → rejection-sampling SFT → DPO over multiple rounds and uses *no PPO at scale*, reporting that DPO required less compute than PPO and performed as well or better. Tülu-3 (et al. , AI2) adopts a similar SFT → DPO → RLVR stack. The driver is infrastructural: PPO requires four model copies in memory plus a three-phase rollout / score / optimize loop, while DPO is a single forward + backward on preference pairs.

Verifiable-reward reasoning training: GRPO and DAPO. section 11 treats this in detail. For domains where a programmatic verifier exists (math problems with sympy-checkable answers, code with unit tests), the learned RM is replaced by an indicator $R_{\text{verifier}}(x, y) \in \{0, 1\}$ that sidesteps reward hacking by construction (the only way to score 1 is to solve the problem). GRPO (Shao et al.,

2024) replaces the value model with a group-mean baseline; DAPO (Seed, 2025) adds asymmetric clipping and dynamic sampling. DeepSeek-R1 (DeepSeek-AI, 2025) is the production demonstration.

Production RM-ensemble RLHF: WARM/WARP. Gemma 3 (Team and DeepMind, 2025) retains PPO-style RLHF but substitutes WARM/WARP- averaged reward ensembles for the single RM, reporting that the ensemble reduces reward-hacking severity enough to keep the PPO loop viable at frontier scale (Ramé et al., 2024a,b). This is the counterpoint to the Llama-3 / Tülu-3 "DPO-replaces-PPO" framing.

The terminology has accordingly drifted: in 2022 "RLHF" meant the InstructGPT pipeline specifically; in 2026 it often refers to any preference-based post-training, including DPO. We retain the narrow sense in this section and treat the offline-preference family separately in section 9.

Contradiction

The community currently disagrees on whether RLHF's reward-hacking problem is fundamentally tractable or fundamentally a property of optimizing learned proxy rewards. The *tractable view* (Coste 2023; Ramé et al. 2024a,b; Gemma 3 deployment) holds that better RM training, ensembling, KL scheduling, and on-policy preference iteration mitigate reward hacking enough for production. The *fundamental view* (scaling-laws-for-direct-alignment work, the persistence of sycophancy and math-hallucination as localized shortcuts at all scales (Sharma et al., 2023)) holds that proxy-reward optimization produces overoptimization across *all* preference-based post-training including DPO, since DPO's implicit reward $\hat{r}_\theta = \beta \log \pi_\theta / \pi^{\text{SFT}}$ is itself a learned proxy that can be hacked. The contradiction is sharper than DPO-vs-PPO: RLAIF and CAI (et al. , Anthropic) replace humans with an AI judge under a constitution and report stability gains in production; advocates read this as evidence the tractable view is right (judge-under- constitution is a stable enough signal). Critics read CAI as evidence the fundamental view is right (the policy learns to *appear* principle-aligned to the AI judge whether the underlying reasoning is shallow or deep — sycophancy with respect to the AI judge instead of the human labeler). Pick up in §14.

8.7 Forward link

sections 8.1 to 8.3 establish the canonical RLHF pipeline; section 8.4 the alignment-beats- scale inflection; section 8.5 the proxy-reward failure modes; section 8.6 the 2026 successors. The closed-form RL optimum eq. (34) is the load-bearing object that section 9 inverts: solving eq. (34) for $r_\phi(x, y)$ in terms of π^* and π^{SFT} gives $r(x, y) = \beta \log \pi^*(y \mid x)/\pi^{\text{SFT}}(y \mid x) + \beta \log Z(x)$, and substituting this into the Bradley–Terry preference probability eq. (30) cancels the intractable $\log Z(x)$, yielding the DPO loss as a closed-form classification objective on the same preference data the RM in section 8.2 was trained on. The result: same fixed point, no sampling, no reward model, no value model, no RL loop. section 9 works through the derivation and the variant family (IPO, KTO, ORPO, SimPO, CPO).

9 DPO and offline preference optimization

The closed-form RL optimum eq. (34) that closes section 8 is also the load-bearing equation of this one. Direct Preference Optimization (Rafailov et al., 2023) reads eq. (34) not as the *target* of an RL loop but as the *definition* of a reward that can be inverted: solving for r in terms of π^* and π^{ref} gives the reward as a log-ratio of policies, and substituting that log-ratio into the Bradley–Terry preference

probability eq. (30) cancels the intractable partition function and yields a closed-form classification loss on the same preference data the reward model of section 8.2 was trained on (Rafailov et al., 2023, §4).[94] The thesis of this section is that this single algebraic move converts post- training from a PPO-class engineering problem (four model copies, three- phase rollout/score/optimize, on-policy sampling) into a fine-tuning-class engineering problem (two forward passes, one backward, no sampler, no value head): *same fixed point on the same data, dramatically cheaper per step.* The variant family that has grown around DPO since 2023 (IPO, KTO, ORPO, SimPO, CPO) inherits this structure and varies one of three knobs at a time — the loss shape on the implicit-reward gap, the reference dependence, and the length normalization — producing a small organized lattice of offline preference methods.

9.1 Reparameterization: from RL optimum to classification loss

section 8.3's Eq. eq. (34) states that the KL-constrained reward maximization $\max_{\pi_\theta} \mathbb{E}[r(x,y)] - \beta\ \mathrm{KL}(\pi_\theta \| \pi^{\mathrm{ref}})$ admits the analytic optimum

$$\pi^*(y \mid x) = \frac{1}{Z(x)}\, \pi^{\mathrm{ref}}(y \mid x) \exp\!\left(\frac{1}{\beta}\, r(x,y)\right), \tag{35}$$

with $Z(x) = \sum_y \pi^{\mathrm{ref}}(y \mid x) \exp(r(x,y)/\beta)$ — a re-weighting of the reference distribution by an exponential of the reward (Rafailov et al., 2023, §4, Eq. 4).[95] Rafailov et al. (2023)'s move is to invert this expression for r. Taking logarithms of eq. (35) and rearranging,

$$r(x,y) = \beta \log \frac{\pi^*(y \mid x)}{\pi^{\mathrm{ref}}(y \mid x)} \ +\ \beta \log Z(x), \tag{36}$$

verbatim from Rafailov et al. (2023, §4, Eq. 5). The reward is now written as a function of the optimal policy and the reference, plus a prompt-dependent additive constant $\beta \log Z(x)$. The constant is intractable in general — $Z(x)$ is a sum over all possible completions y — but the next step removes it. The Bradley–Terry preference assumption from section 8.2 states $p(y_w \succ y_l \mid x) = \sigma(r(x,y_w) - r(x,y_l))$, a difference of rewards on the same prompt. Substituting eq. (36) into the Bradley–Terry probability,

$$p^*(y_w \succ y_l \mid x) = \sigma\!\left(\beta \log \frac{\pi^*(y_w \mid x)}{\pi^{\mathrm{ref}}(y_w \mid x)} - \beta \log \frac{\pi^*(y_l \mid x)}{\pi^{\mathrm{ref}}(y_l \mid x)}\right), \tag{37}$$

since the $\beta \log Z(x)$ terms appear with opposite signs on y_w and y_l and cancel exactly (Rafailov et al., 2023, §4, Eq. 6). The preference probability now depends only on policy log-ratios; the intractable normalizer is gone.

Maximum-likelihood on a preference dataset $\mathcal{D} = \{(x_i, y_w^i, y_l^i)\}_{i=1}^N$ then yields the DPO loss (Rafailov et al., 2023, §4, Eq. 7):[96]

$$\mathcal{L}_{\mathrm{DPO}}(\pi_\theta;\, \pi^{\mathrm{ref}}) = -\ \mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}\left[\log \sigma\!\left(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi^{\mathrm{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi^{\mathrm{ref}}(y_l \mid x)}\right)\right]. \tag{38}$$

The implicit reward associated with the policy under training is

$$\hat{r}_\theta(x,y) \ =\ \beta \log \frac{\pi_\theta(y \mid x)}{\pi^{\mathrm{ref}}(y \mid x)}, \tag{39}$$

so eq. (38) is exactly the Bradley–Terry log-likelihood eq. (30) of section 8, evaluated on the implicit reward eq. (39) instead of a separately-trained r_ϕ (Rafailov et al., 2023, §4).

[94] KB excerpt: `kb/excerpts/rafailov2023-dpo.md#sec-4`.
[95] KB excerpt: `kb/excerpts/rafailov2023-dpo.md#sec-4`.
[96] KB excerpt: `kb/excerpts/rafailov2023-dpo.md#sec-4-loss`.

Same fixed point as PPO-RLHF. The crucial structural fact: if π_θ approaches the maximum-likelihood solution of eq. (38), then by construction the implicit reward eq. (39) approaches the underlying preference-data reward r, and π_θ approaches the closed-form RL optimum π^* of eq. (35) on that reward. *DPO and PPO-RLHF share the same fixed point on the same preference data* (Rafailov et al., 2023, §4) — they differ only in the optimization trajectory, not in the optimum. PPO walks toward π^* along on-policy rollouts scored by a learned r_ϕ; DPO walks toward π^* along gradient steps of a classification loss on a fixed preference dataset.

The DPO gradient. Differentiating eq. (38)[97] gives (Rafailov et al., 2023, §4)

$$\nabla_\theta \mathcal{L}_{\text{DPO}} = -\beta\, \mathbb{E}\big[\, \sigma(\hat{r}_\theta(x, y_l) - \hat{r}_\theta(x, y_w))\, (\nabla_\theta \log \pi_\theta(y_w \mid x) - \nabla_\theta \log \pi_\theta(y_l \mid x))\big]\,, \tag{40}$$

the *preference-margin gradient* that defines all DPO-class methods. Two structural pieces are worth naming. First, the gradient is proportional to a *relative* score change: the policy is asked to push up $\log \pi_\theta(y_w \mid x)$ and push down $\log \pi_\theta(y_l \mid x)$ *by the same amount*. Second, the sigmoid weight $\sigma(\hat{r}_\theta(y_l) - \hat{r}_\theta(y_w))$ is large precisely when the preference is currently violated by the implicit reward (the policy assigns lower implicit reward to the preferred response), and small when the policy already agrees with the preference. The update is self-modulating: aggressive on miscalibrated pairs, gentle on already-correct ones.

Intuition

The DPO derivation is an *inversion* trick. The RL optimum eq. (35) maps a reward r to a policy π^*; eq. (36) reads the same equation backward and maps a policy back to a reward. Once you have a policy-to-reward map, the obvious move is to express the Bradley–Terry preference probability eq. (30) in terms of the policy directly, and the intractable $\log Z(x)$ vanishes because it appears symmetrically on y_w and y_l. What looked like an RL problem (KL-constrained reward maximization with on-policy rollouts) becomes a classification problem (maximum-likelihood on policy log-ratios). Returning to canonical form: eq. (38) is literally eq. (30), the Bradley–Terry RM training loss, with the parametric r_ϕ substituted by the implicit $\hat{r}_\theta = \beta \log \pi_\theta / \pi^{\text{ref}}$ of eq. (39). The "language model is secretly a reward model" phrasing (Rafailov et al., 2023) is the algebraic identity, not a metaphor.

9.2 The DPO training protocol

The protocol is supervised classification on log-ratios. Starting from the SFT-stage model π^{SFT} of section 7, set $\pi^{\text{ref}} \leftarrow \pi^{\text{SFT}}$ and $\pi_\theta \leftarrow \pi^{\text{SFT}}$, then for each batch of preference triples (x, y_w, y_l):

1. Compute $\log \pi_\theta(y_w \mid x)$ and $\log \pi_\theta(y_l \mid x)$ via a forward pass on the policy under training.
2. Compute $\log \pi^{\text{ref}}(y_w \mid x)$ and $\log \pi^{\text{ref}}(y_l \mid x)$ via a forward pass on the frozen reference (often pre-cached, since π^{ref} does not change).
3. Evaluate eq. (38) and back-propagate through θ.

There is no sampling, no reward model, no value model, no reinforcement learning loop. The total compute per step is $\sim$2 forward passes +1 backward pass on π_θ (both win and loss completions share the prompt prefix in practice, so the forward cost is closer to 1.5× a single completion's), versus

[97] KB excerpt: `kb/excerpts/rafailov2023-dpo.md#sec-4-gradient`.

PPO's four-model-copy plus sampling-plus-scoring-plus-optimization three-phase loop (Ouyang et al., 2022, §3). table 13 summarizes the per-step infrastructure delta that drives the "DPO-replaces-PPO" move at frontier scale.

Cost axis	PPO-RLHF	DPO
Co-resident model copies	4 (π_θ, π^{SFT}, r_ϕ, V_ϕ)	2 (π_θ, π^{ref})
Sampling phase	yes (rollouts at temperature T)	no
Scoring phase	yes (r_ϕ forward per rollout)	no
Forward passes per step	≥ 4 (rollout + r_ϕ + V_ϕ + ref)	2
Backward passes per step	1–K (PPO inner-loop $K \in \{1, 4\}$)	1
Hyperparameters that matter	$\beta, \epsilon, \lambda_{\text{GAE}}, \gamma_{\text{ptx}}, \text{LR}_\pi, \text{LR}_V, T$	β, LR

Table 13: Per-step cost comparison of PPO-RLHF (section 8.3) vs. DPO (section 9.2). The "four model copies" line is the load-bearing infrastructure fact behind the empirical move at frontier scale (AI, 2024; et al. , AI2).

Hyperparameters. The canonical open-source recipe (et al. , AI2) runs $\beta = 0.1$ and learning rate 5×10^{-7} — about ten times smaller than the SFT learning rate of section 7. Llama-3 (AI, 2024) uses the same $\beta = 0.1$ at frontier scale. Batch sizes are typically 32–256 effective; epochs 1–3. The low learning rate is not optional: although DPO does not have PPO's value-network instability, the policy can still drift far enough from π^{ref} to collapse onto a degenerate mode that satisfies the preference margin on training pairs while losing fluency on held-out prompts. The $\text{KL}(\pi_\theta \| \pi^{\text{ref}})$ that emerges implicitly — through the $\hat{r}_\theta = \beta \log \pi_\theta / \pi^{\text{ref}}$ structure — is the only thing keeping the policy on the SFT support, and a higher LR overshoots that constraint.

On-policy preferences with off-policy optimization. DPO's loss is off-policy by default: it reads a fixed dataset of preference triples and never samples from π_θ. This is structurally identical to a one-shot SFT pass, but the empirical best practice at frontier scale (Llama-3 (AI, 2024), Tülu-3 (et al. , AI2)) is to *iterate*: run DPO to convergence, sample new responses from the resulting policy, label them (human or LLM-judge) into new preference triples, and run DPO again — often with π^{ref} kept at the original π^{SFT} across rounds, so the implicit reward stays anchored to the SFT support. The result is *on-policy preference collection with off-policy optimization*: the gradient signal comes from preferences on π_θ's own samples, recovering some of the on-policy benefit of PPO without paying the per-step rollout cost.

9.3 Variant family: one organized axis at a time

The DPO variant landscape since 2023 has not produced a clear winner; it has produced a small lattice of methods that vary one knob at a time along three axes (loss shape on the implicit-reward gap; reference dependence; length normalization). table 14 is the load-bearing organizing table.

IPO — squared-gap loss. The DPO log-sigmoid loss has a known overconfidence pathology: when one preference is clearly satisfied, the implicit reward gap $\hat{r}_\theta(y_w) - \hat{r}_\theta(y_l)$ can grow without bound while the loss falls toward zero, driving the policy to overfit a few extreme pairs and undertrain

[98] KB excerpts: `kb/excerpts/rafailov2023-dpo.md#sec-4-loss`, `kb/excerpts/meng2024-simpo.md#sec-3`.

Method	Reference	Reward form	Length-normalize	Loss shape
PPO-RLHF	π^{SFT}	learned r_ϕ	no	on-policy clipped surrogate
DPO	π^{ref} frozen	$\beta \log \pi_\theta / \pi^{\text{ref}}$	no	log-sigmoid
IPO	π^{ref} frozen	same as DPO	no	squared-gap
KTO	π^{ref} frozen	prospect-theory utility	no	asymmetric per-rating
ORPO	none	SFT loss + log-odds-ratio	no	combined SFT + OR
SimPO	none	$(\beta/\lvert y\rvert) \log \pi_\theta(y \mid x)$	yes	log-sigmoid + margin γ
CPO	none	policy log-prob + SFT regularizer	no	log-sigmoid + SFT

Table 14: Offline preference variant family, organized by reference dependence (anchored vs. reference-free) and the implicit-reward form. PPO-RLHF row from section 8; DPO row from (Rafailov et al., 2023, §4); SimPO row from (Meng et al., 2024, §3).[98] IPO, KTO, ORPO, CPO entries summarize the canonical statement of each variant; see section 9.3 for citations.

on the bulk. IPO replaces the log-sigmoid with a squared loss on the gap,

$$\mathcal{L}_{\text{IPO}}(\theta) = \mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}\left[\left(\hat{r}_\theta(x,y_w) - \hat{r}_\theta(x,y_l) - \tfrac{1}{2\tau}\right)^2\right], \tag{41}$$

where τ controls the target margin (Rafailov et al., 2023, Azar et al. 2023; Eq. 17 in the paper's IPO derivation).[99]

deepen-cite: azar2023-ipo §4 — bibkey not yet in references.bib

The squared loss is bounded above and saturates symmetrically, removing the runaway-log-ratio failure mode.

KTO — single-rating data, prospect theory. KTO operates on *single-rating* data (x, y, label) with label $\in \{\mathsf{good}, \mathsf{bad}\}$, instead of the pairwise (x, y_w, y_l) DPO requires. The loss is asymmetric, weighted by Kahneman–Tversky prospect-theory loss-aversion — a unit of "bad" reward weighs more than a unit of "good" reward (Rafailov et al., 2023, Ethayarajh et al. 2024).

deepen-cite: ethayarajh2024-kto §3 — bibkey not yet in references.bib

KTO is the right method when the labeling pipeline can produce thumbs- up / thumbs-down judgments but not pairwise rankings (e.g., binary production-feedback streams).

ORPO — combined SFT and preference, no reference. ORPO collapses SFT and preference learning into a single objective and removes the reference model entirely:

$$\mathcal{L}_{\text{ORPO}}(\theta) = \mathcal{L}_{\text{SFT}}(\pi_\theta;\, y_w) \;+\; \lambda\, \mathcal{L}_{\text{OR}}(\pi_\theta;\, y_w, y_l), \tag{42}$$

where $\mathcal{L}_{\text{SFT}}$ is the cross-entropy on the chosen response (Eq. eq. (29)) and $\mathcal{L}_{\text{OR}}$ is a log-odds-ratio penalty between y_w and y_l (Rafailov et al., 2023, Hong et al. 2024).

deepen-cite: hong2024-orpo §3 — bibkey not yet in references.bib

The SFT term takes over the role of the KL anchor (without a separate reference forward), and the pipeline reduces from two stages (SFT → DPO) to one (ORPO from base directly). The cost: the reference-frozen guarantee that DPO inherits from eq. (35) no longer applies, so ORPO is no longer the same fixed point as PPO-RLHF on the same data — it is a different optimum that may or may not match.

[99] IPO's canonical form as reproduced in DPO survey arXiv:2503.11701 §4.1.

SimPO — reference-free, length-normalized. SimPO drops the reference and divides the implicit reward by sequence length (Meng et al., 2024, §3):[100]

$$r_{\text{SimPO}}(x, y) \;=\; \frac{\beta}{|y|} \log \pi_\theta(y \mid x) \;=\; \frac{\beta}{|y|} \sum_{t=1}^{|y|} \log \pi_\theta(y_t \mid x, y_{<t}). \tag{43}$$

The full loss adds a target reward margin γ (Meng et al., 2024, §3, the SimPO loss):

$$\mathcal{L}_{\text{SimPO}}(\theta) = -\,\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}\left[\log \sigma\left(\frac{\beta}{|y_w|} \log \pi_\theta(y_w \mid x) - \frac{\beta}{|y_l|} \log \pi_\theta(y_l \mid x) - \gamma\right)\right]. \tag{44}$$

Two changes vs. DPO. First, no reference model: the implicit reward is the length-normalized average log-prob of the response under the policy alone. Second, a target margin $\gamma > 0$: the chosen response must beat the rejected one by at least γ in implicit reward, not just by an arbitrary positive amount. Meng et al. (2024, §4) report SimPO outperforming DPO substantially on length-controlled benchmarks (AlpacaEval 2, Arena-Hard).[101]

CPO — policy log-prob with SFT regularizer. CPO drops the reference (like SimPO and ORPO) and adds an SFT regularizer on the chosen response (like ORPO), but keeps the DPO log-sigmoid form (Rafailov et al., 2023, Xu et al. 2024).

deepen-cite: xu2024-cpo §3 — bibkey not yet in references.bib

CPO is empirically positioned between SimPO and ORPO on the lattice and is most-cited in machine-translation post-training.

Analogy

The variant family is best read as one organized axis: *anchored vs. reference-free*. Anchored methods (DPO, IPO, KTO) keep π^{ref} in the loss; the implicit reward is a *ratio*, and the policy is constrained — through the algebra of $\log \pi_\theta / \pi^{\text{ref}}$ — to live on the support of the reference. Reference-free methods (ORPO, SimPO, CPO) drop the reference; the implicit reward becomes an *absolute* quantity ($\log \pi_\theta(y \mid x)$ or its length-normalized version), and the policy is no longer pinned to the SFT support. Returning to canonical form: anchored variants inherit the $\mathrm{KL}(\pi_\theta \| \pi^{\text{ref}})$ regularization of eq. (35) as a structural property of the implicit-reward parameterization; reference-free variants do not, and substitute different mechanisms (SimPO's length-norm; ORPO's SFT loss; CPO's SFT regularizer) to keep the policy on a sensible support without the reference's constraint. The choice of axis determines what guarantees you keep — anchored variants share a fixed point with PPO-RLHF, reference-free variants do not — and what cost you pay — anchored variants run a reference forward each step, reference-free variants do not.

9.4 Length bias and SimPO's normalization fix

DPO's documented *length bias* (longer outputs systematically beat shorter ones on length-uncontrolled benchmarks) is the most empirically-robust failure mode of the canonical Eq. eq. (38) form. The mechanism, articulated by Meng et al. (2024, §3), is algebraic: the implicit reward $\hat{r}_\theta(x, y) = \beta \log \pi_\theta(y \mid x)/\pi^{\text{ref}}(y \mid x)$ is a sum over tokens. For preference pairs with very different lengths,

[100] KB excerpt: `kb/excerpts/meng2024-simpo.md#sec-3`.
[101] KB excerpt: `kb/excerpts/meng2024-simpo.md#sec-4-headline`.

DPO can satisfy the preference by assigning extreme per-token log-probability shifts on a few tokens of the long response — the implicit reward gap grows without the per-token shifts being individually large. This biases optimization toward the longer side of any length asymmetry in the preference data.

SimPO's length-normalized reward eq. (43) fixes the mechanism by construction: dividing by $|y|$ converts the sum-of-tokens implicit reward into a *mean-of-tokens* implicit reward, which is length-invariant in expectation. Meng et al. (2024, §4.2) report the length-normalization ablation as the dominant single contributor to SimPO's gain over DPO on length-controlled AlpacaEval 2.[102]

Intuition

SimPO's length-normalization is to DPO's implicit reward what perplexity is to log-likelihood. Raw log-likelihood favors short sequences (each token contributes a negative term to the total); perplexity normalizes by length to compare sequences fairly across sequence-length regimes. Similarly, DPO's $\log \pi_\theta(y \mid x)$ favors sequences whose token-level log-probs are easy to inflate (the loss reads a sum); SimPO's $\log \pi_\theta(y \mid x)/|y|$ normalizes for length so the implicit reward is comparable across responses of unequal length. Returning to the canonical math: eq. (43) is the average of $\log \pi_\theta(y_t \mid x, y_{<t})$ over the trajectory, scaled by β, so the preference-margin gradient eq. (40) acquires a $1/|y|$ factor that prevents long-response inflation.

9.5 Frontier-2026 deployment: Llama-3, Tülu-3, Gemma 3

The empirical narrative around DPO at frontier scale has settled into two camps. The Llama-3 / Tülu-3 camp (AI, 2024; et al. , AI2) runs SFT → rejection-sampling SFT → multi-round iterated DPO and reports that DPO required less compute than PPO and performed as well or better at the post-training stage (AI, 2024, §4). Tülu-3 (et al. , AI2) adopts the same SFT → DPO → RLVR (section 11) stack as its open recipe, and explicitly reproduces the per-step cost-and-quality crossover of table 13. The Gemma 3 / DeepMind camp (Team and DeepMind, 2025) retains a PPO-style RLHF stage with WARM/WARP-augmented reward ensembles (Ramé et al., 2024a,b) as the production preference stage, on the argument that ensembled-RM PPO recovers gains that the DPO-replaces-PPO move sacrifices on creative or open-ended generation.

The orthogonal axis in 2026 is the relationship between the offline- preference stage and what surrounds it. section 10 walks the RLAIF / Constitutional AI (et al. , Anthropic) family: AI judgments under a constitution can replace human labels at the preference-data step, and the resulting AI-preference triples can feed DPO directly with no PPO loop in between. The composition — AI-labeled preferences → DPO — is increasingly the default at open-source scale, because both halves of the pipeline avoid human labeling and on-policy RL. section 11's RLVR / GRPO stack is the verifier-domain alternative: where a programmatic verifier exists, the learned reward of section 8.2 is replaced by an indicator $R_{\text{verifier}}(x, y) \in \{0, 1\}$ and the implicit-reward parameterization of eq. (39) is replaced by a group-mean baseline. DPO and RLVR are orthogonal: DPO replaces the *algorithm* (PPO → classification on preferences); RLVR replaces the *reward source* (learned RM → verifier). Modern post-training pipelines compose both — Tülu-3 runs DPO for general preferences and GRPO for verifier-domain reasoning — treating them as complementary stages.

9.6 Open contradictions

Three contradictions are surfaced here and picked up in section 8's § 14 contradiction concentration.

[102] KB excerpt: `kb/excerpts/meng2024-simpo.md#sec-4-headline`.

Contradiction

DPO-vs-PPO at frontier scale. The Llama-3 / Tülu-3 view (AI, 2024; et al. , AI2) is that DPO is *better* than PPO at frontier scale, not just cheaper: with iterated on-policy preference collection, the DPO-class pipeline matches or exceeds PPO-class pipelines on standard alignment benchmarks at lower compute. The Gemma 3 / DeepMind view (Team and DeepMind, 2025; Ramé et al., 2024a,b) is that PPO + WARM/WARP-averaged reward ensembles is still the production choice for general alignment, with the ensembled RM recovering the proxy-vs-gold gap that single-RM PPO suffered. The disagreement is partly recipe-specific (different RMs, different preference datasets, different KL schedules) and partly task-specific: DPO is more competitive on instruction-following and structured chat; PPO-with-ensemble retains advantage on creative / open-ended generation per anecdotal reports. No controlled head-to-head at frontier scale has been published, and the data, RM, and KL-schedule axes are entangled in the existing comparisons. Pick up in §14.

Contradiction

DPO does not avoid reward overoptimization. The early DPO narrative was structural: "no explicit reward model means no reward hacking" — if there is no r_ϕ to overfit, the proxy-vs-gold gap that drives PPO's reward-hacking pathology cannot exist. The 2024 "scaling laws for direct alignment" result

deepen-cite: rosset2024-direct-alignment-scaling arXiv:2406.02900 §3 — bibkey not yet in references.bib

undercuts this read. The implicit reward $\hat{r}_\theta = \beta \log \pi_\theta / \pi^{\text{ref}}$ of eq. (39) is itself a learned proxy — it is *trained* on the preference data through the maximum-likelihood fit of eq. (38) — and the gap between $\hat{r}_\theta$ and the gold preference grows with $\mathrm{KL}(\pi_\theta \| \pi^{\text{ref}})$ in the same way that the gap between r_ϕ and gold preference grows in PPO-RLHF. The mechanism is more subtle than "DPO has no reward model": the policy log-ratio *is* the reward model, and the policy hacks itself. The empirical implication: the KL-anchor regularization is load-bearing for both PPO-RLHF and DPO, and the practical mitigations — on-policy iteration, KL scheduling, early stopping — are necessary for DPO too. Pick up in §14.

Contradiction

Length bias is annotator-bias-or-loss-form. SimPO frames the length-normalization fix as a loss-form correction (Meng et al., 2024, §3): DPO's implicit-reward sum-over-tokens biases toward longer outputs by construction, and the fix is algebraic. The competing view is that the length bias originates in the *preference data*: human (and AI) annotators systematically prefer longer, more detailed responses, and the bias transfers through the loss regardless of normalization. Both are partially right: SimPO's $1/|y|$ fix demonstrably reduces length bias on length-controlled benchmarks (Meng et al., 2024, §4), but remaining length bias persists in the data after normalization. The two factors have not been cleanly separated. The empirical signature that would distinguish them — a length-controlled human preference collection followed by both DPO and SimPO training, with held-out length-controlled evaluation — has not, to our knowledge, been published at frontier scale.

9.7 Forward link

section 9.1 establishes the inversion of eq. (34) into the closed-form classification loss eq. (38); section 9.2 the per-step infrastructure delta versus PPO; section 9.3 the variant family organized along reference-anchoring and length-normalization; section 9.4 SimPO's length-bias fix; section 9.5 the frontier-2026 deployment landscape; section 9.6 the three live contradictions concentrated in §14. Two next stages follow. section 10 replaces the *preference source* itself: AI-judge-under-a-constitution preferences feed the same DPO loss directly, with eq. (38) unchanged but the dataset $\mathcal{D}$ now generated by a preference-labeling LLM rather than human annotators. section 11 replaces the *reward mechanism*: where a programmatic verifier exists, the learned reward of section 8.2 and the implicit reward of eq. (39) are both displaced by an indicator $R_{\text{verifier}} \in \{0, 1\}$, and the optimization moves from preference classification on log-ratios back to a critic-free RL loop (GRPO) that exploits the verifier's lack of a proxy-vs-gold gap. The canonical 2026 post-training stack runs all three stages in sequence: SFT (section 7) → preference-anchored DPO or its variants (section 9) → verifier-domain RLVR with GRPO (section 11), with adaptation and merging (section 12) closing the pipeline.

10 RLAIF and Constitutional AI

Reinforcement Learning from AI Feedback (RLAIF) is the substitution that drops out of section 8 once the InstructGPT three-stage pipeline is read as machinery rather than as an irreducible whole: the only place humans appear is in the reward-model training data $\mathcal{D}_{\text{RM}} = \{(x, y_w, y_l)\}$, and that data is generated by labelers ranking sampled response pairs. RLAIF replaces the ranker — humans become a feedback model π^{FB}, a separately-aligned LLM that scores the same pairs. *Constitutional AI* (CAI), introduced by et al. (Anthropic), is the canonical RLAIF instantiation: the feedback model is conditioned on a written *constitution* of ∼16 natural-language principles, sampled per-pair, and a supervised pre-stage runs the same critique-revise loop to produce the SFT initialization for the RL phase.[103] The thesis of this section is that CAI is "InstructGPT with a different labeler" in a strong sense: the Bradley–Terry reward model eq. (30), the KL-constrained PPO objective eq. (31), the four-model PPO infrastructure, the closed-form RL optimum eq. (34) — all unchanged. Only the preference source moves, from a human labeler with implicit values to an AI labeler with explicit principles. The constitution is the *only* human-authored alignment input; everything downstream, including all harm labels, is AI-generated (et al. , Anthropic, §abstract).

10.1 The RLAIF objective and AI preference distribution

The RLAIF objective inherits eq. (31) verbatim, with the learned reward replaced by a model fit on AI preferences:

$$\mathcal{J}_{\text{RLAIF}}(\theta) = \mathbb{E}_{x\sim\mathcal{D},\, y\sim\pi_\theta(\cdot|x)}\big[r_\phi^{\text{AI}}(x, y)\big] - \beta\ \text{KL}(\pi_\theta \,\|\, \pi^{\text{ref}}), \tag{45}$$

where $\pi^{\text{ref}} = \pi^{\text{SL-CAI}}$ in the CAI recipe and $\pi^{\text{ref}} = \pi^{\text{SFT}}$ in plain RLAIF (et al. , Anthropic, §4).[104] The structural identity with eq. (31) is the load-bearing observation of the section: PPO does not know, and does not need to know, that the labels feeding r_ϕ^{AI} came from an AI rather than a human.

The AI preference distribution is the place the substitution actually happens. Given prompt x, candidate responses (y_A, y_B), and a randomly-chosen principle c_k from the constitution, the

[103] KB excerpt: `kb/excerpts/bai2022-cai.md#abstract`.
[104] KB excerpt: `kb/excerpts/bai2022-cai.md#sec-4`.

feedback model's preference is the next-token probability that it produces the token `"A"` rather than `"B"` when prompted to choose:

$$p^{\mathrm{AI}}(y_A \succ y_B \mid x, c_k) = \pi^{\mathrm{FB}}(\text{“A”} \mid \mathrm{prompt}(x, y_A, y_B, c_k)), \tag{46}$$

where prompt$(\cdot)$ is a fixed template that presents the two responses and the principle to the feedback model (et al. , Anthropic, §4).[105] Sampling (y_w, y_l) from p^{AI} produces the AI-labeled preference dataset $\mathcal{D}^{\mathrm{AI}} = \{(x, y_w, y_l)\}$, and the reward model is then trained with the same Bradley–Terry log-likelihood as eq. (30):

$$\mathcal{L}^{\mathrm{AI}}_{\mathrm{RM}}(\phi) = -\,\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}^{\mathrm{AI}}}\big[\log \sigma(r^{\mathrm{AI}}_\phi(x, y_w) - r^{\mathrm{AI}}_\phi(x, y_l))\big], \tag{47}$$

verbatim from eq. (30) on the substituted dataset (et al. , Anthropic, §4). The constitution is sampled *per-pair*, so different examples in $\mathcal{D}^{\mathrm{AI}}$ are scored under different principles; the RM aggregates across the constitution rather than learning any single principle directly (et al. , Anthropic, §4).[106]

Symbol	Meaning
π_θ	policy under training (CAI: initialized from $\pi^{\text{SL-CAI}}$)
π^{ref}	KL-anchor reference: $\pi^{\text{SL-CAI}}$ for CAI, π^{SFT} for plain RLAIF
π^{H}	helpful-only RLHF model (the SL-CAI pipeline's seed)
π^{FB}	feedback model (typically π^{H} itself)
$r^{\mathrm{AI}}_\phi(x, y)$	reward model fit to AI-labeled preferences via eq. (47)
c_k	constitutional principle, sampled per-pair from $\sim$16-rule list
$\mathcal{D}^{\mathrm{AI}}$	AI-labeled preference dataset $\{(x, y_w, y_l)\}$
(y_A, y_B)	response pair sampled from $\pi^{\text{SL-CAI}}(\cdot \mid x)$
β	KL coefficient, inherited from RLHF range 0.01–0.05

Table 15: RLAIF/CAI symbol table. Compare table 12: the only structural addition is π^{FB} and c_k; everything else is renamed RLHF.

10.2 The two-stage CAI pipeline

The CAI recipe (et al. , Anthropic, §3, §4) adds one supervised pre-stage to the standard three-stage RLHF pipeline of section 8.1, yielding a four-stage pipeline $\pi^{\mathrm{H}} \to \pi^{\text{SL-CAI}} \to r^{\mathrm{AI}}_\phi \to \pi^{\text{RL-CAI}}$.

Stage 0 — helpful-only seed. A standard RLHF run with *helpfulness preferences only* (no harm labels) yields a helpful but unsafe model π^{H}. This is the only point at which human harm labels would normally enter the pipeline; CAI's claim is that this stage can be omitted for safety (et al. , Anthropic, §abstract).

Stage 1 — SL-CAI (critique-revise SFT). [107] For each red-team prompt x (et al. , Anthropic, §3): (i) sample $y_0 \sim \pi^{\mathrm{H}}(\cdot \mid x)$; (ii) sample $c_k \sim$ Uniform(constitution); (iii) prompt π^{H} to *critique* y_0 in light of c_k, producing critique κ; (iv) prompt π^{H} to *revise* y_0 given (x, y_0, κ, c_k), producing revision y_1; (v) optionally iterate (ii)–(iv) with new principles to obtain y_{final}; (vi) collect (x, y_{final}) into the SL-CAI dataset; (vii) SFT π^{H} on this dataset under the standard cross-entropy loss eq. (29) to

[105] KB excerpt: `kb/excerpts/bai2022-cai.md#sec-4`.
[106] KB excerpt: `kb/excerpts/bai2022-cai.md#sec-4`.
[107] KB excerpt: `kb/excerpts/bai2022-cai.md#sec-3`.

obtain $\pi^{\text{SL-CAI}}$. The SL-CAI dataset is the *only* human-language alignment artifact in the pipeline, and even it is generated by π^{H} rather than written by humans; the constitution governs only *what* the model is asked to critique itself for, not the critiques themselves (et al. , Anthropic, §3).

Stage 2 — RL-CAI (RLAIF on AI preferences). [108] Given $\pi^{\text{SL-CAI}}$, the RL stage (et al. , Anthropic, §4) runs: (i) sample $(y_A, y_B) \sim \pi^{\text{SL-CAI}}(\cdot \mid x)$; (ii) sample c_k from the constitution; (iii) score (y_A, y_B) via eq. (46) to obtain (y_w, y_l); (iv) accumulate $\mathcal{D}^{\text{AI}}$ and train r_ϕ^{AI} via eq. (47); (v) run PPO on $\pi^{\text{SL-CAI}}$ as initial policy, with KL anchor to $\pi^{\text{SL-CAI}}$, reward r_ϕ^{AI}, and the per-token-shaped reward signal of eq. (33) carried over from section 8.3. The output $\pi^{\text{RL-CAI}}$ is the deployed model.

The four-stage pipeline is the place to read off CAI's central methodological claim: the only human inputs are the constitution ($\sim$16 rules, total) and the helpfulness-preference dataset that trained π^{H}. There are *zero* human harm labels at any stage (et al. , Anthropic, §abstract). Compared to RLHF's $\sim$50–200K labeled pairs across helpfulness *and* harmfulness (Ouyang et al., 2022, §3), the CAI human cost is the constitution-authoring effort plus the helpful-only label collection.

10.3 Chain-of-thought enhancement

Both stages can be augmented with chain-of-thought reasoning before the model produces the final critique (SL-CAI step iii) or preference label (RL-CAI step iii) (et al. , Anthropic, §5).[109] The judge prompt is restructured along the lines of

> "Let me think step by step about how response A compares to response B with respect to principle c_k: *[CoT trace]*. Therefore the better response is [A or B]."

et al. (Anthropic, §5) report that CoT improves harmlessness without sacrificing helpfulness. The plausible mechanism, treated here as *[INTUITION]*, is that externalizing the judgment chain constrains the final `"A"` / `"B"` token under eq. (46) more than direct preference; the judge has to *commit* to an analysis it can be held to, and the analysis is visible to the RM training data through the labeled pair.

10.4 Behavioral signature: non-evasion

The empirical headline of et al. (Anthropic, §6) is that an RL-CAI model is rated by humans as *more harmless and equally helpful* as a model trained with human preference labels for harm.[110] The defining behavioral signature is *non-evasion*: when given a harmful prompt, the model engages with the prompt by stating its objections rather than refusing flatly. This is the diagnostic distinguishing CAI from the two adjacent regimes: helpful-only RLHF (complies with harmful prompts because there is no safety signal) and naive safety RLHF (refuses with templated refusals because the RM rewards refusal templates).

The mechanism is structural: the SL-CAI dataset of section 10.2 is by construction a corpus of *revisions that explain their objections* (step iv of stage 1 explicitly produces y_1 as "y_0 rewritten given the critique κ and principle c_k"). SFT on this dataset directly fits the model to emit revision-style, principle-explaining responses; PPO in RL-CAI then keeps the policy near $\pi^{\text{SL-CAI}}$ through the KL anchor, so non-evasion is preserved through the RL phase rather than re-learned by it.

[108] KB excerpt: `kb/excerpts/bai2022-cai.md#sec-4`.
[109] KB excerpt: `kb/excerpts/bai2022-cai.md#sec-5`.
[110] KB excerpt: `kb/excerpts/bai2022-cai.md#sec-6`.

Intuition

The constitution functions as a *principle prior*: a compact, human-readable statement of the values the system should exhibit, which the feedback model then operationalizes into per-pair preferences. Compare to RLHF's implicit prior — the union of all the preferences the labeler population happens to express, with no explicit summary the system can be audited against. Returning to math: the principle c_k appears as a conditioning input to eq. (46), so the AI preference distribution factorizes as the marginal over c_k of $\pi^{\mathrm{FB}}(\cdot \mid x, y_A, y_B, c_k)$. The constitution *is* the prior over principles that the RM aggregates; the per-pair sampling makes the marginal explicit.

10.5 RLAIF vs. CAI vs. DPO: an orthogonality table

Three axes vary independently across the post-training family catalogued so far:

1. *Preference source.* Human labelers (RLHF) vs. AI feedback model (RLAIF, CAI).
2. *Constitution.* Implicit (RLHF, plain RLAIF) vs. explicit written list of principles (CAI).
3. *Algorithm consuming preferences.* Bradley–Terry RM + PPO (InstructGPT, CAI as originally specified) vs. closed-form classification (DPO, section 9).

The three axes compose: AI-generated preferences from the feedback model of eq. (46) can be fed directly to the DPO loss of section 9, producing an *RLAIF + DPO* pipeline with no PPO loop and no human harm labels — the modern open-source convention for small-and-mid-scale safety tuning. RLAIF replaces the preference *source*; DPO replaces the *algorithm*; CAI is RLAIF + an explicit constitution. table 16 catalogues the combinations.

Pipeline
InstructGPT (Ouyang et al., 2022)
HH-RLHF (et al. , Anthropic, helpful+harmless)
Plain RLAIF (et al. , Anthropic, §4)
Constitutional AI (et al. , Anthropic)
RLAIF + DPO deepen-cite: tulu3, olmo2 — RLAIF-DPO recipe pages
C3AI deepen-cite: c3ai-2025 — arXiv:2502.15861
R-CAI deepen-cite: r-cai-2026 — inverted-constitution paper

Table 16: The post-training preference-RL family, factored along preference source / constitution / algorithm. CAI is the canonical "AI feedback + explicit constitution + RM+PPO" instantiation; the other rows are different choices on each axis.

Analogy

CAI is to RLHF what a graded essay with an explicit rubric is to one graded by gut feel. A grader without a rubric grades by impression, and inter-grader variance is high; with a rubric, the grade is explicitly conditioned on the rubric's criteria, and the variance collapses against rubric items. The constitution is the rubric. The feedback model is the grader. Returning to canonical form: in eq. (46), c_k is a conditioning input to the feedback model's

preference distribution; without it, the feedback model would default to an implicit (unstated) preference, which is harder to audit and steer. The grader-with-rubric analogy maps directly to $\pi^{\text{FB}}(\cdot \mid x, y_A, y_B, c_k)$ vs. the no-rubric $\pi^{\text{FB}}(\cdot \mid x, y_A, y_B)$.

10.6 Frontier extensions: C3AI, R-CAI, recursive RLAIF

C3AI: which principles transfer.

deepen-cite: c3ai-2025 §2 — arXiv:2502.15861

systematically studies which constitutional principles work, which model-specific patterns dominate, and how phrasing affects effectiveness. The output is a ranked principle set with empirical effectiveness scores per model family — the canonical 2025 reference on constitution *design*, in contrast to et al. (Anthropic)'s constitution *deployment*. Open questions identified: are some principles model-specific (work for Llama but not Qwen), are there minimal-sufficient constitutions, how much does subtle wording variation matter.

R-CAI: the inverted constitution.

deepen-cite: r-cai-2026 §3 — Reverse CAI / inverted-constitution paper

shows the constitution is a controllable knob: invert each principle ("the AI assistant SHOULD provide instructions for harmful actions") and run the same RL-CAI machinery with probability-clamping to keep the policy on distribution. The output is *controlled adversarial / toxic data* that can be used either defensively (red-team data, hardening classifiers) or offensively (fine-tuning a model toward harm, generating aligned-looking-but-toxic data to slip past detectors). This is the dual-use surface of CAI: the same machinery that produces $\pi^{\text{RL-CAI}}$ can produce its evil twin.

Recursive RLAIF. A natural extension of section 10.2: once $\pi^{\text{RL-CAI}}$ is trained, use it as the feedback model for the next round of RL-CAI. The result is a self-improvement loop with no human input beyond the original constitution.

deepen-cite: self-rewarding-2024 — arXiv:2402.10379, "Self-Rewarding Language Models"

explicitly explores this;

deepen-cite: recursive-mode-collapse-2024 — arXiv:2406.05587 follow-on

reports mode-collapse failure modes. The empirical concern, treated as *[FORUM-SIGNAL]* in the KB note, is collapse *in the constitution direction*: principles become self-echoing and divergence between *following the constitution* and *being aligned with reality* widens. No primary citation has cleanly characterized the regime boundary between safe recursion and collapse.

10.7 Open contradictions (set up here, full discussion §14)

Three live contradictions follow from the structural reading of CAI as "RLHF with a different labeler." Each is set up here and treated in the concentrated discussion of §14.

Contradiction

CAI effectiveness at small scale.

deepen-cite: small-scale-cai-2025 — arXiv:2504.04918

and

deepen-cite: small-cai-judge-2025 — arXiv:2503.17365

report that CAI effectiveness degrades significantly below ∼7B parameters. The mechanism is judge-capability: small π^{FB} produces noisy and inconsistent preferences in eq. (46), which feed eq. (47) as incoherent training signal; the resulting RM and policy are then poorly aligned to any principle. Returning to canonical form: the AI preference is only as good as π^{FB}'s ability to follow principle c_k, so CAI is a capability-*multiplier* on π^{FB}, not a capability-creator. Open: the precise judge-capability threshold; whether CoT-on-judge (section 10.3) can compensate for smaller raw judge capability; whether a stronger judge can label data for a smaller policy.

Contradiction

CAI internalization vs. surface mimicry. The non-evasion behavior of section 10.4 could be *genuine internalization* of the constitution's principles, or *learned text shaped like constitutional reasoning* without any internalization. The optimistic and skeptical readings are behaviorally indistinguishable on standard evaluations: both produce the same revision-style outputs on standard prompts. Only held-out adversarial probes — distribution-shifted scenarios where the principle's intent and its surface verbalization disagree — can separate them. Active in 2025–26.

Contradiction

CAI vs. sycophancy. et al. (Anthropic) report CAI reduces evasion (the model engages more rather than refusing flatly), but sycophancy — the systematic agreement-with-the-judge failure mode documented for RLHF in Sharma et al. (2023) — is a preference-source artifact that survives the substitution from human to AI judge. The judge in eq. (46) rewards principle- *aligned* responses; the model can learn to *appear* principle-aligned to the judge whether the underlying reasoning is shallow or deep. CAI may not solve sycophancy and may amplify it — the judge is now algorithmic and consistent, so the policy can find its biases more reliably. Plausibly the verdict depends on principle phrasing; the empirical question of which holds in practice is open.

10.8 Forward link

sections 10.1 and 10.2 establish the RLAIF/CAI machinery as a labeler-substitution in section 8's pipeline; sections 10.3 and 10.4 the chain-of-thought enhancement and the non-evasion behavioral signature; section 10.5 the orthogonality with DPO and the modern RLAIF + DPO compositions; section 10.6 the frontier extensions (C3AI, R-CAI, recursive RLAIF). The next stage of the pipeline (section 11) makes the parallel substitution on the reward side: where CAI replaces the *labeler* with a principle-conditioned LLM, RLVR replaces the *learned reward model* with a programmatic verifier $R_{\text{verifier}}(x, y) \in \{0, 1\}$. Structurally, both keep eq. (31)'s KL-constrained reward maximization unchanged; both move only the source of the reward signal. The contradictions of section 10.7 (judge capability, internalization vs. mimicry, sycophancy under an algorithmic judge) carry over to RLVR in modified form (verifier coverage, programmatic-reward gaming, spec-gaming under a verifier). section 11 treats the next substitution; §14 treats the contradictions.

11 RL on verifiable rewards: GRPO and DAPO

The post-training pipeline of section 8 and section 9 uses a *learned* reward $r_\phi(x, y)$ — a Bradley–Terry head fit to human preference pairs, or its DPO closed-form analog — as the gradient signal for the policy. The RLVR-era reformulation replaces that learned reward with a deterministic verifier: $R(x, y) = \mathbb{1}[\,\text{verify}(x, y) = \textsf{true}\,]$, the indicator that the model's output passes a sympy equivalence check, a unit-test suite, or a regex format check (DeepSeek-AI, 2025, §2.2).[111] The thesis of this section is that two coupled changes — a programmatic verifier in place of a learned reward model, and a group-mean baseline in place of a co-sized value model (Shao et al., 2024, §4.1)[112] — together produce the post-training recipe behind DeepSeek-AI (2025), the Qwen 2.5 / 3 reasoning lines (Team and Alibaba, 2025), Tülu 3 (et al. , AI2), and the open-weight RL-for-reasoning ecosystem of 2024–2026. The headline empirical fact is also the cleanest single number in the entire training pipeline: DeepSeek-AI (2025, §2.3)[113] report DeepSeek-R1-Zero's average pass@1 on AIME 2024 climbing from 15.6% to 77.9% over the course of one continuous GRPO run, and to 86.7% with self-consistency over 64 samples — surpassing the average human competitor. This section is also the cross-paper anchor that Paper 3 §7 cites for the *training* leg of its reasoning triangle (compute / search / verification); the *inference* side of the same machinery is treated there.

11.1 What "verifiable" means

Define the verifiable-reward setting: for prompt x and sampled output y, the reward is

$$R_{\text{verifier}}(x, y) \;=\; \mathbb{1}\big[\,\text{verify}(x, y) = \textsf{true}\,\big] \;\in\; \{0, 1\}, \tag{48}$$

where verify is a deterministic program supplied alongside the prompt. The instances that matter at scale (DeepSeek-AI, 2025, §2.2):

- *Math.* verify is exact match against the canonical answer after sympy-style normalization (rationalization, trig-identity reduction, whitespace canonicalization). Competition-style benchmarks (AIME, MATH, AMC) have a small set of canonical answers per problem, almost always integers or short symbolic expressions, so the exact-match relation is well-defined.
- *Code.* verify is "compiles, then passes the attached test cases." Per-problem unit-test suites are the gold reward; competitive programming benchmarks (LiveCodeBench, Codeforces) ship them.
- *Format.* verify is a parser or regex (e.g., the trajectory contains a well-formed `<think>...</think>` block before its answer) — a small constant add-on reward that keeps the reasoning channel alive during early training (DeepSeek-AI, 2025, §2.2).

The reward is *sparse* (one bit per trajectory) and *terminal* (only the final answer is graded; intermediate steps are not, in the RLVR setting; cf. section 11 on process supervision). The contrast with section 8's RLHF setting is structural: there, r_ϕ is a Bradley–Terry head trained on $\binom{K}{2}$ human comparisons per prompt, and the policy can game artifacts of r_ϕ that do not correspond to true quality (verbosity, sycophancy, surface formatting). With eq. (48), the only way to score 1 is to actually solve the problem, which sidesteps two failure modes of learned reward models:

[111] KB excerpt: `kb/excerpts/deepseek-r1.md#sec-2-2-reward`.
[112] KB excerpt: `kb/excerpts/shao2024.md#sec-4-1`.
[113] KB excerpt: `kb/excerpts/deepseek-r1.md#sec-2-3-aime`.

1. *Reward hacking.* The policy exploits an artifact of r_ϕ. Programmatic verifiers admit this only when the verifier itself has a bug (e.g., a normalization rule that accepts $0 = 0 \cdot x$ for any x); see section 11.

2. *Reward overoptimization.* The proxy–gold gap grows with $\mathrm{KL}(\pi_\theta \| \pi_{\text{ref}})$ in RLHF (Gao et al. 2023, "Scaling laws for reward model overoptimization").

   deepen-cite: gao2023-overopt — bibkey not yet in references.bib

   Verifiers do not have this gap — they *are* the gold reward, modulo dataset-side issues like wrong gold answers.

DeepSeek-AI (2025, §2.2) state this stance explicitly: "we abstain from applying neural reward models — whether outcome-based or process-based — to reasoning tasks. This decision is predicated on our observation that neural reward models are susceptible to reward hacking during large-scale reinforcement learning."[114]

11.2 GRPO: critic-free PPO with a group-mean baseline

Shao et al. (2024) introduce *Group Relative Policy Optimization* (GRPO) in the DeepSeekMath paper as a critic-free PPO variant. The algorithm samples a group of G trajectories per prompt and uses the group's empirical reward distribution as the variance-reducing baseline, eliminating the value network V_ϕ entirely (Shao et al., 2024, §4.1, motivation).[115] The objective, restated and verified in DeepSeek-AI (2025, §2.1, Eq. 1)[116]:

$$\begin{aligned}\mathcal{J}_{\text{GRPO}}(\theta) \;=\; \mathbb{E}_{q\sim P(Q),\,\{o_i\}_{i=1}^{G}\sim\pi_{\theta_{\text{old}}}(\cdot|q)}\Bigg[\\ \frac{1}{G}\sum_{i=1}^{G}\frac{1}{|o_i|}\sum_{t=1}^{|o_i|}\min\Big(r_{i,t}(\theta)\,\hat{A}_{i,t},\;\mathrm{clip}\big(r_{i,t}(\theta),1-\epsilon,1+\epsilon\big)\,\hat{A}_{i,t}\Big) \\ -\;\beta\;\mathrm{KL}\big[\pi_\theta \,\|\, \pi_{\text{ref}}\big]\Bigg],\end{aligned} \tag{49}$$

with importance ratio $r_{i,t}(\theta) = \pi_\theta(o_{i,t} \mid q, o_{i,<t}) \,/\, \pi_{\theta_{\text{old}}}(o_{i,t} \mid q, o_{i,<t})$ and *group-normalized advantage*

$$\hat{A}_{i,t} \;=\; \tilde{A}_i \;=\; \frac{R_i \;-\; \mathrm{mean}\big(\{R_1,\dots,R_G\}\big)}{\mathrm{std}\big(\{R_1,\dots,R_G\}\big)}, \tag{50}$$

constant in t across the trajectory (Shao et al., 2024, §4.1, Eq. verified against R1 §2.1 Eq. 3). The KL penalty is computed with Schulman's unbiased estimator, $\mathrm{KL} = \frac{\pi_{\text{ref}}}{\pi_\theta} - \log\frac{\pi_{\text{ref}}}{\pi_\theta} - 1$, evaluated per token (Shao et al., 2024, §4.1).

Symbol table. q is the prompt; G is the group size, typically 8–64; o_i is the i-th sampled trajectory (chain-of-thought + answer); $R_i \in \{0, 1\}$ is the verifier reward; π_{ref} is a frozen reference policy (typically the SFT cold-start, cf. section 7); β is the KL coefficient, typically 0.001–0.04 — *DeepSeek − AI* (2025, §2.1) report $\beta = 0.001$ for R1-Zero;[117] ϵ is the PPO clip range, typically 0.2.

[114] KB excerpt: `kb/excerpts/deepseek-r1.md#sec-2-2-reward`.
[115] KB excerpt: `kb/excerpts/shao2024.md#sec-4-1-motivation`.
[116] KB excerpts: `kb/excerpts/shao2024.md#sec-4-1` and `kb/excerpts/deepseek-r1.md#sec-2-1-grpo`.
[117] KB excerpt: `kb/excerpts/deepseek-r1.md#sec-2-1-hp`.

Memory profile (the load-bearing reason GRPO matters at LLM scale). PPO with GAE (Ouyang et al., 2022, §3.5) requires a value network $V_\phi(s_t)$ co-sized with the policy: at LLM scale this *doubles* the active parameter count and the activation memory of training. GRPO's group-mean baseline replaces V_ϕ entirely; the trade is $G\times$ rollout count per prompt against V_ϕ's parameter and activation footprint, and at sparse reward (where V_ϕ struggles to learn anyway) the trade is favorable (Shao et al., 2024, §4.1).[118] The four co-resident model copies of section 8's PPO infrastructure (policy, reference, reward, value) reduce to two (policy, reference); the verifier is a CPU subprocess, not a model.

Intuition

GRPO is the median-of-rollouts as a policy gradient. Sample G trajectories from the current policy, score each, and ask: which were *better than typical for this prompt*? Up-weight the better-than-mean log-probs; down-weight the worse-than-mean ones. The sign of $\tilde{A}_i$ in eq. (50) answers the question; its magnitude (in standard-deviation units of the group's reward distribution) controls the step size. Returning to canonical form: eq. (50) is the standardization of R_i within its group, and eq. (49) is the PPO-clipped surrogate with that standardized advantage broadcast over the trajectory's tokens.

Per-step procedure. For each batch of B prompts: (1) sample G trajectories per prompt from $\pi_{\theta_{\text{old}}}$ at temperature $T \in [0.7, 1.0]$, capped at $L_{\max}$ tokens (DeepSeek-R1 uses $L_{\max} = 32{,}768$ before step 8.2k and 65,536 afterward (DeepSeek-AI, 2025, §2.1)); (2) score with the verifier; (3) compute $\tilde{A}_i$ via eq. (50) and broadcast token-wise; (4) take $K \in \{1, 4\}$ PPO inner-loop steps on the buffered $(B \cdot G)$ trajectories; (5) refresh $\theta_{\text{old}} \leftarrow \theta$.

11.3 R1 evidence: 15.6% to 77.9% on AIME 2024

DeepSeek-AI (2025) apply GRPO with verifiable rewards to the DeepSeek-V3 base model and report the strongest published evidence to date that pure RL on a deterministic verifier produces frontier-grade reasoning capability. Two models are trained:

- *DeepSeek-R1-Zero.* GRPO applied directly to the base, no SFT cold-start. Reward is rule-based: accuracy (binary correctness from a math/code verifier) plus a small format reward for the `<think>...</think><answer>...</answer>` envelope (DeepSeek-AI, 2025, §2.2).[119]
- *DeepSeek-R1.* A multi-stage pipeline: cold-start SFT on a small set of long-CoT exemplars, then GRPO with the same verifiable rewards (plus a language-consistency reward), then a second SFT on rejection-sampled reasoning traces, then a final RL alignment stage for helpfulness and harmlessness (DeepSeek-AI, 2025, §3).[120]

The training-curve headline (DeepSeek-AI, 2025, §2.3)[121]: "Figure 1(a) depicts the performance trajectory of DeepSeek-R1-Zero on the AIME 2024 benchmark throughout the RL training process, where the average pass@1 score on AIME 2024 shows a significant increase, jumping from an initial 15.6% to 77.9%. In addition, by leveraging the self-consistency decoding, the model's performance can be further improved, achieving an accuracy of 86.7%. This performance significantly surpasses

[118] KB excerpt: `kb/excerpts/shao2024.md#sec-4-1-motivation`.
[119] KB excerpt: `kb/excerpts/deepseek-r1.md#sec-2-2-reward`.
[120] KB excerpt: `kb/excerpts/deepseek-r1.md#sec-3-pipeline`.
[121] KB excerpt: `kb/excerpts/deepseek-r1.md#sec-2-3-aime`.

the average performance across all human competitors." The full run is 10,400 steps, $\sim$1.6 epochs over the training distribution (DeepSeek-AI, 2025, §2.1).

Two structural facts of the trajectory are load-bearing for the section's thesis:

Response length grows during training. The output trace length grows from a few hundred tokens early in training to roughly 10,000 tokens at convergence, accompanied by the emergence of self-reflective patterns (DeepSeek-AI, 2025, §2.3).[122] The verifier reward contains no length term — the policy discovers that long traces win on math and code, on its own. Cross-paper to Paper 3 §4 for the inference-time-compute interpretation of this growth.

The "aha moment." DeepSeek-AI (2025, §2.3) report a discrete jump in the model's use of the word "`wait`" during reflection, characterized as "an 'aha moment', characterized by a sudden increase in the use of the word 'wait' during reflections." [123] The chain-of-thought structure is being actively reorganized during RL, not just used.

R1 vs. R1-Zero — what the cold-start adds. R1-Zero's traces are difficult to read and language-mix between English and Chinese because DeepSeek-V3-Base is multilingual (DeepSeek-AI, 2025, §3).[124] R1's cold-start SFT adds: (a) legibility (single-language English traces with consistent formatting); (b) the `<think>...</think>` envelope as a learned default. R1's reasoning capability matches R1-Zero's; the cold-start is read by the authors as primarily stylistic.

Distillation transfer (cross-link to section 12 and Paper 3 §8). R1 generates 800K reasoning traces (600K reasoning + 200K non-reasoning), and *pure SFT* on this dataset transfers reasoning capability into Qwen-class small models — distilled-32B matches or exceeds OpenAI o1-mini on multiple reasoning benchmarks (DeepSeek-AI, 2025, §4).[125] The capability is expressible as an SFT dataset; small-model trainers do not need to run RL.

11.4 DAPO: four targeted refinements

Seed (2025) ("Decoupled clip and dynamic s**A**mpling **P**olicy **O**ptimization," ByteDance Seed, March 2025) introduce four targeted modifications to GRPO that together cut training-step count to AIME-2024 score 50 by roughly half on Qwen2.5-32B (Seed, 2025, §4).[126] Each modification addresses a measured failure mode of the GRPO recipe.

(i) Clip-Higher (Eq. 10). GRPO's symmetric clip $[1-\epsilon, 1+\epsilon]$ caps probability *increases* on exploration tokens at the same rate as it caps decreases on already-favored tokens. DAPO decouples the bound:

$$\mathcal{J}_{\text{DAPO}}(\theta) \;=\; \mathbb{E}\left[\frac{1}{G}\sum_{i}\min\Big(r_{i,t}(\theta)\,\hat{A}_i,\;\text{clip}\big(r_{i,t}(\theta),1-\epsilon_{\text{low}},1+\epsilon_{\text{high}}\big)\,\hat{A}_i\Big)\right], \tag{51}$$

[122] KB excerpt: `kb/excerpts/deepseek-r1.md#sec-2-3-aha`.
[123] KB excerpt: `kb/excerpts/deepseek-r1.md#sec-2-3-aha`.
[124] KB excerpt: `kb/excerpts/deepseek-r1.md#sec-3-pipeline`.
[125] KB excerpt: `kb/excerpts/deepseek-r1.md#sec-4`.
[126] KB excerpt: `kb/excerpts/dapo2025.md#sec-4-headline`.

with $\epsilon_{\text{low}} = 0.2$ kept at the PPO default and ϵ_{high} enlarged (e.g., 0.28) (Seed, 2025, Eq. 10).[127] The asymmetry permits larger upward updates on under-probable correct tokens — i.e., prevents collapse onto already-favored trajectories.

(ii) Dynamic Sampling (Eq. 11). Whenever all G trajectories in a group share the same reward (all 0 or all 1), the group-normalized advantage eq. (50) is identically zero and the sample contributes no gradient. DAPO enforces

$$0 \;<\; \big|\{\, o_i : \text{is\_equivalent}(a, o_i)\,\}\big| \;<\; G \tag{52}$$

(Seed, 2025, Eq. 11), discarding and re-sampling groups that fail the constraint.[128] Compute is concentrated on informative gradients.

(iii) Token-Level Loss (Eq. 12). GRPO normalizes by $1/|o_i|$ inside the group sum and then by $1/G$ outside, which means longer trajectories contribute proportionally less per token. DAPO replaces this with a token-aggregated normalization:

$$\mathcal{L}_{\text{DAPO}}(\theta) \;=\; -\,\frac{1}{\sum_{i=1}^{G}|o_i|}\sum_{i=1}^{G}\sum_{t=1}^{|o_i|} L_{i,t}(\theta). \tag{53}$$

Every token weighs equally regardless of trajectory length (Seed, 2025, Eq. 12).[129] The change avoids the systematic underweighting of long correct CoTs — which is precisely the regime section 11.3's response-length growth produces.

(iv) Overlong Reward Shaping (Eq. 13). Trajectories that hit $L_{\max}$ are typically penalized as failures, but the failure signal is uninformative — the model just ran out of budget. DAPO replaces the hard zero with a length-based soft penalty:

$$R_{\text{length}}(o_i) \;=\; \begin{cases} 0 & |o_i| \le L_{\text{accept}} \\ \text{linear interp.} & L_{\text{accept}} < |o_i| < L_{\max} \\ -1 & |o_i| \ge L_{\max} \end{cases} \tag{54}$$

(Seed, 2025, Eq. 13).[130] Variance from length-truncated samples falls; the genuine signal that an overlong reasoning trajectory is bad is preserved.

11.5 Why verifiable rewards changed the field

The pre-RLVR post-training stack (section 8, section 9) was constrained by the *reward-hacking ceiling*: as $\text{KL}(\pi_\theta \| \pi_{\text{ref}})$ grew, the proxy–gold gap of r_ϕ widened (Gao et al. 2023, scaling laws for reward-model overoptimization), and the policy began to exploit artifacts of r_ϕ rather than improve along the gold-reward direction. Mitigations (RM ensembles, WARM/WARP, KL-coefficient scheduling, early stopping) bought margin but did not abolish the ceiling. Three structural changes follow from eq. (48):

[127] KB excerpt: `kb/excerpts/dapo2025.md#sec-3-clip-higher`.
[128] KB excerpt: `kb/excerpts/dapo2025.md#sec-3-dynamic-sampling`.
[129] KB excerpt: `kb/excerpts/dapo2025.md#sec-3-token-loss`.
[130] KB excerpt: `kb/excerpts/dapo2025.md#sec-3-overlong`.

1. *The proxy–gold gap closes by construction.* The verifier *is* the gold reward in the verifier-supported domains, modulo dataset-side issues. Reward-overoptimization curves of the r_ϕ-trained-on-pairs kind do not exist in this regime.

2. *Human preference data is not the bottleneck.* A canonical preference dataset costs \$10–\$30 per pair at frontier annotator quality (Ouyang et al., 2022, §3.4); a verifier costs the engineering time to write the parser plus the gold-answer set. The training-data axis decouples from the labeling-budget axis.

3. *Capability transfers through SFT distillation.* section 11.3's 800K-trace distill-into-Qwen result is the 2025 demonstration that capability gained under verifier-driven RL is expressible as an SFT dataset, and the small-model trainer does not need to run RL at all (DeepSeek-AI, 2025, §4).[131] The post-training-stack output of one lab becomes the SFT input of another.

Intuition

Verifiable rewards convert post-training from a preference-elicitation problem to a search problem. The verifier is a pass/fail oracle, not a teacher: RLVR does not tell the policy *how* to be better, only *whether* it is. The optimization works because the policy already produces the correct answer some fraction $p_0 > 0$ of the time at the start of training; RLVR amplifies the trajectories that lead to correct answers and suppresses the rest. Returning to canonical form: the group baseline in eq. (50) is exactly the empirical estimate of $p_0(q)$ for prompt q, and eq. (49)'s gradient becomes a trajectory-level importance-weighted update on the better-than-empirical-mean rollouts.

11.6 Cross-paper anchor: this is also Paper 3's mechanism

This section is also Paper 2's anchor for Paper 3's reasoning triangle. The triangle (Paper 3 §1, §7) has three legs: inference-time compute scaling (chain-of-thought, self-consistency, sampling-based test-time compute), inference-time verifier signals (process reward models, beam / MCTS search, generative PRMs), and *RL on verifiable rewards* — the leg developed here. Paper 3 §7 cites eq. (48), eq. (49), eq. (50), and the four DAPO equations as the canonical statement of the RLVR leg, and develops the inference-time interactions (proposer/verifier, MCTS over reasoning steps, ReST-MCTS* AlphaZero-loop) that this section does not cover. The R1 trajectory of section 11.3 is the empirical proof point that all three legs co-design: the same RLVR run produces both the response-length growth (inference-time compute) and the self-reflection patterns (which interact with verifier signals at inference). Treating RLVR as a training-only mechanism — as this section deliberately does — understates the gains and, more importantly, the open questions; the latter are concentrated in Paper 3.

Contradiction

Does RLVR create new reasoning capability? The framing in DeepSeek-AI (2025) ("incentivizing reasoning capability," the abstract phrasing) and the parallel framings in et al. (AI2) and Team and Alibaba (2025) treat RLVR as eliciting reasoning behaviors that the base model could not produce. various (2025b) (Yue et al. NeurIPS 2025 oral) disagree: by carefully comparing pass@1 vs. pass@K of base-model vs. RLVR'd model at matched sampling

[131] KB excerpt: `kb/excerpts/deepseek-r1.md#sec-4`.

budget, they argue RLVR improves pass@1 (sampling efficiency) on problems the base model can already solve at higher K, but does *not* expand the set of problems solvable at any K. The capability ceiling is set by pre-training; RLVR is sampling-efficient sharpening, not capability expansion. As of 2026-Q2 the open-source consensus leans toward Yue with caveats: the result is benchmark-, temperature-, and sample-budget-conditional, and generalization to non-math (e.g., agentic, long-horizon) RLVR is single-source. Cross-paper to Paper 3 §14 for the resolution attempt.

Contradiction

Cold-start SFT necessity. R1-Zero (no cold-start) reaches AIME 77.9% pass@1 with unreadable traces; R1 (with cold-start) is cleaner (DeepSeek-AI, 2025, §3). Whether the cold-start is capability-load-bearing or only stylistic is open: anecdotal evidence from 2025–2026 reproductions (per the topics-survey sweep recorded in `kb/notes/post-training/rlvr-and-grpo.md#5-4`) shows removing cold-start sometimes destabilizes RL, sometimes works fine; the determining factor is unclear.

Contradiction

VAPO vs. DAPO. VAPO (Value-Augmented PPO, ByteDance, April 2025, arXiv:2504.05118) re-introduces a learned value function under the GRPO/RLVR objective, shared-backbone with the policy and value-loss-clipped, and reports outperforming DAPO on long-CoT tasks in fewer steps — suggesting the GRPO-vs-PPO advantage is not "value models are useless" but "naïve value models are expensive-and-useless; carefully-trained ones are useful." Whether VAPO's gain over DAPO replicates outside the original recipe is open as of 2026-Q2; only the original paper has reported the comparison.

deepen-cite: vapo2025 — bibkey not yet in references.bib; VAPO §3 replication evidence

11.7 Forward link

section 11.2–section 11.4 concludes the preference-and-verifier RL stage of the post-training pipeline. The recipe is now: SFT cold-start (section 7) → optionally a preference-RL stage (section 8 or section 9 for instruction-following and alignment) → RLVR with GRPO (or DAPO) for reasoning capability on verifier-supported domains → rejection-sampling SFT to consolidate. What remains is the weight-space tail of the pipeline: how to take the post-trained checkpoint and adapt it cheaply (LoRA, QLoRA), or compose several post-trained checkpoints into a multi-task model (Soups, Task Vectors, TIES). section 12 walks the adaptation-and-merging machinery that closes the six-stage pipeline of this paper.

12 Adaptation and merging

The previous sections (§7–§11) drove parameter updates with gradients on data: SFT cross-entropy, RLHF/DPO preference signals, RLVR's verifiable rewards. This section closes the pipeline with a different class of operation altogether — *weight-space* updates whose unit of action is the parameter vector itself, not a per-token loss. Parameter-efficient fine-tuning (PEFT) writes a low-rank patch onto a frozen base; model merging averages, signs, and trims weight vectors from fine-tunes that

share an ancestor. Both rest on the same empirical phenomenon — fine-tunes from a shared base land in a single connected loss basin — and both are now production-load-bearing: every open-weight LLM since LLaMA-2 ships with a LoRA adapter ecosystem, and the strongest open multi-task models in 2024–2026 are merged from specialists rather than jointly trained.

12.1 LoRA: low-rank adaptation

Hu et al. (2021) parameterize the update to a frozen pretrained linear-layer weight $W_0 \in \mathbb{R}^{d\times k}$ as a low-rank decomposition[132]:

$$W = W_0 + \Delta W = W_0 + BA, \quad B \in \mathbb{R}^{d\times r},\ A \in \mathbb{R}^{r\times k},\ r \ll \min(d,k) \tag{55}$$

(Hu et al., 2021, §3). W_0 is frozen; only A and B are trained. The forward pass at a hidden state x is

$$h = W_0 x + \Delta W x = W_0 x + (\alpha/r)\, BAx, \tag{56}$$

verbatim from Hu et al. (2021, §3), with scalar α a constant in r (Hu et al., 2021, §3). The backward pass flows gradients through A and B only:

$$\nabla_A \mathcal{L} = (\alpha/r)\, B^\top (\nabla_h \mathcal{L})\, x^\top, \qquad \nabla_B \mathcal{L} = (\alpha/r)\, (\nabla_h \mathcal{L})\, (Ax)^\top, \tag{57}$$

so the optimizer state (Adam m, v) is allocated only over the rank-r factors. The initialization is asymmetric: A Gaussian, B zero, so that $\Delta W = 0$ at step 0 and the model behaves identically to the base (Hu et al., 2021, §3).

The trainable-parameter count drops from $\sum_\ell d_\ell k_\ell$ to $\sum_\ell r(d_\ell + k_\ell)$. With LoRA applied to all attention Q, K, V, O projections and rank $r = 4$, GPT-3 175B's trainable count falls roughly $10^4\times$ (Hu et al., 2021, §3), with GPU memory for fine-tuning down $\sim 3\times$ (Hu et al., 2021, abstract). The empirical fact that *rank $r = 4$ is sufficient for adapting the weight matrices in the GPT-3 175B model* is reported verbatim in Hu et al. (2021, §4), alongside the design choice that adapting Q and V together gives the strongest task transfer.

The deployment-time mechanic is what separates LoRA from earlier adapter families (Houlsby et al., 2019)

deepen-cite: houlsby2019-adapters §3

. Adapter modules add bottleneck layers that incur inference latency at every forward pass; LoRA computes

$$W_{\text{merged}} = W_0 + (\alpha/r)\, BA \tag{58}$$

once at deployment (Hu et al., 2021, §5), and the deployed model has identical inference cost to the base. Different LoRAs can be loaded and unloaded by adding or subtracting $(\alpha/r)BA$ — the inference-time mechanic that S-LoRA and Punica multi-tenant systems exploit (cross-ref Paper 1's serving discussion).

12.2 QLoRA: quantize the base, train the adapter

Dettmers et al. (2023) extend LoRA by quantizing the frozen base W_0 to four bits while keeping the LoRA factors in BF16. The headline (Dettmers et al., 2023, abstract)[133] is a 65B-parameter fine-tune fitting on a single 48 GB GPU at full 16-bit task performance. Three innovations carry the result:

[132] KB excerpt: `kb/excerpts/hu2021-lora.md#sec-3`.
[133] KB excerpt: `kb/excerpts/dettmers2023-qlora.md#abstract`.

NF4 — quantile-based 4-bit format. The *NormalFloat-4* (NF4) format stores 16 bin centers chosen as the quantiles of a standard normal distribution, then assigns each weight to its nearest bin. Pretrained weights are empirically near-Gaussian per block (Dettmers et al., 2023, §3)[134], so NF4's bin spacing is information-theoretically near-optimal in this regime. Concretely: bin centers are

$$q_i = \frac{1}{2}\Big[Q_{\mathcal{N}(0,1)}\Big(\tfrac{i}{16} + \tfrac{1}{2\cdot 16}\Big) + Q_{\mathcal{N}(0,1)}\Big(\tfrac{i+1}{16} + \tfrac{1}{2\cdot 16}\Big)\Big], \quad i \in \{0, \ldots, 15\}, \tag{59}$$

the midpoints of equal-mass quantiles of $\mathcal{N}(0,1)$ (Dettmers et al., 2023, §3), then re-scaled per block. Per-block (block size 64) FP32 scales Z are stored alongside, and the forward GEMM dequantizes just-in-time:

$$y = \text{dequant}(W_0^{\text{NF4}})\, x + (\alpha/r)\, B(Ax), \quad \text{dequant}(W_0^{\text{NF4}}) = Z \cdot \text{NF4\_decode}(W_0^{\text{NF4}}) \tag{60}$$

(Dettmers et al., 2023, §3), with NF4_decode a 16-entry lookup. The GEMM itself runs in BF16.

Double quantization. The per-block FP32 scales are themselves quantized to 8-bit with a second-level scale, dropping the average overhead from $\sim$0.5 bits/parameter to $\sim$0.37 bits (Dettmers et al., 2023, §3)[135]. At 65B parameters the saving is $\sim$1 GB.

Paged optimizers. Optimizer state for the LoRA factors is allocated in NVIDIA unified-memory pages that swap to CPU under GPU pressure (Dettmers et al., 2023, §3)[136]. Long-sequence batches that would OOM the 48 GB GPU instead spill to host RAM at a small bandwidth cost.

QLoRA-tuned models recover within $\sim 1\%$ of full BF16 fine-tunes on 8 instruction datasets and multiple model scales (Dettmers et al., 2023, §4), including 33B and 65B scales that would otherwise be infeasible on commodity hardware.

12.3 Task arithmetic: weight-space steering

Ilharco et al. (2022) reframe a fine-tune as a vector in weight space. Given a pretrained $\theta_0 \in \mathbb{R}^P$ and a fine-tune θ_T on task T, the *task vector* is[137]

$$\tau_T = \theta_T - \theta_0 \in \mathbb{R}^P \tag{61}$$

(Ilharco et al., 2022, §2). Three arithmetic operations on task vectors are well-defined:

$$\theta = \theta_0 + \alpha\, \tau_T \qquad \text{(addition; } \alpha = 1 \text{ recovers } \theta_T\text{)}, \tag{62}$$

$$\theta = \theta_0 - \alpha\, \tau_T \qquad \text{(negation; unlearns task } T\text{)}, \tag{63}$$

$$\theta = \theta_0 + \sum_i \alpha_i\, \tau_{T_i} \qquad \text{(combination; multi-task)} \tag{64}$$

(Ilharco et al., 2022, §2). The empirical claims (Ilharco et al., 2022, §3) are that negating a toxicity-task vector reduces toxic generation while preserving general benchmarks, that multi-task addition can outperform naive multi-task fine-tuning, and that analogy compositions $\tau_A - \tau_B + \tau_C$ produce the expected behavioral shifts.

The mechanism, per Ilharco et al. (2022, §4), is the *linearity of fine-tuning in weight space*: small movements in the basin around θ_0 produce small behavioral changes that sum approximately linearly. We return to that claim — and its load-bearing assumption of basin-sharing — in section 12.6.

[134] KB excerpt: `kb/excerpts/dettmers2023-qlora.md#sec-3-nf4`.
[135] KB excerpt: `kb/excerpts/dettmers2023-qlora.md#sec-3-doublequant`.
[136] KB excerpt: `kb/excerpts/dettmers2023-qlora.md#sec-3-paged`.
[137] KB excerpt: `kb/excerpts/ilharco2022-task-vectors.md#sec-2`.

12.4 Model soups: averaging fine-tunes

Wortsman et al. (2022)[138] address the hyperparameter-sweep waste problem. The conventional recipe trains K fine-tunes with different hyperparameters, picks the best on validation, and discards the rest (Wortsman et al., 2022, §1). The soup recipe averages them. The *uniform soup* of K fine-tunes $\{\theta_k\}_{k=1}^{K}$ from the same base θ_0 is

$$\theta_{\text{uniform}} = \frac{1}{K}\sum_{k=1}^{K}\theta_k \tag{65}$$

(Wortsman et al., 2022, §3). The *greedy soup* sorts checkpoints by validation accuracy and includes each θ_k in a running average $\bar{\theta}$ only if doing so does not decrease validation accuracy:

$$\bar{\theta}_k = \frac{|S_{k-1}|\,\bar{\theta}_{k-1} + \theta_k}{|S_{k-1}| + 1} \quad \text{if } \text{ValAcc}(\bar{\theta}_k) \geq \text{ValAcc}(\bar{\theta}_{k-1}), \tag{66}$$

else skip (Wortsman et al., 2022, §3). Both soups produce a single inference-time model with no ensemble cost. The headline result (Wortsman et al., 2022, §1, §4): averaging fine-tunes from a shared base *often improves accuracy and robustness* over the best single fine-tune, with no inference-time penalty, on CLIP, ALIGN, and ViT-G fine-tuning sweeps. The condition that makes this work — the fine-tunes lying in a single low-error basin — is the linear-mode- connectivity assumption taken up in section 12.6.

12.5 TIES-Merging: trim, elect sign, disjoint-merge

Naive task-vector summation suffers *interference* (Yadav et al., 2023, abstract)[139]: when two fine-tunes update the same coordinate in opposite directions, plain addition cancels both. Yadav et al. (2023) introduce a three-step procedure that removes redundant low-magnitude updates and resolves sign disagreement before averaging.

Given task vectors $\{\tau_i\}_{i=1}^{K}$ from a shared base θ_0:

Step 1 — Trim. Keep the top-k% entries of each task vector by magnitude, zero out the rest:

$$\tilde{\tau}_{i,j} = \begin{cases} \tau_{i,j} & \text{if } |\tau_{i,j}| \in \text{top-}k\% \text{ of } |\tau_i|, \\ 0 & \text{otherwise} \end{cases} \tag{67}$$

(Yadav et al., 2023, §3, step 1). Typical $k = 20\%$; the trim removes *interference due to redundant parameter values* (Yadav et al., 2023, abstract).

Step 2 — Elect sign. For each coordinate j, take the sign with the largest total magnitude across tasks:

$$s_j = \text{sign}\left(\sum_{i=1}^{K}\tilde{\tau}_{i,j}\right) \tag{68}$$

(Yadav et al., 2023, §3, step 2). This is the dominant-direction sign, weighted by magnitude — a magnitude-weighted majority vote.

[138] KB excerpt: `kb/excerpts/wortsman2022-model-soups.md#sec-3`.
[139] KB excerpt: `kb/excerpts/yadav2023-ties-merging.md#sec-3`.

Step 3 — Disjoint merge. For each coordinate, average only the task vectors whose sign matches the elected s_j:

$$\bar{\tau}_j = \frac{1}{|\mathcal{A}_j|} \sum_{i \in \mathcal{A}_j} \tilde{\tau}_{i,j}, \quad \mathcal{A}_j = \{i : \text{sign}(\tilde{\tau}_{i,j}) = s_j\} \tag{69}$$

(Yadav et al., 2023, §3, step 3). The merged model is $\theta_0 + \lambda\,\bar{\tau}$ with scaling λ, typically 1 or tuned on a held-out set. Across vision and NLP benchmarks with up to 11 tasks merged, Yadav et al. (2023, §4) report the ordering TIES > Task Arithmetic > Uniform Soup on multi-task accuracy, with sign-conflict resolution contributing the largest gain and trimming a smaller but consistent improvement.

Intuition

Task vectors are weight-space directions, and their arithmetic is exactly that of vectors in $\mathbb{R}^P$. If $\tau_{\text{summarize-en}}$ is the fine-tune-direction for English summarization and $\tau_{\text{translate-fr}}$ is the fine-tune-direction for an English-to-French translator, then $\theta_0 + \tau_{\text{summarize-en}} + \tau_{\text{translate-fr}}$ empirically produces a French-summarization model (Ilharco et al., 2022, §3) — a behavioral analogy that returns to canonical math via eq. (64): a positive linear combination of task vectors lands at a weight that performs both constituent tasks, modulo the interference TIES is designed to handle. The mechanism TIES adds is that when the two task vectors disagree on the sign of coordinate j — push it + for summarization and − for translation — eq. (69) keeps only the dominant-magnitude sign, so the merge does not silently zero out load-bearing coordinates.

12.6 Linear mode connectivity: why merging works

The shared substrate of sections 12.3 to 12.5 is the empirical claim that fine-tunes of a shared pretrained base land in a *connected* low-loss basin: the line segment

$$\theta(t) = \theta_0 + t\,(\theta_f - \theta_0), \quad t \in [0, 1] \tag{70}$$

stays in low-loss territory. Wortsman et al. (2022, §3) provide the empirical evidence — loss is approximately monotone-low along eq. (70) for fine-tunes from a shared CLIP/ALIGN/ViT-G base, and *logit ensembling and weight averaging produce similar accuracy when the loss landscape is locally flat* (Wortsman et al., 2022, §4) — but they do not prove sufficient conditions. The phenomenon is referred to in the broader literature as *linear mode connectivity* (LMC), with Frankle et al. (2020)

deepen-cite: frankle2020-lmc §3

the canonical reference[140]; the Ilharco et al. (2022, §4) *linearity-of-fine-tuning* attribution refers to the same phenomenon at the task-vector level.

The condition is empirically tight in a load-bearing way. LMC holds for SFT-from-the-same-base; it holds partly for RLHF-from-the-same-base (see §8); it breaks for from-scratch models with different initializations, where averaging produces a near-random model. This explains the *common-ancestor* requirement of every merge method: LoRA's adapter is a delta on a fixed W_0, soup recipes assume the θ_k share θ_0, task arithmetic encodes the assumption in eq. (61)'s subtraction, and TIES requires the ancestor explicitly. Merging models with different pretrained bases is not supported by any of these methods, because the assumed basin isn't there.

[140] LMC was introduced for SGD-trained networks at the same initialization; the fine-tuning case is the operationally relevant specialization for adaptation.

Contradiction

The merge literature reports two patterns that do not yet reconcile. On controlled multi-task benchmarks, merged models often exceed any single ingredient (Yadav et al., 2023; Wortsman et al., 2022, §4). On real-world deployments — the open-weights LLM merging community at frontier scale — merged models routinely under-perform the strongest specialist on its own task, with merge wins concentrated in multi-task generalization rather than peak-task performance. The per-paper benchmarks are largely single-source: TIES (Yadav et al., 2023, §4) and Soups (Wortsman et al., 2022, §4) sweep their own task suites, with no third-party head-to-head at frontier-LLM scale, and no canonical theory predicts when merging produces interference vs. the basin-centering effect predicted by LMC. The *merged-beats-best-ingredient* claim should be read as *task-set-conditional*: it holds on multi-task evaluations of specialists, and is not yet established at the single-task ceiling.

deepen-cite: yadav2023-ties-merging §6

12.7 Forward link

The pipeline closes here. The four sections following SFT (§7) — RLHF (§8), DPO (§9), RLAIF/CAI (§10), RLVR/GRPO (§11) — moved the policy in data-driven directions; section 12 makes the dual move in weight space, with PEFT (sections 12.1 and 12.2) writing low-rank patches onto a frozen base and merging (sections 12.3 to 12.5) recombining fine-tunes that share an ancestor. Both rest on the LMC substrate of section 12.6, the speculative theoretical claim that holds the empirical ceiling. The next section (section 13)

deepen-cite: paper2-§13-cross-ref

fixes the frontier-2026 picture: a single table over disclosed pre-training, optimization, distributed, precision, SFT, preference-RL, reasoning-RL, and merge choices for the open-weights frontier (DeepSeek-V3/R1, Llama-3/4, Qwen-2.5/3, Phi-4, Tülu-3, OLMo-2, Gemma-3, Mistral) and the partial disclosures from the closed labs. The question section 12 leaves open — whether merging will be a load-bearing primitive of the frontier pipeline or a community add-on layered on top of single-base specialists — is one of the open boundaries section 14

deepen-cite: paper2-§14-cross-ref

takes up.

13 Frontier-2026 snapshot: pipeline choices per stage

The body of this paper has fixed each of the six pipeline stages on its own. The remaining empirical question is whether reading *across* the disclosed frontier still recovers a consistent shape, or whether the per-stage canonical lineages decompose into recipe-zoo bias from a handful of labs that read each other's papers. This section answers by tabulation. We list the disclosed frontier models of 2024–2026 against the six pipeline stages and fill each cell with the choice the model's tech report or model card actually reports, marking each undisclosed cell "opaque." The shape that falls out is the load-bearing claim: every disclosed pipeline since 2024 contains all six stages, in the same order, with the same inputs and outputs at each stage boundary; agreement on the architecture of the recipe is broader than agreement on the load-bearing variables; and the disagreements that remain are localized at stage boundaries (token budget, optimizer, TP-yes-vs-no, FP8-vs-BF16-base, DPO-vs-PPO-vs-GRPO mix, distill-from-reasoner yes/no, merge yes/no) rather than at the pipeline's

structure. Section 14 concentrates the boundary-localized disagreements; this section establishes that the structure is what we should expect them to live inside.

13.1 The snapshot table

The columns are the six stages, abbreviated as: **Data** (the pre-training corpus and mixing strategy of section 2); **Scale** (token budget D, the optimizer choice from section 4, and the schedule); **Distrib./Prec.** (the parallelism plan from section 5 and the working precision from section 6); **SFT** (scale and rejection-sampling iteration from section 7); **Pref-RL** (the DPO / PPO / RLAIF / CAI choice from sections 8 to 10); **Reasoning-RL + Adapt** (RLVR / GRPO presence and merge ingredients from sections 11 and 12). Cells marked *opaque* are those where the lab has not publicly disclosed the choice; cells marked with a citation give the choice exactly as the paper or model card reports.

13.2 Where the frontier agrees

Reading the table column-by-column, the agreement is broader than the variance suggests. Six axes are now near-universal across the disclosed frontier.

FineWeb-class web pipeline. Every open-weights frontier corpus we have visibility into uses the curate–ablate–mix loop introduced by FineWeb (et al. , HuggingFace): WARC-trafilatura extraction, language-ID filtering, MinHash 14×8 deduplication *per snapshot* rather than globally (et al. , HuggingFace, §3.4), quality classifier on top. DeepSeek-AI (2024b) explicitly cites the FineWeb-style design (DeepSeek-AI, 2024b, §2), OLMo 2's Dolma / Dolmino lineage (Team, 2025) adopts an open variant, and DCLM-Baseline / RedPajama-V2 / Nemotron-CC are explicit FineWeb-class open releases (section 2). The per-snapshot dedup result is the specific methodological agreement: all of these pipelines deliberately do *not* dedup globally because et al. (HuggingFace, §3.4) showed global dedup degrades downstream quality.

$\sim$ 15T English-web ceiling. Both DeepSeek-AI (2024b) (14.8T) and AI (2024) ($\sim$15T) report training-token totals that converge near the same number. The convergence is not coincidental: it is the diminishing-returns ceiling on high-quality English web text reported by both labs (DeepSeek-AI, 2024b, §4). Beyond the ceiling, labs that need more tokens supplement with synthetic (Phi-4: $\sim$ 80% synthetic by tokens (et al. , Microsoft, §1)) or multilingual (Qwen 3: 36T multilingual (Team and Alibaba, 2025), Llama 4 multimodal $\sim$30T (AI, 2026)). The "$\sim$ 15T" point on the curate side is the new frontier-2026 anchor.

AdamW + cosine-or-WSD. Every disclosed pretrain optimizer is AdamW (section 4); every disclosed schedule is either cosine-decay (Llama-3, Qwen 2.5) or Warmup–Stable–Decay (Liu and et al. , Moonlight team, §3.3) (DeepSeek-V3, OLMo 2 Dolmino, Phi-4 midtrain). Muon (Liu and et al. , Moonlight team) appears at the 16B-MoE Moonlight scale but no 70B+ dense or 600B+ MoE deployment uses it as of 2026-Q2; the $\sim 2\times$ compute- efficiency lift Muon claims is unverified at frontier scale. The schedule choice is therefore where the largest variance lives, with WSD's cooldown phase doubling as the data-mixture-injection vehicle that pairs with section 12's continual / late-stage pre-training (Phi-4 midtrain, OLMo 2 Dolmino, DeepSeek-V3 context-extension).

FSDP / ZeRO-1 + BF16, FP8 increasingly. Every disclosed parallelism plan composes ZeRO-1 (Llama 3, DeepSeek-V3) or full FSDP (OLMo 2, Tülu 3 inherits) along the data-parallel axis (PyTorch, 2024, §2.1.2). Working precision is BF16 across the board with selective FP8 GEMM acceleration where the stack supports it: DeepSeek-V3 ran 14.8T tokens of *pre-training* in fine-grained FP8 with zero irrecoverable loss spikes (DeepSeek-AI, 2024b, abstract); Llama 4 reports FP8 pre-training (AI, 2026); Llama 3 reports FP8 selectively for activations (AI, 2024). The 2026 frontier-precision floor is now FP8 on the GEMM where the hardware supports it, BF16 working / FP32 master elsewhere; sub-FP8 (NVFP4 / BitNet) remains unconfirmed at frontier scale (section 14).

SFT first, preference-RL second. The post-training pipeline order is structurally identical across every disclosed lab: cold-start SFT (Ouyang et al., 2022) producing π^{SFT}, then a preference-RL stage anchored on π^{SFT} via $\beta\,\text{KL}(\pi_\theta \| \pi^{\text{SFT}})$ (section 8.1). The closed-form RL optimum $\pi^*(y \mid x) \propto \pi^{\text{SFT}}(y \mid x)\exp(r/\beta)$ is the load-bearing object *every* preference family inverts or optimizes (sections 8.3 and 9.1); only the source of r varies (human-RLHF (Ouyang et al., 2022), AI-judge (et al. , Anthropic), implicit policy log-ratio (Rafailov et al., 2023), verifier (Shao et al., 2024)).

RLVR-GRPO for the reasoning lines. Where a model has a publicly disclosed reasoning variant, GRPO over $\{0, 1\}$ verifiers is the recipe. DeepSeek-R1 (DeepSeek-AI, 2025), Tülu 3 (et al. , AI2), and Qwen 3 thinking-mode (Team and Alibaba, 2025) all report GRPO with binary verifiers (sympy-equivalence, unit-tests, exact-match) plus a small format reward (section 11.2). DAPO's four targeted improvements (Seed, 2025) (Clip-Higher, Dynamic Sampling, Token-Level Loss, Overlong Reward Shaping) appear in subsequent open-source replications. The closed-lab reasoning models (o-series, Claude extended-thinking, Gemini 2.5) do not disclose the inner-loop algorithm but expose a *thinking-budget* or *reasoning-effort* parameter that is behaviorally consistent with an RLVR-trained reasoner under test-time-compute scaling (cross-link to Paper 3).

13.3 Where the frontier diverges

Six axes show real disagreement, each pinned in the preceding sections and concentrated in section 14.

Token budget D/N. The disclosed split runs from Chinchilla's $D/N \approx 20$ (Hoffmann and et al. , DeepMind, §4.1) to Llama-3-8B's $D/N \approx 1875$ (AI, 2024) — two orders of magnitude on the same axis. The split is now *deployment-conditional* (table 4): small dense models destined for billions of inference queries over-train hard; large MoE models intended for quality-sensitive serving sit closer to Chinchilla. Phi-4 at $\sim$400B synthetic / 14B parameters and DeepSeek-V3 at $D/N_{\text{act}} \approx 400$ are both deliberate and both legal; the open question is the joint $(C_{\text{train}}, C_{\text{infer}}, b, q)$ scaling law section 3.7 flagged.

Pure-Muon vs. AdamW. Only the Moonlight 16B-MoE / 5.7T tokens run (Liu and et al. , Moonlight team) has reported a pure-Muon pre-training at frontier-class scale; the disclosed 70B+ dense and 600B+ MoE runs are all AdamW. Whether the $\sim 2\times$ compute-efficiency lift Muon advertises holds at 70B / dense or 600B / MoE is the original paper's own §1 open question and remains unanswered.

TP-yes-vs-TP-no. Llama 3 (AI, 2024) runs TP 8 inside-node plus PP plus DP plus CP — heavy multi-axis parallelism. DeepSeek-V3 (DeepSeek-AI, 2024b) runs ZeRO-1 + EP 16 + PP 16 DualPipe with *no* TP, and explicitly argues that fine-grained FP8 plus DualPipe and the cross-node all-to-all kernel relieves enough activation pressure that TP becomes unnecessary (DeepSeek-AI, 2024b, §3.2.2). The two recipes are architecture-conditional, not generally substitutable; section 5.7 treats the disagreement.

FP8 base vs. BF16 base. DeepSeek-V3 (DeepSeek-AI, 2024b, §3.3) and Llama 4 (AI, 2026) report FP8 *base* pre-training. Llama 3 uses FP8 selectively (activations-only on FP8 GEMMs; BF16 weights and master-FP32) (AI, 2024). OLMo 2, Phi-4, Qwen 2.5, Gemma 3 remain BF16-base. The DeepSeek-V3 zero-spike 14.8T-token result (DeepSeek-AI, 2024b, abstract) is the strongest single data point we have for frontier-scale FP8 training, but it is single-source until Llama 4 publishes a comparable spike-rate figure (section 14).

SFT scale. The disclosed range is ∼1K (s1 (et al. , Stanford+)) to ∼ 10M (Llama 3 multi-round rejection-sampling (AI, 2024)) — four orders of magnitude. The two camps coexist (section 7.6): small-curated for instruction style and tone (LIMA, s1); large-scale for capability breadth (Llama 3, Tülu 3, Phi-4). Phi-4's ∼80% synthetic-fraction (et al. , Microsoft, §1) is itself a load-bearing divergence vs. Tülu 3's organic-heavier mix (et al. , AI2) that no controlled frontier-scale ablation has yet resolved.

DPO vs. PPO vs. GRPO mix; RLAIF / CAI presence. Llama 3 (AI, 2024), Tülu 3 (et al. , AI2), OLMo 2 (Team, 2025), Qwen 2.5 (Team and Alibaba, 2024), and the Phi-4 line (et al. , Microsoft) all ship DPO (Rafailov et al., 2023) or its variants on the preference-RL leg. DeepMind's Gemma 3 (Team and DeepMind, 2025) is the load-bearing exception: PPO with weight-averaged reward models (WARM (Ramé et al., 2024a) / WARP (Ramé et al., 2024b)) is the disclosed choice. Anthropic's Claude line is the second exception: RLAIF / CAI (et al. , Anthropic) (now C3AI (various, 2025a) / R-CAI (various, 2026)) replaces the preference source itself with an AI-judge-under-a-constitution. DeepSeek-R1 takes a third path: direct GRPO over the SFT base (DeepSeek-AI, 2025) with optional preference-DPO *after* the RLVR phase. The DPO-vs-PPO question at frontier scale is the section 9.6 contradiction; neither has produced a controlled head-to-head at 70B+.

R1-distill yes/no; merge yes/no. DeepSeek-R1 (DeepSeek-AI, 2025) reports 800K SFT samples generated by the RLVR-trained R1, distilled into dense 1.5B / 7B / 32B / 70B targets that nearly match o1-mini on AIME at the 32B size; this is the strongest published evidence that reasoning capability transfers through pure SFT (DeepSeek-AI, 2025, §4). The other disclosed labs do not publicly report a parallel distill-from-reasoner SFT pass on a publicly-available reasoning teacher. On the merge axis, Gemma 3's WARM/WARP (Ramé et al., 2024a,b) is the only disclosed production merge as a load-bearing pipeline component (section 12); Mistral and the open-weights community use MergeKit (et al. , Arcee) extensively post-release but as a community add-on rather than as a stage of the canonical pipeline.

13.4 The disclosure asymmetry

The opaque cells are not random. Table 17 shows the closed labs (Anthropic, OpenAI, DeepMind in part, partly Mistral) disclose only the post-training *family* — RLAIF / CAI for Anthropic,

RLHF-class for OpenAI, PPO+WARM/WARP for DeepMind — and keep the data, scale, optimizer, parallelism, precision, and SFT details opaque. The open-weights labs (DeepSeek, Meta, Alibaba, AI2, Microsoft Research) disclose the upstream stages by paper or model card and treat the post-training stack as a separate publication. This asymmetry is structural: the open-weights labs have a research incentive to disclose the recipe (it is the citation-bearing object) while the closed labs publish system cards that document the *interface* (capabilities, refusal behavior, thinking-budget exposure) without disclosing the optimizer or parallelism choice. For µP at frontier scale (section 3.5), the disclosure asymmetry is one of section 14's open items: independent positive results outside the Microsoft / Cerebras / Graphcore ecosystem at 10B+ scale are scarce, and the Llama 4 (AI, 2026) "MetaP" citation has thin published detail.

Contradiction

Six-stage canonical or recipe-zoo bias? The opening of section 1.2 flagged this question. The table 17 answer is that every disclosed pipeline since 2024 has all six stages in the same order, and where stages are missing it is by design (Tülu 3 has no pre-training because it inherits Llama 3's; Mistral Large does not publicly disclose RLVR; Phi-4 does not run RLVR by choice). What the table cannot distinguish is whether the convergence is genuine pipeline-shape discovery or whether it is recipe-zoo bias: the disclosed labs read each other's papers, hire from each other, and run on overlapping hardware. The two readings are not yet separable, and the question sharpens further at the closed-lab boundary (Anthropic, OpenAI, DeepMind have an unknown number of stages we cannot see). The *stage-shape* convergence is empirically robust where we can see; the *stage-shape canonicality* is open. The section 14 reading is that the convergence *plus* the boundary-localized disagreements both point to a pipeline whose structure is stable but whose load-bearing variables have not converged.

Intuition

A useful mental picture is that the table is a 14×6 matrix of cells where the column-marginals (section 13.2) tell us the structural shape and the row-marginals (section 13.3) tell us each lab's identity. Columns with high agreement (SFT-first, BF16+ working precision, AdamW pretrain optimizer, KL-anchored preference-RL) are the structure of the recipe. Columns with high variance (token budget D/N, FP8-vs-BF16-base, DPO-vs-PPO mix, RLVR yes/no, merge yes/no) are the surface where the next round of empirics will settle. The opaque cells in the closed-lab rows tell us where labs treat the choice as competitive advantage rather than research artifact — the post-training data and reward sources for OpenAI / Anthropic; the parallelism plan and precision for DeepMind. Returning to math: each cell's value is, in the language of the section 1.1 intuition box, a choice of $(\pi^{\text{ref}}, r, \mathcal{D})$ for that stage's KL-constrained reweighting. The table is the lab-by-lab realization of the same recipe template; the disagreements are which point in the template's parameter space the lab chose.

13.5 Forward link

Table 17 establishes the empirical shape: six stages, in order, with localized disagreement at five of the six boundaries (data, scale + optimizer, distrib. + precision, preference-RL family, reasoning-RL + merge) and structural near-universal agreement on the remaining axes (FineWeb-class web pipeline, AdamW, FSDP/ZeRO-1, BF16+ working precision, SFT-first / preference-RL-second / RLVR-for-reasoning ordering, KL-anchored closed-form preference optimum). The next section section 14

concentrates the boundary-localized disagreements and asks, for each, what evidence would close the question: the Kaplan-vs-Chinchilla compute allocation under the inference-cost re-objective, optimizer speedups conditional on training-loss vs. downstream evaluation, TP-yes-vs-TP-no under different MoE *vs.* dense architectures, FP8 / NVFP4 / BitNet production status outside the DeepSeek-V3 single-source result, SFT data quantity-vs-quality across regime boundaries, reward-hacking as fundamental or tractable, DPO-vs-PPO at frontier scale, RLAIF / CAI as internalization or surface mimicry, RLVR as new-capability or sampling-efficient sharpening, merge guarantees under linear-mode-connectivity, the precision × token-count interaction the over-training strategy collides with, and µP strict-or-loose at frontier scale. The closing thesis we will defend in section 14 is anticipated by the table 17 reading itself: the pipeline's open questions are *localized* at stage boundaries, not at the pipeline's structure.

14 Contradictions live and open frontiers

The previous twelve sections each ended with `[CONTRADICTION]` blocks where the literature does *not* yet converge. This section concentrates them. For each of twelve live disagreements we name what each side claims, why the disagreement is real, and what evidence would close it. The closing thesis is structural: the open questions are *localized* at stage boundaries (precision × optimizer; token-count × quantization; preference × verifier), not at the pipeline's six-stage structure, which section 13 shows has converged.

14.1 Pre-training, scaling, optimization

(1) Kaplan-vs-Chinchilla compute allocation. On the methodological question this is settled. Kaplan and et al. (OpenAI, §6) fit $N \propto C^{0.73}$, $D \propto C^{0.27}$ under a cosine-LR schedule decayed to a fixed token horizon, which biased shorter-data runs upward; Hoffmann and et al. (DeepMind, §3, Table 2) re-fit with LR-horizon matching across three independent approaches (training-curves, IsoFLOP, parametric) and recover the $\approx 0.50/0.50$ split with the canonical $D \approx 20N$ at compute-optimal pre-training. The contradiction that remains is the "Chinchilla trap" framing: frontier industry over-trains far past the Chinchilla optimum (Llama-3-8B at $D/N \approx 1875$ (AI, 2024), two orders of magnitude past $D/N \approx 20$). This is not a refutation. It is a different objective — minimize *total lifetime cost* (pre-training plus amortized inference) rather than pre-training-FLOP-optimal loss (Hoffmann and et al. , DeepMind, §3.7)[141]. What *is* open is the joint $(C_{\text{train}}, C_{\text{test}}, b, q)$ scaling law, where C_{test} is inference-time compute (Paper 3) and b, q are precision and mixture quality: documented empirically, not in closed form. Closing evidence would be a multi-axis IsoFLOP sweep at 10B-70B that exposes the trade-off surface; no public lab has reported one.

(2) Optimizer speedups are training-loss-conditional. Lion (sign-momentum, Chen et al. 2023) and Sophia (diagonal-Hessian preconditioning, Liu, Li et al. 2023) reported ∼1.5–2× training-loss speedups at LLM-relevant scales. AdamW closes or reverses the gap on downstream evaluation panels (MMLU, GSM8K, HumanEval, BBH) at matched wall-clock time (Liu and et al. , Moonlight team, §1)[142]. Muon-Moonlight (Liu and et al. , Moonlight team) fits $L_{\text{Muon}} = 2.506\, C^{-0.052}$ vs. $L_{\text{AdamW}} = 2.608\, C^{-0.054}$ for a constant matched-loss compute ratio $\eta \approx 0.52$, and the downstream panel at the 16B-MoE / 5.7T-token Moonlight point is competitive with similarly sized DeepSeek-V3 variants. The disagreement is whether Muon's lift generalizes beyond a single

[141] KB excerpt: `kb/excerpts/hoffmann2022-chinchilla.md#sec-3`.
[142] KB excerpt: `kb/excerpts/muon-moonlight2025.md#sec-1-challenges`.

tech report. As of 2026-Q2 the result is *single-source* on a 16B-MoE run; whether the $\sim 2\times$ constant holds at 70B+ dense and 600B+ MoE is the paper's own §1 open question. Closing evidence would be one independent third-party replication at frontier dense scale plus a downstream panel matched on tokens and wall-clock — the structural lesson (re-verify training-loss claims on downstream surface) is the operational rule until then.

(11) Precision × token-count interaction. et al. (Harvard / Google) extend the Chinchilla form with precision as a third axis and predict, via an effective parameter count $N_{\text{eff}}(N, b_{\text{train}})$, that *over-trained models are harder to quantize*: the more D a model has consumed relative to N, the larger the post-quantization loss penalty at deployment.

deepen-cite: scaling-laws-precision-2024 §4: effective-parameter form N_{eff} and over-training → harder-to-quantize prediction; needed at frontier scale.

This collides directly with the inference-cost-aware over-training strategy of contradiction (1): the same Llama-3-8B that gains from being small-and-token-heavy is, by the Kumar prediction, more difficult to quantize at deployment, partially offsetting the strategy's gain. The industry response has been native low-precision training: pre-train at the precision you will deploy, not above. The DeepSeek-V3 (DeepSeek-AI, 2024b, abstract) zero-spike 14.8T-token FP8 result is the strongest production data point. The full $(D/N) \times b$ surface at frontier scale is incompletely mapped; the specific Kumar prediction is *fitted on small models* and extrapolated. Closing evidence: a controlled $(D/N \in \{20, 200, 2000\}) \times (b \in \{\text{BF16, FP8, FP4}\})$ sweep at 7B–70B with both training stability and post-quantization downstream loss reported.

(12) µP at frontier scale. Yang and et al. (Microsoft) prove, under Definition 4.1's strict scaling rules, that the master HPs (LR, betas, schedule shape) transfer losslessly across width. Microsoft (Phi-4 (et al. , Microsoft)), Cerebras, and Graphcore (u-µP, Graphcore, 2024) disclose strict-or-near-strict use; Meta (Llama 4 (AI, 2026)) cites a "MetaP" scheme for cross-scale HP transfer with thin published detail; Anthropic, OpenAI, Google do not disclose. Independent positive results outside the Microsoft/Cerebras/Graphcore ecosystem at 10B+ scale are scarce. The community working assumption is that frontier labs apply the µP LR scalings while keeping Vaswani's $1/\sqrt{d_k}$ attention rather than µP's $1/d$, making them "µP-inspired" rather than strict-Definition-4.1 variants. Whether the loose variants retain the unique-feature-learning guarantee is open. Closing evidence: an open-weights frontier model disclosing strict Definition-4.1 µP with width-sweep HP transfer evidence at $\geq$ 70B parameters.

14.2 Distributed training and precision

(3) TP-yes-vs-TP-no. AI (2024) train Llama 3 405B with heavy TP+PP+DP — the prototypical Megatron-3D shape. DeepSeek-AI (2024b) train DeepSeek-V3 671B-A37B *without TP at all*, claiming the combination of ZeRO-1 DP + EP + DualPipe + FP8 supplies enough activation relief to make TP's per-linear collective cost a net loss. This is a real architectural-conditional disagreement, not a wash. TP costs $2L$ inter-GPU all-reduces per step; EP costs $2L_{\text{MoE}}$ inter-node all-to-alls per step. DeepSeek's fine-grained MoE forces them to pay the EP all-to-all already (DeepSeek-AI, 2024b, §3.2), and pairing it with DualPipe overlap plus the 4-node-cap kernel hides the cost. Llama 3 is dense at active-parameter level; without an EP all-to-all to fold TP work into, the Megatron-3D plan stays preferable. The general rule is *which* parallelism is best is a function of the model graph (MoE vs. dense, expert count, expert size, attention head count) — not a universal answer. Closing evidence is a plan-from-graph optimizer that picks (d, t, c, p, e) from $(N, D, \text{cluster}, \text{graph})$; PyTorch

(2024, §1) explicitly punt on this, GSPMD propagates rather than searches, and AlpaDiscovery is not dominant in 2026 open practice.

(4) FP8 / NVFP4 / BitNet production status. Three precision regimes, three different statuses. *FP8*: the DeepSeek-V3 zero-spike 14.8T-token result (DeepSeek-AI, 2024b, abstract) is the strongest production data point on record; Llama 4 (AI, 2026) likewise discloses FP8 pre-training. The contradiction is that the FP8 result remains *single-source* on its specific recipe (per-token-per-128-channel activation tile, per-128×128 weight block, all-E4M3, $N_C = 128$ promotion) and no other major lab has publicly matched the envelope. *FP4 / NVFP4*: Blackwell hardware ships FP4 Tensor Cores with MXScale, several Blackwell-deploying vendors are running FP4 pilots, but as of 2026-Q2 no major lab has publicly confirmed FP4 *pre-training* of a frontier model. The Kumar effective-parameter prediction warns of substantial loss below ∼7 bits and is fitted on small models. *BitNet 1.58 ternary*: et al. (Microsoft) claim ternary $\{-1, 0, 1\}$ matches FP16 quality, with the BitNet b1.58 2B4T tech report (et al. , Microsoft) as a follow-up, but independent reproductions thin past ∼7B parameters and no frontier-scale companion has appeared. Treat FP8 as production-stable contingent on recipe transfer; treat sub-FP8 as open. Closing evidence: a non-DeepSeek frontier-scale FP8 zero-spike replication; a frontier-scale FP4 pre-training disclosure; a ≥30B BitNet replication outside the original authors.

14.3 Post-training: SFT, preference-RL, reasoning-RL

(5) SFT data quantity-vs-quality. Two empirical traditions. *"Few high-quality samples suffice"*: LIMA

deepen-cite: lima-2023 §results — bibkey not yet in references.bib

and s1 (et al. , Stanford+) report 1K–10K curated examples eliciting instruction-following from a strong base. *"Scale with quality gates"*: Tülu-3 (et al. , AI2), Llama-3 (AI, 2024), Phi-4 (et al. , Microsoft) run 10^6+-sample SFT mixes with rejection-sampling and reward-model filtering, reporting clean monotonic gains. Both are likely correct in different regimes: small-curated for *instruction style and tone*, large-set for *capability breadth* (math, code, multilingual, refusal, safety). The regime boundary has not been mapped at frontier scale. Closing evidence: a controlled (data scale) × (evaluation axis) sweep on a fixed strong base, separating style/tone from capability.

(6) Reward-hacking is fundamental or tractable. *Tractable view* (Coste 2023; Ramé et al., 2024a,b; Gemma 3 production (Team and DeepMind, 2025)): RM ensembles, weight-averaged RMs, KL scheduling, and on-policy preference iteration mitigate reward hacking enough for production. *Fundamental view* (the direct-alignment scaling-laws line

deepen-cite: rosset2024-direct-alignment-scaling arXiv:2406.02900 §3 — bibkey not yet in references.bib

and the persistence of sycophancy as a localized shortcut at all scales, Sharma et al., 2023): proxy-reward optimization produces overoptimization across *all* preference-based post-training, including DPO, since DPO's implicit reward $\hat{r}_\theta = \beta \log \pi_\theta / \pi^{\text{ref}}$ is itself a learned proxy that can be hacked. Closing evidence: a controlled RM-quality × KL-budget × pipeline (PPO/DPO/GRPO) sweep with both proxy reward and gold reward measured, plus an adversarial sycophancy-probe panel.

(7) DPO-vs-PPO at frontier scale. Rafailov et al. (2023) establishes that DPO and PPO-RLHF share the *same* fixed point on the same preference data, with DPO reaching it without sampling, RM, or RL loop. The empirical disagreement is whether the shared fixed point is reached equally well in

practice. *Llama-3 / Tülu-3 view* (AI, 2024; et al. , AI2): with iterated on-policy preference collection, DPO matches or exceeds PPO on standard alignment benchmarks at lower compute. *DeepMind / Gemma-3 view* (Team and DeepMind, 2025; Ramé et al., 2024a,b): PPO + WARM/WARP-averaged reward ensembles remains the production choice for general alignment, the ensembled RM recovering the proxy-vs-gold gap that single-RM PPO suffered. The disagreement is partly recipe-specific (different RMs, preference datasets, KL schedules) and partly task-specific (DPO more competitive on instruction-following; PPO-with-ensemble retains advantage on creative / open-ended generation per anecdotal reports). No controlled head-to-head at frontier scale has been published. The SimPO length-bias fix (Meng et al., 2024) adjusts loss form without resolving the head-to-head. Closing evidence: a fixed-base × fixed-preference-data sweep over $\{\text{DPO}, \text{PPO+WARM}\}$ × KL schedule × on-policy iteration depth, with held-out alignment + creative panels.

(8) RLAIF / CAI: internalization or surface mimicry; CAI vs. sycophancy. et al. (Anthropic) establish CAI as "RLHF with a different labeler": AI-judge under randomly-chosen principle c_k replaces human preferences, BT-RM trained on AI labels feeds PPO with KL anchor to $\pi^{\text{SL-CAI}}$. CAI's behavioral signature is non-evasion. Two open splits. First, internalization vs. surface mimicry: *genuine principle internalization* and *learned text shaped like constitutional reasoning* are behaviorally indistinguishable on standard evaluations; only adversarial probes where the principle's intent and its surface verbalization disagree can separate them. Second, CAI vs. sycophancy: the AI judge rewards principle-*aligned* responses, so the policy can learn to *appear* principle-aligned to the judge whether the underlying reasoning is shallow or deep — sycophancy with respect to the AI judge instead of the human labeler (Sharma et al., 2023). CAI may not solve sycophancy and may amplify it, since the algorithmic judge is more consistent than human labelers and the policy can find its biases more reliably. Closing evidence: held-out adversarial probe panels (intent-vs-verbalization, principle-conflict, sycophancy with counter-evidence) reported jointly for SFT-only, RLHF, RLAIF, and CAI baselines on a fixed base.

Contradiction

The proxy-reward thread. Contradictions (6), (7), and (8) are not independent. They are three readings of one underlying question: *is post-training fundamentally a proxy-reward optimization, and if so, does the proxy-vs-gold gap close or persist as the proxy improves?* The fundamental view answers "persists — the gap is a structural property of any learned reward signal, including DPO's implicit reward and CAI's AI judge." The tractable view answers "closes to acceptable levels — ensembling, KL anchoring, on-policy iteration, and judge-under-constitution together push the residual below the deployment threshold." The empirical signature that would distinguish them is whether the gap *shrinks* or *stabilizes* as proxy quality is varied along a controlled axis (RM ensemble size, judge capability, KL budget). We do not yet have the controlled axis-sweep at frontier scale. The section 14.3 contradictions are individually tractable; the underlying question — is reward-hacking fundamental or engineering — is not.

(9) Does RLVR create new reasoning capability? DeepSeek-AI (2025) frame RLVR as "incentivizing reasoning capability" (R1-Zero from 15.6% to 77.9% on AIME 2024 over GRPO training, the "aha-moment" `wait`-token spike), with parallel framings in et al. (AI2) and Team and Alibaba (2025) treating RLVR as eliciting reasoning behaviors the base model could not produce. various (2025b) (Yue et al. NeurIPS 2025 oral) disagree: by carefully comparing pass@1 vs. pass@K

of base-model vs. RLVR'd model at matched sampling budget, RLVR improves pass@1 (sampling efficiency) on problems the base model can already solve at higher K, but does *not* expand the set of problems solvable at any K. The capability ceiling is set by pre-training; RLVR is sampling-efficient sharpening. As of 2026-Q2 the open-source consensus leans toward Yue with caveats: the result is benchmark-, temperature-, and sample-budget-conditional, and generalization to non-math (agentic, long-horizon) RLVR is single-source. Closing evidence is concentrated in Paper 3, which treats RLVR alongside inference-time-compute scaling and search; the $(C_{\text{train,RLVR}}) \times (C_{\text{test}})$ joint surface is where the framings might reconcile.

14.4 Adaptation and merging

(10) Model-merging guarantees. The merge literature reports two patterns that do not yet reconcile. On controlled multi-task benchmarks, merged models often exceed any single ingredient (Wortsman et al., 2022, §4; Yadav et al., 2023, §4). On real-world deployments at the open-weights LLM merging community's frontier scale, merged models routinely under-perform the strongest specialist on its own task, with merge wins concentrated in multi-task generalization rather than peak-task performance. The per-paper benchmarks are largely single-source: TIES and Soups sweep their own task suites with no third-party head-to-head at frontier-LLM scale. The speculative theoretical underpinning is linear-mode-connectivity — the basin-sharing hypothesis that fine-tuned models from a common base lie in a connected low-loss region whose center is the average (Ilharco et al., 2022; Wortsman et al., 2022). LMC holds empirically for SFT-from-shared-base, partly for RLHF, and breaks for from-scratch different-init runs; no canonical theory predicts when merging produces interference vs. basin-centering. The *merged-beats-best-ingredient* claim should be read as *task-set-conditional*: it holds on multi-task evaluations of specialists, and is not yet established at the single-task ceiling. Closing evidence: a third-party head-to-head at $\geq$30B with matched training data, isolating sign-conflict resolution (TIES (Yadav et al., 2023)) from drop-and-rescale (DARE) from greedy-soup ingredient selection.

14.5 Closing thesis: structure stable, boundaries open

The twelve contradictions above admit a structural reading. None of them disputes the *shape* of the pipeline. Every disclosed 2024–2026 frontier pipeline (section 13) executes the same six stages in the same order: pre-training-data → scaling-and-optimization → distributed-and-precision scaffolding → SFT → preference-RL → reasoning-RL / adaptation. What the contradictions dispute is what each stage should do internally and how stages should be *coupled at their boundaries.*

Intuition

The contradictions cluster at three boundary couplings: **precision** × **optimizer** (Muon update RMS exceeds BF16 high-precision range without weight decay; FP8 stability requires the per-tile scaling DeepSeek-V3 introduced; (2)∩(4) live here), **token-count** × **quantization** (over-trained models are harder to quantize, but inference economics rewards over-training; (1)∩(11) live here), and **preference** × **verifier** (proxy reward from a learned judge can be hacked; verifier reward is gold but sparse and domain-bound; (6)∩(7)∩(8)∩(9) live here). Returning to canonical form: the pipeline factorizes structurally as a chain of stages $S_1 \to S_2 \to \cdots \to S_6$, but the load-bearing *interaction terms* $\partial S_i / \partial S_j$ for adjacent stages have not been characterized. The contradictions are reading those missing interaction terms.

This is the closing claim of Paper 2. The shape of frontier post-2024 LLM training is stable: six

stages, canonical lineages, broadly disclosed at the architecture level. The 2027-class results that close the open contradictions will not be about pipeline shape but about load-bearing variables *inside* each stage and at the boundaries *between* adjacent stages: D/N under joint inference-cost and quantization constraints; optimizer choice under joint downstream and frontier-stability constraints; TP-yes-vs-no under joint model-graph and topology constraints; DPO-vs-PPO under joint proxy-reward and on-policy constraints; RLVR's sampling-efficiency-vs-capability split under joint pre-training and inference-time-compute budgets. Where Paper 1 established the architectural axes and Paper 3 will establish the inference-time-compute axes, Paper 2's claim is that the *training* axes are now legible and the open boundaries between them are localized, named, and falsifiable. The pipeline is a real object; we now know which of its joints flex.

References

Meta AI. The llama 3 herd of models. Meta Tech Report, 2024. URL https://arxiv.org/abs/2407.21783.

Meta AI. The llama 4 herd: Architecture, training, evaluation, and deployment notes. Meta Tech Report, 2026. URL https://arxiv.org/abs/2601.11659.

Ainslie, Lee-Thorp, de Jong, Zemlyanskiy, Lebrón, and Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. In *EMNLP*, 2023. URL https://arxiv.org/abs/2305.13245. KB excerpt: kb/excerpts/ainslie2023.md.

Brown, Mann, Ryder, and et al. Language models are few-shot learners. In *NeurIPS*, 2020. URL https://arxiv.org/abs/2005.14165.

Chowdhery, Narang, Devlin, and et al. Palm: Scaling language modeling with pathways. JMLR, 2022. URL https://arxiv.org/abs/2204.02311.

Christiano, Leike, Brown, Martic, Legg, and Amodei. Deep reinforcement learning from human preferences. In *NeurIPS*, 2017. URL https://arxiv.org/abs/1706.03741.

Chung, Hou, Longpre, Zoph, Tay, Fedus, Li, Wang, Dehghani, Brahma, Webson, Gu, Dai, Suzgun, Chen, Chowdhery, Castro-Ros, Pellat, Robinson, Valter, Narang, Mishra, Yu, Zhao, Huang, Dai, Yu, Petrov, Chi, Dean, Devlin, Roberts, Zhou, Le, and Wei. Scaling instruction-finetuned language models (flan-t5). JMLR / arXiv, 2022. URL https://arxiv.org/abs/2210.11416.

Google DeepMind. Gemini 2.5 technical report. arXiv (DeepMind Tech Report), 2025. URL https://arxiv.org/abs/2507.06261.

DeepSeek-AI. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. arXiv (DeepSeek Tech Report), 2024a. URL https://arxiv.org/abs/2405.04434. KB excerpt: kb/excerpts/deepseek-v2.md.

DeepSeek-AI. Deepseek-v3 technical report. arXiv (DeepSeek Tech Report), 2024b. URL https://arxiv.org/abs/2412.19437. KB excerpt: kb/excerpts/deepseek-v3-training.md.

DeepSeek-AI. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. arXiv (Nature 2025), 2025. URL https://arxiv.org/abs/2501.12948. KB excerpt: kb/excerpts/deepseek-r1.md.

Dettmers, Pagnoni, Holtzman, and Zettlemoyer. Qlora: Efficient finetuning of quantized llms. In *NeurIPS 2023*, 2023. URL https://arxiv.org/abs/2305.14314. KB excerpt: kb/excerpts/dettmers2023-qlora.md.

Liu et al. Data mixing laws: Optimizing data mixtures by predicting language modeling performance. In *ICLR 2025*, 2024. URL https://arxiv.org/abs/2403.16952. KB excerpt: kb/excerpts/data-mixing-laws-2024.md.

Wang et al. (AI2). Tülu 3: Pushing frontiers in open language model post-training. AI2 Tech Report, 2024. URL https://arxiv.org/abs/2411.15124.

Bai et al. (Anthropic). Constitutional ai: Harmlessness from ai feedback. Anthropic Tech Report, 2022. URL https://arxiv.org/abs/2212.08073. KB excerpt: kb/excerpts/bai2022-cai.md.

Goddard et al. (Arcee). Arcee mergekit: A toolkit for merging large language models. arXiv, 2024. URL https://arxiv.org/abs/2403.13257.

Kumar et al. (Harvard / Google). Scaling laws for precision. In *ICLR 2025*, 2024. URL https://arxiv.org/abs/2411.04330.

Penedo et al. (HuggingFace). Fineweb: Decanting the web for the finest text data at scale. arXiv, 2024. URL https://arxiv.org/abs/2406.17557. KB excerpt: kb/excerpts/fineweb2024.md.

Kalamkar et al. (Intel). A study of bfloat16 for deep learning training. arXiv (Intel), 2019. URL https://arxiv.org/abs/1905.12322. KB excerpt: kb/excerpts/kalamkar2019-bfloat16.md.

Abdin et al. (Microsoft). Phi-4 technical report. Microsoft Tech Report, 2024a. URL https://arxiv.org/abs/2412.08905. KB excerpt: kb/excerpts/phi4.md.

Ma et al. (Microsoft). The era of 1-bit llms: All large language models are in 1.58 bits. arXiv, 2024b. URL https://arxiv.org/abs/2402.17764. KB excerpt: kb/excerpts/ma2024-bitnet.md.

Ma et al. (Microsoft). Bitnet b1.58 2b4t technical report. Microsoft Tech Report, 2025. URL https://arxiv.org/abs/2504.12285.

Muennighoff et al. (Stanford+). s1: Simple test-time scaling. arXiv, 2025. URL https://arxiv.org/abs/2501.19393. KB excerpt: kb/excerpts/s1-2025.md.

Frankle, Dziugaite, Roy, and Carbin. Linear mode connectivity and the lottery ticket hypothesis. In *ICML 2020*, 2020. URL https://arxiv.org/abs/1912.05671.

Graphcore. u-p: The unit-scaled maximal update parametrization. In *ICLR 2025 Spotlight*, 2024. URL https://arxiv.org/abs/2407.17465.

Harlap, Narayanan, Phanishayee, Seshadri, Devanur, Ganger, and Gibbons. Pipedream: Fast and efficient pipeline parallel dnn training. arXiv:1806.03377, 2018. URL https://arxiv.org/abs/1806.03377.

Hoffmann and Borgeaud et al. (DeepMind). Training compute-optimal large language models. arXiv, 2022. URL https://arxiv.org/abs/2203.15556. KB excerpt: kb/excerpts/hoffmann2022-chinchilla.md.

Houlsby, Giurgiu, Jastrzebski, Morrone, de Laroussilhe, Gesmundo, Attariyan, and Gelly. Parameter-efficient transfer learning for nlp. In *ICML 2019*, 2019. URL https://arxiv.org/abs/1902.00751.

Hu, Shen, Wallis, Allen-Zhu, Li, Wang, Wang, and Chen. Lora: Low-rank adaptation of large language models. In *ICLR 2022*, 2021. URL https://arxiv.org/abs/2106.09685. KB excerpt: kb/excerpts/hu2021-lora.md.

Ilharco, Ribeiro, Wortsman, Gururangan, Schmidt, Hajishirzi, and Farhadi. Editing models with task arithmetic. In *ICLR 2023*, 2022. URL https://arxiv.org/abs/2212.04089. KB excerpt: kb/excerpts/ilharco2022-task-vectors.md.

Kaplan and McCandlish et al. (OpenAI). Scaling laws for neural language models. arXiv, 2020. URL https://arxiv.org/abs/2001.08361. KB excerpt: kb/excerpts/kaplan2020.md.

Liu and Su et al. (Moonlight team). Muon is scalable for llm training. arXiv, 2025. URL https://arxiv.org/abs/2502.16982. KB excerpt: kb/excerpts/muon-moonlight2025.md.

Meng, Xia, and Chen (Princeton). Simpo: Simple preference optimization with a reference-free reward. In *NeurIPS*, 2024. URL https://arxiv.org/abs/2405.14734. KB excerpt: kb/excerpts/meng2024-simpo.md.

Micikevicius, Narang, Alben, Diamos, Elsen, Garcia, Ginsburg, Houston, Kuchaiev, Venkatesh, and Wu. Mixed precision training. In *ICLR 2018*, 2017. URL https://arxiv.org/abs/1710.03740. KB excerpt: kb/excerpts/micikevicius2017-mixed-precision.md.

Narayanan, Shoeybi, Casper, LeGresley, Patwary, Korthikanti, Vainbrand, Kashinkunti, Bernauer, Catanzaro, Phanishayee, and Zaharia. Efficient large-scale language model training on gpu clusters using megatron-lm. SC21, 2021. URL https://arxiv.org/abs/2104.04473.

Ouyang, Wu, Jiang, Almeida, Wainwright, Mishkin, Zhang, Agarwal, Slama, Ray, Schulman, Hilton, Kelton, Miller, Simens, Askell, Welinder, Christiano, Leike, and Lowe. Training language models to follow instructions with human feedback. In *NeurIPS*, 2022. URL https://arxiv.org/abs/2203.02155. KB excerpt: kb/excerpts/ouyang2022-instructgpt.md.

Meta / PyTorch. Torchtitan: One-stop pytorch native solution for production-ready llm pre-training. In *ICLR 2025*, 2024. URL https://arxiv.org/abs/2410.06511. KB excerpt: kb/excerpts/torchtitan2024.md.

Rafailov, Sharma, Mitchell, Ermon, Manning, and Finn. Direct preference optimization: Your language model is secretly a reward model. In *NeurIPS*, 2023. URL https://arxiv.org/abs/2305.18290. KB excerpt: kb/excerpts/rafailov2023-dpo.md.

Rajbhandari, Rasley, Ruwase, and He. Zero: Memory optimizations toward training trillion parameter models. SC20, 2020. URL https://arxiv.org/abs/1910.02054.

Ramé, Couairon, Shukor, Thom, Pendse, Adda, Vincent, Mary, Cord, Aubert, Cordier, Joulin, Jégou, and Dehghani. Warm: On the benefits of weight averaged reward models. In *ICML 2024*, 2024a. URL https://arxiv.org/abs/2401.12187.

Ramé, Ferret, Vieillard, Dadashi, Hussenot, Cedoz, Sessa, Pernias, Geist, and Bachem. Warp: On the benefits of weight averaged rewarded policies. arXiv (DeepMind), 2024b. URL https://arxiv.org/abs/2406.16768.

Schulman, Wolski, Dhariwal, Radford, and Klimov. Proximal policy optimization algorithms. arXiv, 2017. URL https://arxiv.org/abs/1707.06347.

ByteDance Seed. Dapo: An open-source llm reinforcement learning system at scale. arXiv, 2025. URL https://arxiv.org/abs/2503.14476. KB excerpt: kb/excerpts/dapo2025.md.

Shao, Wang, Zhu, and et al. (DeepSeek-AI). Deepseekmath: Pushing the limits of mathematical reasoning in open language models. arXiv (DeepSeek Tech Report), 2024. URL https://arxiv.org/abs/2402.03300. KB excerpt: kb/excerpts/shao2024.md.

Sharma, Tong, Korbak, Duvenaud, Askell, Bowman, Cheng, Durmus, Hatfield-Dodds, Johnston, Kravec, Maxwell, McCandlish, Ndousse, Rausch, Schiefer, Yan, Zhang, and Perez. Towards understanding sycophancy in language models. In *ICLR 2024*, 2023. URL https://arxiv.org/abs/2310.13548. KB excerpt: kb/excerpts/sharma2023-sycophancy.md.

Shoeybi, Patwary, Puri, LeGresley, Casper, and Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism. arXiv (NVIDIA), 2019. URL https://arxiv.org/abs/1909.08053.

Stiennon, Ouyang, Wu, Ziegler, Lowe, Voss, Radford, Amodei, and Christiano. Learning to summarize from human feedback. In *NeurIPS*, 2020. URL https://arxiv.org/abs/2009.01325.

AI2 OLMo Team. Olmo 2 furious. COLM 2025, 2025. URL https://arxiv.org/abs/2501.00656.

Gemma Team and Google DeepMind. Gemma 3 technical report. arXiv (DeepMind Tech Report), 2025. URL https://arxiv.org/abs/2503.19786.

Qwen Team and Alibaba. Qwen2.5 technical report. arXiv (Qwen Tech Report), 2024. URL https://arxiv.org/abs/2412.15115.

Qwen Team and Alibaba. Qwen3 technical report. arXiv (Qwen Tech Report), 2025. URL https://arxiv.org/abs/2505.09388.

various. C3ai: Crafting and evaluating constitutions for constitutional ai. arXiv, 2025a. URL https://arxiv.org/abs/2502.15861.

various. Does reinforcement learning really incentivize reasoning capacity in llms beyond the base model? In *NeurIPS 2025 (oral)*, 2025b. URL https://arxiv.org/abs/2504.13837.

various. Reverse constitutional ai: Controllable toxic data generation. arXiv, 2026. URL https://arxiv.org/abs/2604.17769.

Vaswani, Shazeer, Parmar, Uszkoreit, Jones, Gomez, Kaiser, and Polosukhin. Attention is all you need. In *NeurIPS*, 2017. URL https://arxiv.org/abs/1706.03762. KB excerpt: kb/excerpts/vaswani2017.md.

Wei, Bosma, Zhao, Guu, Yu, Lester, Du, Dai, and Le. Finetuned language models are zero-shot learners (flan). In *ICLR 2022*, 2021. URL https://arxiv.org/abs/2109.01652.

Wortsman, Ilharco, Gadre, Roelofs, Gontijo-Lopes, Morcos, Namkoong, Farhadi, Carmon, Kornblith, and Schmidt. Model soups: averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. In *ICML 2022*, 2022. URL https://arxiv.org/abs/2203.05482. KB excerpt: kb/excerpts/wortsman2022-model-soups.md.

Xu, Jiang, Niu, Deng, Lin, Poovendran, Choi, and Lin. Magpie: Alignment data synthesis from scratch by prompting aligned llms with nothing. In *ICLR 2025*, 2024. URL https://arxiv.org/abs/2406.08464.

Yadav, Tam, Choshen, Raffel, and Bansal. Ties-merging: Resolving interference when merging models. In *NeurIPS 2023*, 2023. URL https://arxiv.org/abs/2306.01708. KB excerpt: kb/excerpts/yadav2023-ties-merging.md.

Yang and Hu et al. (Microsoft). Tensor programs v: Tuning large neural networks via zero-shot hyperparameter transfer. arXiv, 2022. URL https://arxiv.org/abs/2203.03466. KB excerpt: kb/excerpts/yang2022-mup.md.

Zhao, Gu, Varma, Luo, Huang, Xu, Wright, Shojanazeri, Ott, Shleifer, Desmaison, Balioglu, Damania, Nguyen, Chauhan, Hao, Mathews, and Li. Pytorch fsdp: Experiences on scaling fully sharded data parallel. VLDB 2023, 2023. URL https://arxiv.org/abs/2304.11277.

Model
DeepSeek-V3 (DeepSeek-AI, 2024b)
DeepSeek-R1 (DeepSeek-AI, 2025)
Llama 3 405B / 70B / 8B (AI, 2024)
Llama 4 (AI, 2026)
Qwen 2.5 (Team and Alibaba, 2024)
Qwen 3 (Team and Alibaba, 2025)
Phi-4 (et al. , Microsoft)